

ระบบแดชบอร์ดและแชตบอตปัญญาประดิษฐ์เพื่อติดตามและสรุปสถานะคดีส่งตรวจพิสูจน์ DNA

Dashboard System and AI Chatbot for Monitoring and Summarizing DNA Forensic Case Submissions

โดย

นายวงศธร อินคง รหัสนักศึกษา B6621834

อาจารย์ที่ปรึกษา

[รอใส่: ชื่ออาจารย์ที่ปรึกษา]

รายงานเล่มนี้เป็นส่วนหนึ่งของรายวิชา [รอใส่: รหัสและชื่อวิชา]

สำนักวิชาวิทยาศาสตร์ สาขาวิชาวิทยาศาสตร์การแพทย์

มหาวิทยาลัยเทคโนโลยีสุรนารี

ประจำภาคการศึกษาที่ [รอใส่: ภาคการศึกษา] ปีการศึกษา [รอใส่: ปีการศึกษา]

ชื่อโครงงาน ระบบแดชบอร์ดและแชตบอตปัญญาประดิษฐ์เพื่อติดตามและสรุปสถานะคดีส่งตรวจพิสูจน์ DNA (Dashboard System and AI Chatbot for Monitoring and Summarizing DNA Forensic Case Submissions)

ผู้จัดทำ นายวงศธร อินคง รหัสนักศึกษา B6621834

อาจารย์ที่ปรึกษา [รอใส่: ชื่ออาจารย์ที่ปรึกษา]

ภาคการศึกษา [รอใส่: ภาคการศึกษา]/[รอใส่: ปีการศึกษา]

บทคัดย่อ

[รอใส่: บทคัดย่อภาษาไทย — ยังไม่มีไฟล์ thesis/drafts/00-บทคัดย่อ.md ให้ thesis-writer เขียนหลังบทที่ 4–5 เสร็จ (ความยาวไม่เกิน 1 หน้า)]

Project Title	Dashboard System and AI Chatbot for Monitoring and Summarizing DNA Forensic Case Submissions
Student	[รอใส่: ชื่อ-นามสกุลภาษาอังกฤษ] Student ID B6621834
Advisor	[รอใส่: Advisor name]
Semester	[รอใส่: ภาคการศึกษา]/[รอใส่: ปีการศึกษา]

ABSTRACT

[รอใส่: English abstract — thesis-editor จะแปลจากบทคัดย่อภาษาไทยเมื่อฉบับไทยเสร็จ]

กิตติกรรมประกาศ

[รอใส่: กิตติกรรมประกาศ — ขอบคุณอาจารย์ที่ปรึกษา หน่วยงาน และผู้ที่ช่วยเหลือ (เจ้าของโครงงานเขียนเอง)]

ผู้จัดทำ

สารบัญ

เนื้อหา	หน้า
บทคัดย่อ	**ก**
Abstract	**ข**
กิตติกรรมประกาศ	**ค**
สารบัญ	**ง**
สารบัญตาราง	**ฉ**
สารบัญรูป	**ฌ**
บทที่ 1 บทนำ	**1**
1.1 ความเป็นมาและความสำคัญของปัญหา	1
1.2 วัตถุประสงค์ของโครงงาน	4
1.3 ขอบเขตของโครงงาน	4
1.4 ประโยชน์ที่คาดว่าจะได้รับ	7
1.5 ขั้นตอนและแผนการดำเนินงาน	8
1.6 นิยามศัพท์เฉพาะ	10
บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง	**12**
2.1 การตรวจพิสูจน์ DNA ทางนิติวิทยาศาสตร์และกระบวนการรับ–ส่งของกลาง	12
2.2 ระบบสารสนเทศและ Dashboard	16
2.3 Google Sheets และ Google Apps Script	19
2.4 LINE Messaging API	23
2.5 แชตบอตและแบบจำลองภาษาขนาดใหญ่	26
2.6 พระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562 และหลักการรักษาความมั่นคงปลอดภัย	30

2.7 งานวิจัยที่เกี่ยวข้อง 34
บทที่ 3 การออกแบบระบบ 39
3.1 ภาพรวมและสถาปัตยกรรมระบบ 39
3.2 การออกแบบโครงสร้างข้อมูล 44
3.3 การออกแบบส่วนแสดงผล Dashboard 49
3.4 การควบคุมสิทธิ์และการทำงานฝั่งเซิร์ฟเวอร์ 55
3.5 การออกแบบแชตบอต LINE 69
3.6 การออกแบบรายงานสรุปอัตโนมัติ 81
3.7 การออกแบบด้านความปลอดภัยและ PDPA 85
บทที่ 4 การทดสอบระบบและวิเคราะห์ผล 87
4.1 วิธีการและสภาพแวดล้อมการทดสอบ 87
4.2 การทดสอบความถูกต้องของข้อมูลจำลองและ Dashboard 94
4.3 การทดสอบการควบคุมสิทธิ์ 101
4.4 การทดสอบความถูกต้องของสถิติสรุป 104
4.5 การทดสอบแชตบอต LINE 108
4.6 การทดสอบด้าน PDPA 115
4.7 สรุปผลการทดสอบ 117
4.8 การทดสอบกับการใช้งานจริง 120
เอกสารอ้างอิง 122
ภาคผนวก 131
ภาคผนวก ก 132
ภาคผนวก ข 133

สารบัญตาราง

ตารางที่ **หน้า**

ตารางที่ 1.1 แผนการดำเนินงานโครงงาน 9
ตารางที่ 2.1 คำถามของผู้ใช้และรูปแบบการแสดงผล 18
ตารางที่ 2.2 โควตาของ Google Apps Script ที่เกี่ยวข้องกับโครงงาน (บัญชีผู้ใช้ทั่วไป) 22
ตารางที่ 2.3 ตัวอย่างข้อความกับเจตนาและเอนทิตีที่ต้องการ (ข้อความสมมติ) 27
ตารางที่ 2.4 เปรียบเทียบงานวิจัยที่เกี่ยวข้องกับโครงงาน 36
ตารางที่ 3.1 คอลัมน์ในแท็บข้อมูลคดี 44
ตารางที่ 3.2 คอลัมน์ในแท็บผู้ใช้ 46
ตารางที่ 3.3 คอลัมน์ที่เป็นข้อมูลส่วนบุคคลและการจัดการ 48
ตารางที่ 3.4 เมนูของ Dashboard แยกตามรูปแบบข้อมูล 50
ตารางที่ 3.5 การตั้งค่า deploy ของ Dashboard 56
ตารางที่ 3.6 ผลการตรวจสิทธิ์ของ checkAccess_ 57
ตารางที่ 3.7 ฟังก์ชันหลักใน Code.gs 60
ตารางที่ 3.8 คอลัมน์ของแท็บข้อมูลคดีที่ส่งและไม่ส่งไปยังเบราว์เซอร์ 63
ตารางที่ 3.9 Script Properties ของแชตบอตและสรุปอัตโนมัติ 69
ตารางที่ 3.10 วิธีเรียกแชตบอตและผลที่ได้ 70
ตารางที่ 3.11 ตัวอย่างการตัดข้อมูลก่อนส่งให้ Gemini (ข้อความสมมติ) 72

ตารางที่ 3.12 มาตรการทดแทนการตรวจลายมือชื่อ webhook ใน LineBot.gs 76
ตารางที่ 4.1 ลักษณะของข้อมูลจำลองที่ใช้ทดสอบ 88
ตารางที่ 4.2 เครื่องมือและสภาพแวดล้อมของการรันชุดทดสอบอัตโนมัติ 92
ตารางที่ 4.3 รายการที่ยังไม่ได้ทดสอบและเหตุผล 93
ตารางที่ 4.4 ผลการทดสอบข้อมูลจำลองและ Dashboard ต้นแบบ 94
ตารางที่ 4.5 ผลการทดสอบการอ่านข้อมูลฝั่งเซิร์ฟเวอร์ 97
ตารางที่ 4.6 ผลการทดสอบการแสดงผลของ Dashboard รุ่น Apps Script 98
ตารางที่ 4.7 ตัวชี้วัดบน Dashboard เทียบกับค่าที่คำนวณด้วย Python 100
ตารางที่ 4.8 ผลการทดสอบการควบคุมสิทธิ์ของ Dashboard 101
ตารางที่ 4.9 ผลการทดสอบการป้องกันจาก deployment ของแชตบอตและฟังก์ชันผู้ดูแล 103
ตารางที่ 4.10 สถิติจาก Stats.gs เทียบกับ Python และ Dashboard (จำนวนคดี / ของกลาง) 105
ตารางที่ 4.11 ผลการทดสอบการคำนวณช่วงเวลาของรายงาน 106
ตารางที่ 4.12 ผลการทดสอบคดีค้าง การค้นคดี และความตรงกันระหว่างส่วนต่าง ๆ ของระบบ 107
ตารางที่ 4.13 ผลการทดสอบการเรียกบอต การเงียบ และข้อความตอบ 109
ตารางที่ 4.14 ผลการทดสอบการส่งข้อความให้ Gemini 111

ตารางที่ 4.15 ผลการทดสอบการจำกัดกลุ่มและการป้องกัน webhook 112
ตารางที่ 4.16 ผลการทดสอบรายงานสรุปอัตโนมัติ 113
ตารางที่ 4.17 รายการตรวจด้าน PDPA และผลการทดสอบ 115
ตารางที่ 4.18 สรุปจำนวนกรณีทดสอบแยกตามหมวด 117

สารบัญรูป

รูปที่		หน้า
รูปที่ 2.1	ขั้นตอนการตรวจพิสูจน์ DNA ตั้งแต่รับของกลางจนออกรายงาน	13
รูปที่ 2.2	ลำดับสถานะคดีตั้งแต่รับคดีจนรับผล	15
รูปที่ 2.3	ตัวอย่างรูปแบบกราฟที่ใช้ใน Dashboard	19
รูปที่ 2.4	การรับคำขอ GET/POST ของเว็บแอป Apps Script	22
รูปที่ 2.5	ลำดับการทำงานของ LINE Messaging API ระหว่างผู้ใช้ LINE Platform และ bot server	24
รูปที่ 2.6	เปรียบเทียบการไหลของข้อมูลระหว่างแนวทางที่ส่งข้อมูลให้ LLM กับแนวทางของโครงงานที่ส่งเฉพาะข้อความคำถาม	30
รูปที่ 3.1	สถาปัตยกรรมระบบและเส้นทางการไหลของข้อมูล	42
รูปที่ 3.2	โครงหน้าจอ Dashboard แถบเมนูด้านซ้าย ส่วนหัว และแถบเลือกปี	50
รูปที่ 3.3	หน้าภาพรวมของ Dashboard ข้อมูลคดี	51
รูปที่ 3.4	หน้าพื้นที่และประเภทคดี	51
รูปที่ 3.5	หน้าประเภทคดีและผู้ตรวจรายเดือน	52
รูปที่ 3.6	หน้าระยะเวลาและงานค้าง	53
รูปที่ 3.7	หน้าตรวจสอบข้อมูล	53
รูปที่ 3.8	ลำดับการทำงานเมื่อผู้ใช้เปิด Dashboard ตั้งแต่ตรวจสิทธิ์จนถึงการขอข้อมูลซ้ำตามรอบเวลา	66
รูปที่ 3.9	ผังการทำงานของแชตบอต LINE	80

รูปที่ 3.10 ตัวอย่างข้อความสรุปรายเดือนในกลุ่ม LINE 84
รูปที่ 4.1 โครงสร้างของชุดทดสอบ ตั้งแต่การสร้างข้อมูลจำลองจนถึงการตัดสินผล 91
รูปที่ 4.2 หน้าภาพรวมของ Dashboard รุ่น Apps Script เมื่อกรองปี 2568 ขณะทดสอบ 100
รูปที่ 4.3 หน้าภาพรวมบนหน้าจอโทรศัพท์ความกว้าง 390 พิกเซล 100
รูปที่ 4.4 หน้า “ไม่มีสิทธิ์เข้าถึง” เมื่อเปิดด้วยอีเมลที่ไม่อยู่ในรายชื่อ 103

บทที่ 1
บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

การตรวจพิสูจน์ดีเอ็นเอ (DNA) เป็นเครื่องมือทางนิติวิทยาศาสตร์ที่มีบทบาทสำคัญในกระบวนการยุติธรรมทางอาญา เนื่องจากสามารถเชื่อมโยงวัตถุพยานทางชีวภาพที่เก็บได้จากสถานที่เกิดเหตุกับตัวบุคคลได้อย่างมีหลักฐานทางวิทยาศาสตร์รองรับ [1] อย่างไรก็ตาม ผลการตรวจจะมีน้ำหนักในชั้นศาลได้ก็ต่อเมื่อหน่วยงานสามารถแสดงความต่อเนื่องของการครอบครองวัตถุพยาน (chain of custody) ได้ครบถ้วน กล่าวคือ ต้องมีบันทึกว่าของกลางแต่ละชิ้นอยู่ในความดูแลของผู้ใด ณ ที่ใด และถูกส่งต่อเมื่อใด ตั้งแต่การเก็บจนถึงการจำหน่ายออก [2] งานรับ–ส่งคดีและการติดตามสถานะของกลางจึงเป็นงานธุรการที่ต้องการความถูกต้องสูง ไม่ต่างจากงานตรวจวิเคราะห์ในห้องปฏิบัติการ

กลุ่มงานตรวจชีววิทยาและดีเอ็นเอ ศูนย์พิสูจน์หลักฐาน 3 รับคดีที่สถานีตำรวจในพื้นที่รับผิดชอบส่งตรวจ [รอใส่: จังหวัดในพื้นที่รับผิดชอบของ ศพฐ.3 หรือคำบรรยายพื้นที่โดยย่อ] ในแต่ละปีมีคดีส่งตรวจจำนวนมาก [รอใส่: จำนวนคดีส่งตรวจต่อปีโดยประมาณ ถ้าเปิดเผยได้] แต่ละคดีมีข้อมูลที่ต้องติดตามหลายรายการ ได้แก่ เลขคดี เลขรายงาน สถานีตำรวจและจังหวัดที่ส่ง เลขหนังสือนำส่ง วันที่รับ วันที่ตรวจเสร็จ วันที่มารับผล สถานะ ประเภทคดี ผู้ตรวจ และจำนวนของกลาง นอกจากข้อมูลรายคดีแล้ว กลุ่มงานยังต้องติดตามข้อมูลด้านห้องปฏิบัติการ ได้แก่

จำนวนตัวอย่างตรวจรายเดือน รอบการเบิกพัสดุ (Lot) และงบประมาณรายปีงบประมาณ

ที่ผ่านมาเจ้าหน้าที่บันทึกและติดตามข้อมูลเหล่านี้ด้วยไฟล์ Microsoft Excel และ Google Sheets โดยการกรอกและสรุปด้วยมือ วิธีนี้มีข้อจำกัดหลายประการ ประการแรก การตอบคำถามประจำวัน เช่น “คดีเลขนี้ตรวจเสร็จหรือยัง” หรือ “ตอนนี้มีคดีค้างกี่คดี” ต้องเปิดไฟล์แล้วค้นหาเอง ทำให้เสียเวลาและต้องพึ่งเจ้าหน้าที่ที่เข้าถึงไฟล์ได้ ประการที่สอง การจัดทำสรุปรายเดือนและรายปี เช่น จำนวนคดีแยกตามประเภทหรือแยกตามผู้ตรวจ ต้องคำนวณซ้ำทุกรอบ จึงมีโอกาสคลาดเคลื่อนและได้ตัวเลขไม่ตรงกันระหว่างรายงานแต่ละฉบับ ประการที่สาม ข้อมูลระยะเวลาดำเนินการ (turnaround time) และงานค้าง (backlog) ซึ่งเป็นตัวชี้วัดที่แนวปฏิบัติด้านห้องปฏิบัติการ DNA แนะนำให้ติดตามอย่างสม่ำเสมอ [3] ไม่ได้ถูกแสดงในรูปที่ผู้บริหารมองเห็นได้ทันที และประการที่สี่ ไฟล์ที่แชร์กันหลายคนมีความเสี่ยงด้านการเข้าถึงข้อมูลเกินความจำเป็น

แนวปฏิบัติของสถาบันยุติธรรมแห่งชาติสหรัฐอเมริกา (NIJ) ระบุว่าห้องปฏิบัติการ DNA ควรใช้ระบบสารสนเทศจัดการห้องปฏิบัติการ (Laboratory Information Management System: LIMS) เพื่อติดตามงาน ระบุงานค้างและคอขวด และติดตามคดีที่ส่งเข้ามา [3] แต่ระบบ LIMS เชิงพาณิชย์มีต้นทุนสูงและต้องมีบุคลากรด้านเทคโนโลยีสารสนเทศดูแลเฉพาะ ซึ่งไม่สอดคล้องกับขนาดและทรัพยากรของกลุ่มงาน โครงงานนี้จึงเลือกแนวทางที่ต้นทุนต่ำกว่า คือ ต่อยอดจาก Google Sheets ที่เจ้า

หน้าที่ใช้อยู่แล้ว โดยไม่เปลี่ยนวิธีกรอกข้อมูลเดิม แล้วเพิ่มเครื่องมือ 2 ส่วน ได้แก่

1) **Dashboard บนเว็บ** สำหรับแสดงสถิติและตัวชี้วัดในรูปกราฟและตาราง ซึ่งเป็นการนำเสนอข้อมูลสำคัญให้มองเห็นได้ในหน้าจอเดียว [4] พัฒนาบน Google Apps Script ซึ่งเป็นแพลตฟอร์มพัฒนาโปรแกรมบนคลาวด์ที่ทำงานร่วมกับ Google Workspace ได้โดยตรง [5]
2) **แชตบอตบน LINE** สำหรับกลุ่ม LINE ของเจ้าหน้าที่ เชื่อมต่อผ่าน LINE Messaging API [6] เพื่อตอบสถานะคดีจากเลขคดี สรุปคดีค้าง และส่งสรุปประจำเดือนและประจำปีให้อัตโนมัติ

ข้อมูลคดีมีข้อมูลส่วนบุคคล เช่น ชื่อเจ้าหน้าที่ และอาจเชื่อมโยงไปถึงบุคคลในคดีได้ ขณะที่ข้อมูลพันธุกรรมจัดเป็นข้อมูลส่วนบุคคลที่มีความอ่อนไหวตามพระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562 [7] การออกแบบจึงยึดหลักว่าระบบจะไม่แสดงหรือจัดเก็บ DNA profile และข้อมูลคดีจาก Google Sheets จะไม่ถูกส่งออกไปยังบริการปัญญาประดิษฐ์ภายนอก แชตบอตใช้แบบจำลองภาษาขนาดใหญ่ Gemini เฉพาะการตีความข้อความคำถามของผู้ใช้ เช่น แยกว่าผู้ใช้ถามสถานะคดีหรือถามสรุปคดีค้าง โดยตัดเลขคดีและข้อความที่ระบุตัวบุคคลออกก่อนส่ง เลขคดีดึงด้วยโค้ดของระบบ และคำตอบสร้างจากโค้ดของระบบเอง ทั้งนี้ เงื่อนไขการใช้บริการ Gemini API แบบไม่เสียค่าใช้จ่ายระบุให้ผู้ใช้ไม่ส่งข้อมูลที่อ่อนไหว ข้อมูลลับ หรือข้อมูลส่วนบุคคลเข้าสู่บริการ [8] ซึ่งสอดคล้องกับแนวทางของโครงงาน

1.2 วัตถุประสงค์ของโครงงาน

1) เพื่อพัฒนา Dashboard บนเว็บด้วย Google Apps Script ที่อ่านข้อมูลจาก Google Sheets ติดตามคดีส่งตรวจ DNA และแสดงสถิติแยกตามเดือน ปี จังหวัด สถานีตำรวจ ประเภทคดี และผู้ตรวจ รวมทั้งคดีที่ยังไม่รับผล ระยะเวลาดำเนินการ ข้อมูล Lot พัสดุ และงบประมาณ โดยจำกัดการเข้าถึงเฉพาะเจ้าหน้าที่ที่มีรายชื่อ
2) เพื่อพัฒนาแชตบอตบน LINE สำหรับกลุ่มเจ้าหน้าที่ ที่ตอบสถานะคดีจากเลขคดีและสรุปคดีค้างได้ โดยใช้แบบจำลองภาษาขนาดใหญ่เพื่อตีความคำถามเท่านั้น และไม่ส่งข้อมูลคดีออกไปยังบริการปัญญาประดิษฐ์ภายนอก
3) เพื่อจัดทำระบบสรุปผลอัตโนมัติที่ส่งเข้ากลุ่ม LINE ทุกวันที่ 1 ของเดือน (สรุปเดือนปฏิทินก่อนหน้า) และทุกวันที่ 1 มกราคม (สรุปปีปฏิทินก่อนหน้า) โดยตัวเลขตรงกับ Dashboard
4) เพื่อทดสอบความถูกต้องของการคำนวณ การควบคุมสิทธิ์ และการคุ้มครองข้อมูลส่วนบุคคลของระบบด้วยข้อมูลจำลอง

1.3 ขอบเขตของโครงงาน

1.3.1 ขอบเขตที่อยู่ในโครงงาน

1) **แหล่งข้อมูล** ระบบอ่านข้อมูลจาก Google Sheets 2 ไฟล์ที่กลุ่มงานใช้อยู่ ได้แก่ ไฟล์ข้อมูลคดี (แท็บ

“ข้อมูลคดี”) และไฟล์ข้อมูลห้องปฏิบัติการ (แท็บ “ข้อมูลเคส” “ข้อมูลงบ” และ “ข้อมูล Lot”) โดยอ่านอย่างเดียว ไม่แก้ไขข้อมูลต้นฉบับ และเพิ่มแท็บ “ผู้ใช้” สำหรับรายชื่ออีเมลที่มีสิทธิ์เปิด Dashboard

2) **Dashboard** แสดงผลเป็นหน้าเว็บผ่าน HtmlService ของ Google Apps Script แบ่งเป็นกลุ่มเมนู
   2.1) ข้อมูลคดี: ภาพรวม, พื้นที่และประเภทคดี, ประเภท/ผู้ตรวจรายเดือน, ระยะเวลาและงานค้าง และตรวจสอบข้อมูล
   2.2) ข้อมูลห้องปฏิบัติการ: ภาพรวม, งบประมาณ และ Lot พัสดุ
3) **การควบคุมสิทธิ์** Dashboard เปิดได้เฉพาะบัญชี Google ที่มีอีเมลอยู่ในแท็บ “ผู้ใช้” และได้รับแชร์ Google Sheets แบบผู้มีสิทธิ์อ่าน และแชตบอตตอบเฉพาะกลุ่ม LINE ที่กำหนดไว้กลุ่มเดียว
4) **แชตบอต** รองรับคำถาม 2 ประเภท คือ (1) สถานะคดีจากเลขคดี (เลข บก. รูปแบบ ลำดับ/ปี พ.ศ.) และ (2) สรุปคดีค้าง โดยแสดงจำนวนคดี “กำลังดำเนินการ” และ “เสร็จแล้ว รอรับผล” แยกกัน ใช้ Gemini API เพื่อแยกเจตนาจากข้อความคำถามที่ตัดเลขคดีและข้อความที่ระบุตัวบุคคลออกแล้วเท่านั้น ข้อความที่ไม่ใช่คำสั่งบอตจะไม่ตอบ
5) **สรุปอัตโนมัติ** ส่งข้อความเข้ากลุ่ม LINE ตามเวลาที่กำหนด มีเนื้อหาเป็นจำนวนคดี จำนวนของกลาง

จำนวนคดีแยกประเภท และจำนวนคดีของผู้ตรวจแต่ละคนแยกตามประเภท (แสดงเป็นจำนวน ไม่แสดงเลขคดี) พร้อมลิงก์ไปยัง Dashboard

6) **การพัฒนาและทดสอบ** ใช้ข้อมูลจำลองที่มีโครงสร้างคอลัมน์เหมือนข้อมูลจริงทั้งหมด ภาพหน้าจอในเล่มใช้ข้อมูลจำลอง
7) **ค่าใช้จ่าย** ใช้บริการในระดับที่ไม่มีค่าใช้จ่าย ได้แก่ Google Apps Script ด้วยบัญชี Google, LINE Official Account และ Gemini API ระดับฟรี

1.3.2 สิ่งที่อยู่นอกขอบเขตของโครงงาน

1) ไม่จัดเก็บ ไม่ประมวลผล และไม่แสดง DNA profile หรือผลการตรวจวิเคราะห์ทางพันธุกรรมใด ๆ
2) ไม่เปลี่ยนแปลงขั้นตอนการตรวจในห้องปฏิบัติการ ขั้นตอนรับ–ส่งของกลาง หรือแบบฟอร์มบันทึก chain of custody ที่ใช้อยู่ ระบบเป็นเพียงเครื่องมือช่วยดูและสรุปข้อมูลที่บันทึกไว้แล้ว
3) ไม่ใช่ระบบ LIMS เต็มรูปแบบ ไม่มีการบันทึก แก้ไข หรืออนุมัติข้อมูลคดีผ่านระบบ การกรอกข้อมูลยังทำใน Google Sheets ตามเดิม
4) แชตบอตใช้ในกลุ่ม LINE ของเจ้าหน้าที่กลุ่มเดียว ไม่ให้บริการประชาชน พนักงานสอบสวน หรือสถานีตำรวจภายนอก และไม่ตอบคำถามนอกเหนือจาก 2 ประเภทที่กำหนด
5) ไม่ส่งข้อมูลคดีจาก Google Sheets ไปยัง Gemini หรือบริการปัญญาประดิษฐ์ภายนอกอื่นใด

6) ไม่เชื่อมต่อกับระบบสารสนเทศอื่นของสำนักงานตำรวจแห่งชาติ
7) การนำระบบขึ้นใช้งานกับข้อมูลจริงและกลุ่ม LINE จริงเป็นการตัดสินใจของเจ้าของโครงงานและหน่วยงาน ผลการทดสอบในเล่มนี้มาจากข้อมูลจำลองเท่านั้น

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1) เจ้าหน้าที่ทราบสถานะคดีได้จากการพิมพ์ถามในกลุ่ม LINE โดยไม่ต้องเปิดไฟล์และค้นหาเอง
2) หัวหน้ากลุ่มงานเห็นภาพรวมปริมาณงาน คดีค้าง และระยะเวลาดำเนินการจาก Dashboard ซึ่งช่วยในการวางแผนกำลังคนและการเบิกพัสดุ
3) ลดเวลาและความคลาดเคลื่อนในการจัดทำสรุปรายเดือนและรายปี เพราะคำนวณด้วยชุดฟังก์ชันเดียวกันทั้งใน Dashboard และข้อความสรุป
4) ข้อมูลคดีมีความเป็นส่วนตัวมากขึ้น เนื่องจาก Google Sheets ไม่ต้องเปิดแชร์แบบสาธารณะ และการเข้าถึง Dashboard จำกัดตามรายชื่อ
5) เป็นตัวอย่างการประยุกต์ใช้แบบจำลองภาษาขนาดใหญ่ในงานที่มีข้อมูลอ่อนไหว โดยแยกการตีความภาษาออกจากการเข้าถึงข้อมูล
6) ผู้จัดทำได้ประยุกต์ความรู้ด้านวิทยาศาสตร์การแพทย์และนิติวิทยาศาสตร์ร่วมกับเทคโนโลยีสารสนเทศ และ

ได้เรียนรู้การปฏิบัติตามหลักการคุ้มครองข้อมูลส่วนบุคคล

1.5 ขั้นตอนและแผนการดำเนินงาน

1.5.1 ขั้นตอนการดำเนินงาน

1) **ศึกษาความต้องการและระบบงานเดิม** สัมภาษณ์เจ้าหน้าที่ ศึกษาโครงสร้างไฟล์ Excel/Google Sheets ที่ใช้อยู่ และสรุปเป็นเอกสารโครงสร้างข้อมูล (เฉพาะชื่อคอลัมน์ ไม่มีค่าข้อมูลจริง)
2) **ศึกษาทฤษฎีและเครื่องมือ** ได้แก่ กระบวนการตรวจพิสูจน์ DNA และการรับ–ส่งของกลาง หลักการออกแบบ Dashboard, Google Apps Script, LINE Messaging API, Gemini API และ พ.ร.บ.คุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562
3) **สร้างข้อมูลจำลอง** ที่มีโครงสร้างตรงกับข้อมูลจริง สำหรับใช้พัฒนาและทดสอบ
4) **ออกแบบระบบ** ได้แก่ สถาปัตยกรรม การไหลของข้อมูล การควบคุมสิทธิ์ และหน้าจอ Dashboard
5) **พัฒนา Dashboard** ย้ายจาก Dashboard ต้นแบบมาให้บริการผ่าน Google Apps Script อ่านข้อมูลฝั่งเซิร์ฟเวอร์ และตรวจสิทธิ์ตามรายชื่อผู้ใช้
6) **พัฒนาแชตบอต LINE** ได้แก่ webhook การแยกเจตนาด้วย Gemini การค้นหาสถานะคดี และการสรุปคดีค้าง

7) **พัฒนาระบบสรุปอัตโนมัติ** ด้วย time-driven trigger รายเดือนและรายปี
8) **ทดสอบระบบ** ด้วยข้อมูลจำลอง ทั้งความถูกต้องของตัวเลข การควบคุมสิทธิ์ และการตรวจว่าไม่มีข้อมูลคดีถูกส่งออกไปยังบริการภายนอก
9) **ให้เจ้าของงานทดลองใช้และปรับปรุง** [รอใส่: จะมีการทดลองใช้กับเจ้าหน้าที่จริงหรือประเมินความพึงพอใจหรือไม่]
10) **จัดทำรูปเล่มและนำเสนอ**

1.5.2 แผนการดำเนินงาน

แผนการดำเนินงานตามขั้นตอนในหัวข้อ 1.5.1 แบ่งเป็นรายสัปดาห์ ดังตารางที่ 1.1

ตารางที่ 1.1 แผนการดำเนินงานโครงงาน

ขั้นตอน	**[รอใส่: เดือนที่ 1]**				**[รอใส่: เดือนที่ 2]**				**[รอใส่: เดือนที่ 3]**				**[รอใส่: เดือนที่ 4]**			
สัปดาห์ที่	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1. ศึกษาความต้องการและระบบงานเดิม	x	x														
2. ศึกษาทฤษฎีและเครื่องมือ	x	x	x													
3. สร้างข้อมูลจำลอง			x													
4. ออกแบบระบบ			x	x												
5. พัฒนา Dashboard				x	x	x	x									
6. พัฒนาแชตบอต LINE							x	x	x	x						

ขั้นตอน	[รอใส่: เดือนที่ 1]				[รอใส่: เดือนที่ 2]				[รอใส่: เดือนที่ 3]				[รอใส่: เดือนที่ 4]			
7. พัฒนาระบบสรุปอัตโนมัติ										x	x					
8. ทดสอบระบบด้วยข้อมูลจำลอง						x				x	x	x				
9. ทดลองใช้และปรับปรุง												x	x			
10. จัดทำรูปเล่มและนำเสนอ			x	x				x				x	x	x	x	x

หมายเหตุ: ช่วงเวลาในตารางเป็นแผนเบื้องต้น [รอใส่: ชื่อเดือนตามภาคการศึกษาจริง และปรับสัปดาห์ให้ตรงกับที่ทำจริง]

1.6 นิยามศัพท์เฉพาะ

- **คดีส่งตรวจ** หมายถึง คดีที่สถานีตำรวจส่งของกลางมาตรวจพิสูจน์ DNA ที่กลุ่มงานตรวจชีววิทยาและดีเอ็นเอ ศูนย์พิสูจน์หลักฐาน 3 หนึ่งคดีอาจมีของกลางหลายชิ้น
- **เลขคดี (เลข บก.)** หมายถึง เลขที่กลุ่มงานออกให้แต่ละคดีในรูปแบบ ลำดับ/ปี พ.ศ. ใช้เป็นคีย์ที่แชตบอตใช้ค้นหา
- **ของกลาง** หมายถึง วัตถุพยานที่ส่งมาตรวจ นับเป็นชิ้นตามที่บันทึกในคอลัมน์ “ของกลาง (ชิ้น)”
- **สถานะคดี** มี 3 ค่า ได้แก่ “กำลังดำเนินการ” “เสร็จแล้ว รอรับผล” และ “รับผลไปแล้ว”
- **คดีค้าง** หมายถึง คดีที่ยังไม่รับผล โดยรายงาน**แยกเป็น 2 กลุ่ม** คือ “กำลังดำเนินการ” (รับคดีแล้ว ยัง

ตรวจไม่เสร็จ) และ “เสร็จแล้ว รอรับผล” (ตรวจเสร็จแล้ว รอหน่วยงานผู้ส่งมารับผล) ทั้ง Dashboard และแชตบอตแสดงจำนวนของแต่ละกลุ่มแยกกัน

- **ระยะเวลาดำเนินการ (turnaround time)** หมายถึง จำนวนวันตั้งแต่วันที่รับคดีถึงวันที่ตรวจเสร็จ
- **Lot พัสดุ** หมายถึง รอบการใช้หรือเบิกน้ำยาและวัสดุสำหรับตรวจ นับรอบตั้งแต่วันที่ 26 ถึงวันที่ 25 ของเดือนถัดไป
- **ปีงบประมาณ** หมายถึง ช่วงวันที่ 1 ตุลาคม ถึง 30 กันยายนของปีถัดไป
- **Dashboard** หมายถึง หน้าจอแสดงข้อมูลสำคัญในรูปกราฟและตารางเพื่อให้ติดตามได้ในหน้าจอเดียว
- **แชตบอต (chatbot)** หมายถึง โปรแกรมที่โต้ตอบกับผู้ใช้ด้วยข้อความในแอปพลิเคชันสนทนา
- **แบบจำลองภาษาขนาดใหญ่ (Large Language Model: LLM)** หมายถึง แบบจำลองปัญญาประดิษฐ์ที่ฝึกด้วยข้อความจำนวนมาก สามารถเข้าใจและสร้างข้อความภาษาธรรมชาติ ในโครงงานนี้คือ Gemini
- **ข้อมูลจำลอง (mock data)** หมายถึง ข้อมูลที่สร้างขึ้นโดยโปรแกรม มีโครงสร้างเหมือนข้อมูลจริงแต่ไม่มีค่าที่มาจากคดีจริง

บทที่ 2
ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

ในบทนี้กล่าวถึงทฤษฎีและงานวิจัยที่เกี่ยวข้องกับโครงงาน แบ่งเป็น 7 หัวข้อ ได้แก่ การตรวจพิสูจน์ DNA ทางนิติวิทยาศาสตร์และกระบวนการรับ–ส่งของกลาง ระบบสารสนเทศและ Dashboard, Google Sheets และ Google Apps Script, LINE Messaging API แชตบอตและแบบจำลองภาษาขนาดใหญ่ พระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562 และหลักการรักษาความมั่นคงปลอดภัย และงานวิจัยที่เกี่ยวข้อง

2.1 การตรวจพิสูจน์ DNA ทางนิติวิทยาศาสตร์และกระบวนการรับ–ส่งของกลาง

2.1.1 หลักการตรวจพิสูจน์ DNA ทางนิติวิทยาศาสตร์

DNA (deoxyribonucleic acid) เป็นสารพันธุกรรมที่พบในเซลล์ที่มีนิวเคลียสเกือบทุกชนิดของร่างกาย และมีลำดับเบสบางตำแหน่งที่แตกต่างกันมากระหว่างบุคคล การตรวจพิสูจน์ DNA ทางนิติวิทยาศาสตร์ (forensic DNA typing) อาศัยความแตกต่างนี้ในการเปรียบเทียบ DNA จากวัตถุพยาน เช่น คราบเลือด คราบอสุจิ เส้นผมที่มีรากผม หรือเซลล์ที่ติดอยู่บนวัตถุ กับ DNA ของบุคคลอ้างอิง [1] ปัจจุบันเครื่องหมายพันธุกรรมที่ใช้เป็นหลักในงานนิติวิทยาศาสตร์คือ short tandem repeat (STR) ซึ่งเป็นลำดับเบสสั้น ๆ ที่ซ้ำกันต่อเนื่อง จำนวนครั้งที่ซ้ำแตกต่างกันระหว่างบุคคล [1]

ขั้นตอนการตรวจโดยทั่วไปประกอบด้วย 7 ขั้นตอน [1] ดังแสดงในรูปที่ 2.1

1) การเก็บและรับวัตถุพยาน พร้อมบันทึกการส่งมอบ
2) การตรวจคัดกรองเบื้องต้น (screening) เพื่อหาคราบชีววัตถุบนของกลาง
3) การสกัด DNA (DNA extraction)
4) การวัดปริมาณ DNA (DNA quantitation)
5) การเพิ่มปริมาณ DNA บริเวณ STR ด้วยปฏิกิริยาลูกโซ่พอลิเมอเรส (PCR amplification)
6) การแยกขนาดชิ้น DNA ด้วย capillary electrophoresis และการอ่านผลเป็น DNA profile
7) การแปลผล การคำนวณทางสถิติ และการจัดทำรายงานผลการตรวจ

[รอใส่ภาพ: รูปที่ 2.1]

ขั้นตอนการตรวจพิสูจน์ DNA ตั้งแต่รับของกลางจนออกรายงาน (วาดใหม่โดยผู้จัดทำ)

รูปที่ 2.1 ขั้นตอนการตรวจพิสูจน์ DNA ตั้งแต่รับของกลางจนออกรายงาน

ผลลัพธ์ของขั้นตอนที่ 6 และ 7 คือ DNA profile ซึ่งเป็นข้อมูลพันธุกรรมที่ระบุตัวบุคคลได้ โครงงานนี้ไม่เกี่ยวข้องกับขั้นตอนการตรวจวิเคราะห์ และไม่จัดเก็บหรือแสดง DNA profile ระบบเกี่ยวข้องเฉพาะข้อมูลเชิงบริหารของคดี ได้แก่ วันที่รับ วันที่

ตรวจเสร็จ วันที่มารับผล สถานะ ประเภทคดี ผู้ตรวจ และจำนวนของกลาง

กลุ่มงานตรวจชีววิทยาและดีเอ็นเอ ศูนย์พิสูจน์หลักฐาน 3 เป็นหน่วยงานที่ทำหน้าที่ดังกล่าวในพื้นที่รับผิดชอบ [รอใส่: คำอธิบายหน่วยงานโดยย่อ เช่น สังกัด ที่ตั้ง พื้นที่รับผิดชอบ และแหล่งอ้างอิงที่เปิดเผยได้ (เช่น เว็บไซต์ของหน่วยงาน)]

2.1.2 กระบวนการรับ–ส่งของกลางและความต่อเนื่องของการครอบครอง (chain of custody)

คู่มือการเก็บรักษาวัตถุพยานทางชีวภาพของ National Institute of Standards and Technology (NIST) อธิบายว่า บันทึก chain of custody เป็นเอกสารที่บันทึกการเคลื่อนย้าย ตำแหน่งที่เก็บ และผู้ครอบครองวัตถุพยานตามลำดับเวลา ตั้งแต่การเก็บจนถึงการจำหน่ายออก การส่งมอบวัตถุพยานระหว่างบุคคลหรือระหว่างสถานที่เก็บทุกครั้งต้องได้รับการบันทึก และหากความต่อเนื่องนี้ขาดตอน อาจเป็นเหตุให้มีการโต้แย้งการรับฟังพยานหลักฐานในชั้นศาลได้ [2] นอกจากนี้ สิ่งที่ได้จากการแยกตัวอย่างหลัก เช่น DNA ที่สกัดได้ ก็ควรมี chain of custody ของตัวเองในระดับเดียวกับวัตถุพยานต้นทาง [2] คู่มือเดียวกันระบุว่าบันทึกนี้อาจจัดทำด้วยกระดาษ ระบบอิเล็กทรอนิกส์ หรือทั้งสองแบบร่วมกัน [2]

เมื่อพิจารณาโครงสร้างข้อมูลที่กลุ่มงานบันทึกอยู่ (ดูรายละเอียดในบทที่ 3) สามารถสรุปกระบวนการรับ–ส่งคดีเป็นลำดับเหตุการณ์ได้ดังนี้ และแสดงลำดับสถานะคดีในรูปที่ 2.2

1) สถานีตำรวจส่งของกลางพร้อมหนังสือนำส่ง (บันทึกเป็น “เลขหนังสือนำส่ง” และ “ลงวันที่”)

2) กลุ่มงานรับคดี ออกเลขคดี (เลข บก.) และบันทึก “วันที่รับ” สถานะเป็น “กำลังดำเนินการ”
3) มอบหมายผู้ตรวจ (“ผู้ตรวจ”) และดำเนินการตรวจ
4) ตรวจเสร็จและออกรายงาน (“เลขรายงาน” และ “วันที่ตรวจเสร็จ”) สถานะเป็น “เสร็จแล้ว รอรับผล”
5) หน่วยงานผู้ส่งมารับผล (“วันที่มารับผล”) สถานะเป็น “รับผลไปแล้ว”

[รอใส่ภาพ: รูปที่ 2.2]
ลำดับสถานะคดีตั้งแต่รับคดีจนรับผล (วาดใหม่โดยผู้จัดทำ)

รูปที่ 2.2 ลำดับสถานะคดีตั้งแต่รับคดีจนรับผล

[รอใส่: ให้เจ้าของโครงงานยืนยันว่าลำดับขั้นตอนข้างต้นตรงกับการปฏิบัติงานจริงของกลุ่มงาน และมีขั้นตอนอื่นที่ควรระบุหรือไม่]

บันทึก chain of custody อย่างเป็นทางการของหน่วยงานยังคงเป็นเอกสารตามระเบียบเดิม โครงงานนี้ไม่ได้แทนที่บันทึกดังกล่าว แต่ใช้ข้อมูลสถานะและวันที่ที่บันทึกไว้แล้วมาสรุปและแสดงผลเพื่อการติดตามงาน

2.1.3 ตัวชี้วัดการดำเนินงานของห้องปฏิบัติการ DNA

แนวปฏิบัติของ National Institute of Justice (NIJ) แนะนำให้ผู้บริหารห้องปฏิบัติการ DNA ใช้ตัวชี้วัด (metrics) เพื่อติดตามปริมาณงานที่ส่งเข้ามา ปริมาณการตรวจ ระยะเวลาดำเนิน

การ (turnaround time) และผลการปฏิบัติงานของผู้ตรวจ รวมทั้งนิยามและวัดปริมาณงานค้าง (backlog) ให้ชัดเจน [3] ตัวชี้วัดที่โครงงานนำมาแสดงผลมีดังนี้

1) **จำนวนคดีและจำนวนของกลาง** รายเดือนและรายปี นับตามวันที่รับคดี
2) **การกระจายตามพื้นที่และประเภทคดี** ได้แก่ จังหวัด สถานีตำรวจ และประเภทคดี
3) **ปริมาณงานของผู้ตรวจ** จำนวนคดีที่ผู้ตรวจแต่ละคนได้รับ แยกตามเดือนและประเภทคดี
4) **ระยะเวลาดำเนินการ** คำนวณจากผลต่างของวันที่ตรวจเสร็จกับวันที่รับคดี ดังสมการที่ (2.1)

$$TAT_i = d^{done}_i - d^{recv}_i \quad (2.1)$$

เมื่อ TAT_i คือระยะเวลาดำเนินการของคดีที่ i (วัน), d^{done}_i คือวันที่ตรวจเสร็จ และ d^{recv}_i คือวันที่รับคดี

5) **งานค้าง** คดีที่ยังไม่รับผล และจำนวนวันที่ค้างนับจากวันที่รับจนถึงวันปัจจุบัน
6) **ข้อมูลห้องปฏิบัติการ** จำนวนตัวอย่างตรวจ รอบ Lot พัสดุ และงบประมาณรายปีงบประมาณ

2.2 ระบบสารสนเทศและ Dashboard

2.2.1 ระบบสารสนเทศเพื่อการจัดการห้องปฏิบัติการ

ระบบสารสนเทศจัดการห้องปฏิบัติการ (Laboratory Information Management System: LIMS) เป็นซอฟต์แวร์ที่เชื่อมโยงบุคลากร เครื่องมือ และข้อมูลภายในห้องปฏิบัติการ แนวปฏิบัติของ NIJ ระบุว่า LIMS สามารถติดตามและบันทึกกิจกรรม

ตั้งแต่การรับคดี chain of custody การดำเนินการกับวัตถุพยานและตัวอย่าง การออกรายงาน การทบทวน ไปจนถึงการติดตามคลังวัสดุสิ้นเปลือง ข้อมูลบัญชีและงบประมาณ และการระบุคอขวดของกระบวนการ ซึ่งช่วยลดระยะเวลาดำเนินการและงานค้าง [3] NIJ ยังแนะนำให้ห้องปฏิบัติการใช้ LIMS เพื่อติดตามแนวโน้มการรับคดีเทียบกับขีดความสามารถของห้องปฏิบัติการ และให้พิจารณาความต้องการของระบบ ค่าใช้จ่ายในการปรับแต่ง และการสนับสนุนด้านเทคโนโลยีสารสนเทศก่อนจัดหา [3]

โครงงานนี้ไม่ได้พัฒนา LIMS เต็มรูปแบบ แต่เลือกทำหน้าที่บางส่วนของ LIMS ที่เกี่ยวกับการติดตามและสรุปข้อมูล ได้แก่ การติดตามสถานะคดี งานค้าง ระยะเวลาดำเนินการ ข้อมูลพัสดุ และงบประมาณ โดยใช้ข้อมูลจาก Google Sheets ที่มีอยู่

2.2.2 Dashboard และหลักการออกแบบ

Few [4] ให้ความหมายของ Dashboard ว่าเป็นการแสดงผลด้วยภาพของข้อมูลที่สำคัญที่สุดซึ่งจำเป็นต่อการบรรลุวัตถุประสงค์หนึ่งหรือหลายข้อ โดยรวมและจัดวางไว้ในหน้าจอเดียวเพื่อให้ติดตามได้ในพริบตา จากความหมายนี้และแนวคิดการออกแบบในหนังสือเล่มเดียวกัน [4] ผู้จัดทำกำหนดหลักการออกแบบ Dashboard ของโครงงานไว้ดังนี้

1) **แสดงเฉพาะข้อมูลที่จำเป็นต่อการตัดสินใจ** และจัดลำดับความสำคัญ ตัวเลขสรุปหลัก (เช่น จำนวนคดี คดีค้าง) อยู่ด้านบน รายละเอียดอยู่ด้านล่างหรือในหน้าย่อย
2) **ลดองค์ประกอบตกแต่งที่ไม่สื่อข้อมูล** เช่น ภาพสามมิติ เงา และสีที่ไม่มีความหมาย

3) **ใช้สีอย่างมีความหมาย** ใช้สีเน้นเฉพาะสิ่งที่ต้องการให้สังเกต เช่น คดีที่ค้างนานผิดปกติ
4) **ให้บริบทแก่ตัวเลข** เช่น เปรียบเทียบกับเดือนก่อนหน้าหรือปีก่อนหน้า
5) **ให้ผู้ใช้เจาะดูรายละเอียดได้** เช่น คลิกแถวในตารางสรุปเพื่อดูรายการคดีที่เกี่ยวข้อง (ข้อนี้เป็นความต้องการของผู้ใช้ในโครงงาน ไม่ได้มาจาก [4])

2.2.3 การเลือกรูปแบบการนำเสนอข้อมูลด้วยภาพ (data visualization)

การเลือกชนิดของกราฟขึ้นกับคำถามที่ต้องการตอบ [4] ตารางที่ 2.1 สรุปความสัมพันธ์ระหว่างคำถามของผู้ใช้กับรูปแบบการแสดงผลที่ใช้ใน Dashboard ของโครงงาน

ตารางที่ 2.1 คำถามของผู้ใช้และรูปแบบการแสดงผล

คำถามของผู้ใช้	ลักษณะข้อมูล	รูปแบบการแสดงผล
ปริมาณคดีเปลี่ยนไปอย่างไรในแต่ละเดือน	อนุกรมเวลา	กราฟเส้นหรือกราฟแท่งเรียงตามเวลา
จังหวัดหรือสถานีตำรวจใดส่งตรวจมากที่สุด	การเปรียบเทียบหมวดหมู่	กราฟแท่งแนวนอนเรียงจากมากไปน้อย
ประเภทคดีในแต่ละเดือนเป็นอย่างไร	สองมิติ (หมวดหมู่ × เวลา)	ตารางไขว้หรือแผนภาพความร้อน (heatmap)
ผู้ตรวจแต่ละคนได้รับคดีเท่าใด	สองมิติ (บุคคล × เวลา)	ตารางไขว้
คดีใช้เวลาตรวจนานเท่าใด	การกระจายของค่าตัวเลข	กราฟแท่งแสดงจำนวนคดีตามช่วงจำนวนวัน
คดีใดยังค้างอยู่	รายการ	ตารางเรียงตามจำนวนวันที่ค้าง

ตัวอย่างรูปแบบกราฟที่ Dashboard ของโครงงานใช้ตามตารางที่ 2.1 แสดงในรูปที่ 2.3

[รอใส่ภาพ: รูปที่ 2.3]

ตัวอย่างรูปแบบกราฟที่ใช้ใน Dashboard (ภาพจากข้อมูลจำลอง)

รูปที่ 2.3 ตัวอย่างรูปแบบกราฟที่ใช้ใน Dashboard

2.3 Google Sheets และ Google Apps Script

2.3.1 Google Sheets

Google Sheets เป็นโปรแกรมตารางคำนวณออนไลน์ใน Google Workspace ผู้ใช้สร้างและจัดรูปแบบตารางคำนวณ และแชร์ไฟล์ให้ผู้อื่นโดยกำหนดสิทธิ์ได้ว่าให้ดู แสดงความคิดเห็น หรือแก้ไข [9] กลุ่มงานใช้ Google Sheets เป็นที่บันทึกข้อมูลคดีและข้อมูลห้องปฏิบัติการ โครงงานจึงใช้ Google Sheets เป็นแหล่งข้อมูลหลักต่อไปโดยไม่เปลี่ยนวิธีกรอกข้อมูลของเจ้าหน้าที่

2.3.2 Google Apps Script

Google Apps Script เป็นแพลตฟอร์มพัฒนาโปรแกรมอย่างรวดเร็วสำหรับสร้างโปรแกรมที่ทำงานร่วมกับ Google Workspace ผู้พัฒนาเขียนโปรแกรมด้วยภาษา JavaScript ในโปรแกรมแก้ไขบนเว็บเบราว์เซอร์โดยไม่ต้องติดตั้งซอฟต์แวร์ใด ๆ สคริปต์ถูกบันทึกไว้ใน Google Drive และทำงานบนเซิร์ฟเวอร์ของ

Google [5] บริการ (service) ของ Apps Script ที่เกี่ยวข้องกับโครงงานมีดังนี้

1) **SpreadsheetApp** ใช้เข้าถึงและสร้างไฟล์ Google Sheets เช่น เมธอด `openById(id)` เปิดไฟล์จากรหัสไฟล์ที่ปรากฏใน URL [10] การอ่านข้อมูลด้วยบริการนี้ทำบนเซิร์ฟเวอร์ ไฟล์ Google Sheets จึงไม่จำเป็นต้องแชร์แบบ “ทุกคนที่มีลิงก์”
2) **HtmlService** ใช้สร้างหน้า HTML และเนื้อหาข้อความส่งให้เบราว์เซอร์ โดยเนื้อหาจะผ่านการกรอง (sanitize) เพื่อป้องกันการกระทำที่เป็นอันตราย [11]
3) **Session** เมธอด `Session.getActiveUser().getEmail()` คืนค่าอีเมลของผู้ใช้ปัจจุบัน แต่จะคืนค่าเป็นสตริงว่างหากนโยบายความปลอดภัยไม่อนุญาตให้เข้าถึงตัวตนของผู้ใช้ เช่น ในเว็บแอป (web app) ที่ตั้งค่าให้ทำงานในนามผู้พัฒนา ข้อจำกัดนี้โดยทั่วไปไม่เกิดขึ้นเมื่อผู้ใช้อยู่ในโดเมน Google Workspace เดียวกับผู้พัฒนา [12] ข้อจำกัดนี้มีผลโดยตรงต่อการออกแบบการตรวจสิทธิ์ผู้ใช้ Dashboard ซึ่งจะกล่าวในบทที่ 3
4) **PropertiesService** เมธอด `getScriptProperties()` ให้ที่เก็บข้อมูลแบบคีย์–ค่าที่ใช้ได้เฉพาะภายในสคริปต์นั้น สคริปต์อื่นเข้าถึงไม่ได้ [13] โครงงานใช้เก็บค่าที่เป็นความลับ เช่น channel access token ของ LINE และ API key ของ Gemini แทนการเขียนไว้ในโค้ด

5) **UrlFetchApp** ใช้ติดต่อกับเครื่องบริการอื่นผ่านอินเทอร์เน็ต [14] โครงงานใช้เรียก LINE Messaging API และ Gemini API

2.3.3 เว็บแอปใน Google Apps Script

สคริปต์ที่จะเผยแพร่เป็นเว็บแอปต้องมีฟังก์ชัน `doGet(e)` หรือ `doPost(e)` ซึ่ง Apps Script เรียกเมื่อได้รับคำขอ HTTP GET หรือ POST ตามลำดับ พารามิเตอร์ `e` เก็บข้อมูลคำขอ เช่น `e.parameter` สำหรับพารามิเตอร์ใน URL และ `e.postData.contents` สำหรับเนื้อหาที่ส่งมาด้วย POST ฟังก์ชันต้องคืนค่าเป็นอ็อบเจกต์ `HtmlOutput` หรือ `TextOutput` [15] การเผยแพร่ทำโดยการ deploy และเลือกชนิดเป็นเว็บแอป และกำหนดว่าสคริปต์จะทำงานในนามใคร ซึ่งมี 2 แบบ [15]

1) **Execute as me** สคริปต์ทำงานในนามเจ้าของสคริปต์เสมอ ไม่ว่าใครเป็นผู้เปิด
2) **Execute as user accessing the web app** สคริปต์ทำงานในนามผู้ที่เปิดเว็บแอป ผู้ใช้ต้องให้สิทธิ์ (authorize) ก่อนใช้งาน

การรับคำขอ GET และ POST ของเว็บแอปแสดงในรูปที่ 2.4 ในโครงงาน `doGet` ใช้ให้บริการหน้า Dashboard ซึ่ง deploy แบบ Execute as user accessing the web app เพื่อให้ทราบอีเมลของผู้ใช้ (ดูข้อจำกัดของ `Session` ในหัวข้อ 2.3.2) และ `doPost` ใช้เป็น webhook รับเหตุการณ์จาก LINE ซึ่ง deploy แยกต่างหาก รายละเอียดอยู่ในบทที่ 3

[รอใส่ภาพ: รูปที่ 2.4]

การรับคำขอ GET/POST ของเว็บแอป Apps Script (วาดใหม่โดยผู้จัดทำ)

รูปที่ 2.4 การรับคำขอ GET/POST ของเว็บแอป Apps Script

2.3.4 ทริกเกอร์ตามเวลา (time-driven trigger)

ทริกเกอร์ตามเวลาเป็นทริกเกอร์ชนิดติดตั้ง (installable trigger) ที่ทำงานคล้ายกับ cron ของระบบปฏิบัติการ Unix คือ เรียกฟังก์ชันตามเวลาที่กำหนดหรือเป็นรอบ ตั้งได้ถี่สุดทุกนาทีและห่างสุดเดือนละครั้ง [16] เวลาเริ่มทำงานจริงอาจถูกสุ่มภายในชั่วโมงที่กำหนด เช่น ทริกเกอร์ที่ตั้งเวลา 9 นาฬิกาอาจทำงานในช่วง 9–10 นาฬิกา [16] โครงงานใช้ทริกเกอร์นี้ส่งสรุปรายเดือนและรายปี ดังนั้นข้อความสรุปจึงอาจไม่ได้ส่งตรงนาทีที่กำหนด แต่อยู่ภายในชั่วโมงเดียวกัน

2.3.5 โควตาและข้อจำกัดของ Google Apps Script

Google กำหนดโควตาการใช้งาน Apps Script ไว้หลายรายการ [17] รายการที่เกี่ยวข้องกับโครงงานสำหรับบัญชีผู้ใช้ทั่วไปแสดงในตารางที่ 2.2

ตารางที่ 2.2 โควตาของ Google Apps Script ที่เกี่ยวข้องกับโครงงาน (บัญชีผู้ใช้ทั่วไป)

รายการ	ขีดจำกัด
เวลาทำงานของสคริปต์ต่อการเรียกหนึ่งครั้ง	6 นาที
เวลาทำงานรวมของทริกเกอร์ต่อวัน	90 นาที
จำนวนการเรียก URL Fetch ต่อวัน	20,000 ครั้ง

ข้อจำกัดเหล่านี้เพียงพอสำหรับปริมาณข้อมูลระดับหลักร้อยถึงหลักพันแถว และจำนวนคำถามในกลุ่มเจ้าหน้าที่กลุ่มเดียว แต่ต้องออกแบบให้อ่านข้อมูลจาก Sheet อย่างมีประสิทธิภาพ เช่น อ่านทั้งช่วงข้อมูลครั้งเดียวแทนการอ่านทีละเซลล์ ทั้งนี้ โควตาอาจเปลี่ยนแปลงได้ ควรตรวจสอบจากเอกสารทางการก่อนใช้งานจริง

2.4 LINE Messaging API

2.4.1 หลักการทำงาน

LINE Messaging API เป็นส่วนต่อประสานโปรแกรมประยุกต์ที่ให้เซิร์ฟเวอร์ของผู้พัฒนา (bot server) แลกเปลี่ยนข้อมูลกับ LINE Platform ผ่านโปรโตคอล HTTPS ในรูปแบบ JSON ผ่านบัญชี LINE Official Account การทำงานมี 3 ขั้นตอน [6] ดังแสดงในรูปที่ 2.5

1) ผู้ใช้ส่งข้อความถึง LINE Official Account
2) LINE Platform ส่งเหตุการณ์ (webhook event) ไปยัง URL ของ webhook ที่ผู้พัฒนาตั้งไว้
3) bot server ประมวลผลเหตุการณ์ แล้วตอบกลับผ่าน LINE Platform

[รอใส่ภาพ: รูปที่ 2.5]

ลำดับการทำงานของ LINE Messaging API ระหว่างผู้ใช้ LINE Platform และ bot server (วาดใหม่โดยผู้จัดทำ)

รูปที่ 2.5 ลำดับการทำงานของ LINE Messaging API ระหว่างผู้ใช้ LINE Platform และ bot server

ในโครงงาน bot server คือเว็บแอป Apps Script ที่รับเหตุการณ์ผ่านฟังก์ชัน `doPost`

2.4.2 การส่งข้อความแบบตอบกลับ (reply) และแบบส่งเอง (push)

1) **ข้อความตอบกลับ (reply message)** ใช้ตอบการกระทำของผู้ใช้ โดยระบุ `replyToken` ที่ได้รับมากับเหตุการณ์ webhook ส่งได้สูงสุด 5 ข้อความต่อคำขอ [18]
2) **ข้อความส่งเอง (push message)** ส่งถึงผู้ใช้ กลุ่ม หรือห้องสนทนาได้ทุกเวลาโดยไม่ต้องมีการกระทำของผู้ใช้ก่อน [18]

ในด้านค่าใช้จ่าย จำนวนข้อความที่นับรวมในแผนบริการรายเดือนของ LINE Official Account คือข้อความแบบ push, multicast, broadcast และ narrowcast ส่วนข้อความแบบ reply ไม่นับรวม [19] และจำนวนข้อความที่นับคือจำนวนผู้รับ ไม่ใช่จำนวนอ็อบเจกต์ข้อความในคำขอ เช่น ส่ง 4 ข้อความเข้าห้องสนทนาที่มี 5 คน นับเป็น 5 ข้อความ [18] โครงงานจึงใช้ reply สำหรับตอบคำถามทั้งหมด และใช้ push เฉพาะสรุปตามเวลา จำนวนข้อความที่นับต่อเดือนจึงประมาณเท่ากับจำนวนสมาชิกในกลุ่มคูณจำนวนครั้งที่ส่งสรุป กลุ่ม LINE ของเจ้าหน้าที่มีสมาชิกประมาณ 20 คน การส่งสรุปรายเดือน 1 ครั้งจึงนับประมาณ 20 ข้อความ ส่วนเดือนมกราคมซึ่งส่งทั้งสรุปเดือนธันวาคมและสรุปทั้ง

ปี ถ้าส่งเป็น 2 ข้อความในคำขอ push เดียวกันก็ยังนับประมาณ 20 ข้อความ และการตอบคำถามเป็น reply ซึ่งไม่นับ [19] ปริมาณนี้ต่ำกว่าโควตาข้อความของแพ็กเกจฟรีของ LINE Official Account ในประเทศไทย ซึ่ง LINE ประกาศปรับเป็น 300 ข้อความต่อเดือนตั้งแต่วันที่ 1 สิงหาคม 2567 [20] ทั้งนี้ โควตาอาจเปลี่ยนแปลงได้ ควรตรวจสอบจากหน้าแพ็กเกจของบัญชีก่อนใช้งานจริง

2.4.3 การใช้งานในกลุ่มสนทนา

LINE Official Account เข้าร่วมกลุ่มสนทนาได้เมื่อเปิดการตั้งค่า “Allow bot to join group chats” ในหน้า LINE Developers Console ซึ่งปิดอยู่เป็นค่าเริ่มต้น และในหนึ่งกลุ่มมี LINE Official Account ได้เพียงบัญชีเดียว [21] เมื่อสมาชิกส่งข้อความในกลุ่ม อ็อบเจกต์ `source` ของเหตุการณ์จะมี `groupId` ของกลุ่มและ `userId` ของผู้ส่ง [21] การส่ง push เข้ากลุ่มทำได้โดยระบุรหัสกลุ่มในฟิลด์ `to` และสมาชิกทุกคนจะเห็นข้อความ [21] โครงงานใช้ `groupId` ในการจำกัดให้บอตตอบเฉพาะกลุ่มที่อนุญาต

2.4.4 การตรวจสอบลายมือชื่อของ webhook

เพื่อยืนยันว่าคำขอที่เข้ามายัง webhook มาจาก LINE Platform จริงและไม่ถูกแก้ไขระหว่างทาง LINE Platform จะคำนวณค่า HMAC-SHA256 ของเนื้อหาคำขอโดยใช้ channel secret เป็นกุญแจ และส่งค่านี้มาในส่วนหัว `x-line-signature` เซิร์ฟเวอร์ต้องคำนวณค่าเดียวกันจากเนื้อหาคำขอดิบ (ห้ามแปลงหรือจัดรูปแบบใหม่ก่อน) แล้วเปรียบเทียบ หากไม่ตรงกันหรือไม่มีส่วนหัวนี้ ไม่ควรประมวลผลเหตุการณ์ [22] เอกสารยังระบุว่าไม่

ควรใช้ที่อยู่ IP ในการยืนยันแหล่งที่มา เนื่องจาก LINE ไม่เปิดเผยที่อยู่ IP ที่ใช้ส่ง webhook [22] อย่างไรก็ตาม อ็อบเจกต์เหตุการณ์ที่ Apps Script ส่งให้ `doPost` มีเพียงพารามิเตอร์และเนื้อหาของคำขอ ไม่มีส่วนหัวของคำขอ [15] เว็บแอป Apps Script จึงอ่านส่วนหัว `x-line-signature` และตรวจลายมือชื่อตามที่ LINE แนะนำไม่ได้ โครงงานถือเป็นข้อจำกัดที่ทราบ และใช้มาตรการทดแทนหลายชั้น เช่น รหัสลับใน URL ของ webhook การตอบเฉพาะกลุ่มที่อนุญาต และการตอบผ่าน reply token เท่านั้น รายละเอียดอยู่ในหัวข้อ 3.5.5

2.5 แชตบอตและแบบจำลองภาษาขนาดใหญ่

2.5.1 แชตบอต

แชตบอต (chatbot) คือโปรแกรมที่โต้ตอบกับผู้ใช้ด้วยภาษาธรรมชาติ Adamopoulou และ Moussiades [23] ทบทวนวรรณกรรมด้านแชตบอตและจำแนกแนวทางการพัฒนาเป็น 2 แนวทางหลัก ได้แก่

1) **แบบจับคู่รูปแบบ (pattern matching / rule-based)** ใช้กฎหรือรูปแบบข้อความที่กำหนดไว้ล่วงหน้า ทำงานคาดเดาได้แต่ไม่ยืดหยุ่นต่อการพิมพ์ที่หลากหลาย
2) **แบบการเรียนรู้ของเครื่อง (machine learning)** ใช้แบบจำลองทางภาษาในการทำความเข้าใจข้อความยืดหยุ่นกว่าแต่ผลลัพธ์อาจคาดเดาได้ยากกว่า

องค์ประกอบสำคัญของแชตบอตคือส่วนทำความเข้าใจภาษาธรรมชาติ (natural language understanding) ซึ่งทำหน้าที่

จำแนกเจตนา (intent classification) คือระบุว่าผู้ใช้ต้องการอะไร และ **สกัดเอนทิตี (entity extraction)** คือดึงค่าที่เกี่ยวข้องออกจากข้อความ [รอใส่: แหล่งอ้างอิงเรื่ององค์ประกอบการทำความเข้าใจภาษาธรรมชาติ (intent/entity) ของแชตบอต] ในโครงงานนี้มีเจตนา 4 แบบ ได้แก่ ถามสถานะคดี ถามสรุปคดีค้าง ถามวิธีใช้ และอื่น ๆ ส่วนเอนทิตีคือเลขคดีรูปแบบ ลำดับ/ปี พ.ศ. ซึ่งโครงงานสกัดด้วย regular expression ในโค้ด ไม่ใช้ LLM นอกจากนี้ แชตบอตในกลุ่มสนทนาต้องแยกก่อนว่าข้อความใดเป็นการเรียกบอต ข้อความที่ไม่ได้เรียกบอตจะไม่ถูกจำแนกเจตนาเลย ตัวอย่างข้อความกับเจตนาและเอนทิตีแสดงในตารางที่ 2.3

ตารางที่ 2.3 ตัวอย่างข้อความกับเจตนาและเอนทิตีที่ต้องการ (ข้อความสมมติ)

ข้อความจากผู้ใช้ (สมมติ)	เจตนา	เอนทิตี
“คดี 123/2568 ถึงไหนแล้ว”	สถานะคดี	123/2568
“บอต ขอสถานะ ๔๕/๒๕๖๘ หน่อย”	สถานะคดี	45/2568 (แปลงเลขไทยเป็นเลขอารบิก)
“คดีค้าง”	สรุปคดีค้าง	–
“บอต ใช้ยังไง”	วิธีใช้	–
“สวัสดีครับ”	ไม่ได้เรียกบอต (ไม่จำแนก)	–

2.5.2 แบบจำลองภาษาขนาดใหญ่

แบบจำลองภาษาขนาดใหญ่ (Large Language Model: LLM) ส่วนใหญ่ในปัจจุบันสร้างบนสถาปัตยกรรม Transformer ซึ่งอาศัยกลไก attention ในการประมวลผลลำดับข้อความ [24] Brown และคณะ [25] แสดงว่าแบบจำลองภาษาที่มีขนาดใหญ่เพียงพอสามารถทำงานใหม่ได้จากคำสั่งและตัวอย่างเพียงไม่กี่ตัวอย่างในข้อความนำเข้า (prompt) โดยไม่ต้องปรับค่าน้ำหนักของแบบจำลอง (few-shot / in-context learning) คุณสมบัตินี้ทำให้สามารถใช้ LLM จำแนกเจตนาของข้อความภาษาไทยได้โดยไม่ต้องรวบรวมชุดข้อมูลฝึกสอนเอง

Gemini เป็นตระกูลแบบจำลองของ Google ที่ให้บริการผ่าน Gemini API ความสามารถที่เกี่ยวข้องกับโครงงานมีดังนี้

1) **คำสั่งระบบ (system instruction)** ใช้กำหนดบทบาทและพฤติกรรมของแบบจำลอง [26] เช่น กำหนดให้ตอบเฉพาะการจำแนกเจตนาและห้ามตอบคำถามอื่น
2) **ผลลัพธ์แบบมีโครงสร้าง (structured output)** กำหนดให้แบบจำลองตอบตาม JSON Schema ที่ผู้พัฒนากำหนด ทำให้ได้ผลลัพธ์ที่คาดเดาได้และนำไปใช้ในโปรแกรมได้ทันที เหมาะกับงานสกัดข้อมูลและจำแนกประเภท [27] เช่น `{"intent": "case_status"}`
3) **ขีดจำกัดอัตราการใช้งาน (rate limit)** วัดเป็นจำนวนคำขอต่อนาที (RPM) จำนวนโทเค็นนำเข้าต่อนาที (TPM) และจำนวนคำขอต่อวัน (RPD) คิดต่อโปร

เจกต์ มีระดับใช้งานฟรี และโควตารายวันเริ่มนับใหม่ตอนเที่ยงคืนตามเวลาแปซิฟิก [28] หากเกินโควตาหรือเรียกไม่สำเร็จ ระบบต้องมีทางสำรอง ในโครงงานนี้คำถามที่พบบ่อยแยกเจตนาด้วยกฎโดยไม่เรียก Gemini อยู่แล้ว และเมื่อ Gemini ใช้งานไม่ได้ แชตบอตจะตอบวิธีใช้แทน

2.5.3 การใช้ LLM โดยไม่ส่งข้อมูลคดีออกนอกระบบ

เงื่อนไขการใช้บริการเพิ่มเติมของ Gemini API ระบุว่าสำหรับบริการแบบไม่เสียค่าใช้จ่าย (Unpaid Services) Google อาจนำข้อมูลนำเข้าและผลลัพธ์ไปใช้พัฒนาผลิตภัณฑ์และเทคโนโลยีการเรียนรู้ของเครื่อง และอาจมีผู้ตรวจสอบที่เป็นมนุษย์อ่านข้อมูลดังกล่าว พร้อมระบุว่า “Do not submit sensitive, confidential, or personal information to the Unpaid Services.” [8]

โครงงานจึงออกแบบให้ LLM ทำหน้าที่เพียง **ตีความข้อความคำถาม** เท่านั้น ข้อมูลที่ส่งไปยัง Gemini มีเพียงข้อความที่ผู้ใช้พิมพ์ซึ่งตัดเลขคดีและข้อความที่ระบุตัวบุคคลออกแล้ว ส่วนการค้นหาข้อมูลใน Google Sheets และการสร้างข้อความตอบกลับทำด้วยโค้ดใน Apps Script ทั้งหมด แนวทางนี้ต่างจากแนวทาง retrieval-augmented generation ที่ส่งข้อมูลจากฐานข้อมูลไปให้ LLM สร้างคำตอบ ซึ่งไม่เหมาะกับข้อมูลคดี การไหลของข้อมูลของทั้งสองแนวทางเปรียบเทียบไว้ในรูปที่ 2.6

[รอใส่ภาพ: รูปที่ 2.6]

เปรียบเทียบการไหลของข้อมูลระหว่างแนวทางที่ส่งข้อมูลให้ LLM กับแนวทางของโครงงานที่ส่งเฉพาะข้อความคำถาม (วาดใหม่โดยผู้จัดทำ)

รูปที่ 2.6 เปรียบเทียบการไหลของข้อมูลระหว่างแนวทางที่ส่งข้อมูลให้ LLM กับแนวทางของโครงงานที่ส่งเฉพาะข้อความคำถาม

ผู้ใช้อาจพิมพ์ข้อมูลส่วนบุคคลลงในคำถามเอง เช่น ชื่อผู้ต้องหา โครงงานจึงกำหนดมาตรการ 2 ข้อ คือ (1) แยกเจตนาด้วย regular expression ก่อน และเรียก Gemini เฉพาะเมื่อผู้ใช้เรียกบอตแล้วแต่กฎแยกเจตนาไม่ได้ และ (2) ตัดตัวเลขทั้งหมด (รวมเลขคดี) ชื่อที่มีคำนำหน้า อีเมล ลิงก์ และชื่อหน่วยออกจากข้อความก่อนส่งให้ Gemini ข้อจำกัดที่ยังคงอยู่คือการกรองข้อความอัตโนมัติอาจไม่ครอบคลุมข้อมูลส่วนบุคคลทุกรูปแบบ รายละเอียดการดำเนินการอยู่ในหัวข้อ 3.5

2.6 พระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562 และหลักการรักษาความมั่นคงปลอดภัย

2.6.1 สาระสำคัญของพระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562

พระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562 (Personal Data Protection Act: PDPA) ประกาศในราชกิจจานุเบกษา เล่ม 136 ตอนที่ 69 ก เมื่อวันที่ 27 พฤษภาคม 2562 [7] สาระสำคัญที่เกี่ยวข้องกับโครงงานมีดังนี้ [7], [29]

1) **ข้อมูลส่วนบุคคล** หมายถึง ข้อมูลเกี่ยวกับบุคคลซึ่งทำให้สามารถระบุตัวบุคคลนั้นได้ไม่ว่าทางตรงหรือทางอ้อม ไม่รวมข้อมูลของผู้ถึงแก่กรรม (มาตรา 6)
2) **ผู้ควบคุมข้อมูลส่วนบุคคล (data controller)** คือ บุคคลหรือนิติบุคคลที่มีอำนาจตัดสินใจเกี่ยวกับการเก็บรวบรวม ใช้ หรือเปิดเผยข้อมูลส่วนบุคคล และ **ผู้ประมวลผลข้อมูลส่วนบุคคล (data processor)** คือผู้ที่ดำเนินการตามคำสั่งหรือในนามของผู้ควบคุม (มาตรา 6)
3) **การเก็บรวบรวมเท่าที่จำเป็น** การเก็บรวบรวมข้อมูลส่วนบุคคลให้เก็บได้เท่าที่จำเป็นภายใต้วัตถุประสงค์อันชอบด้วยกฎหมายของผู้ควบคุม (มาตรา 22)
4) **ข้อมูลส่วนบุคคลที่มีความอ่อนไหว** ได้แก่ ข้อมูลเกี่ยวกับเชื้อชาติ เผ่าพันธุ์ ความคิดเห็นทางการเมือง ความเชื่อ พฤติกรรมทางเพศ ประวัติอาชญากรรม ข้อมูลสุขภาพ ความพิการ ข้อมูลสหภาพแรงงาน **ข้อมูลพันธุกรรม** ข้อมูลชีวภาพ หรือข้อมูลอื่นที่กระทบต่อเจ้าของข้อมูลในทำนองเดียวกัน ซึ่งมีเงื่อนไขการเก็บรวบรวมเข้มงวดกว่าข้อมูลทั่วไป (มาตรา 26)
5) **หน้าที่ด้านความมั่นคงปลอดภัย** ผู้ควบคุมข้อมูลต้องจัดให้มีมาตรการรักษาความมั่นคงปลอดภัยที่เหมาะสม เพื่อป้องกันการสูญหาย เข้าถึง ใช้ เปลี่ยนแปลง แก้ไข หรือเปิดเผยข้อมูลโดยไม่มีอำนาจหรือโดยมิชอบ และต้องแจ้งเหตุละเมิดข้อมูลส่วนบุคคลต่อสำนักงานคณะกรรมการคุ้มครองข้อมูลส่วนบุคคลภายใน 72

ชั่วโมงนับแต่ทราบเหตุ เว้นแต่การละเมิดนั้นไม่มีความเสี่ยงต่อสิทธิและเสรีภาพของบุคคล (มาตรา 37)

ข้อมูลในระบบของโครงงานที่เข้าข่ายข้อมูลส่วนบุคคล ได้แก่ ชื่อผู้ตรวจ อีเมลของผู้ใช้ Dashboard และ LINE userId ของผู้ส่งข้อความที่มากับเหตุการณ์ webhook ซึ่งระบบไม่จัดเก็บ (คอลัมน์ User ID (LINE) ในไฟล์เดิมถูกลบออกจาก Google Sheets แล้วตามหลักการใช้ข้อมูลเท่าที่จำเป็น) ส่วนข้อมูลคดีอาจเชื่อมโยงไปถึงบุคคลในคดีได้โดยอ้อม และ DNA profile เป็นข้อมูลพันธุกรรมซึ่งเป็นข้อมูลอ่อนไหวตามมาตรา 26 โครงงานจึงกำหนดไว้ในขอบเขตว่าไม่จัดเก็บและไม่แสดง DNA profile

[รอใส่: ความเห็นของหน่วยงานหรือเจ้าหน้าที่คุ้มครองข้อมูลส่วนบุคคล (DPO) ของสำนักงานตำรวจแห่งชาติ ว่ากิจกรรมนี้อยู่ภายใต้ข้อยกเว้นตามมาตรา 4 หรือไม่ — ถ้าไม่มีให้ตัดย่อหน้านี้ออก]

2.6.2 หลักการรักษาความมั่นคงปลอดภัยของข้อมูลที่ใช้ในโครงงาน

1) **หลักการให้สิทธิ์น้อยที่สุด (least privilege)**

Saltzer และ Schroeder [30] เสนอเป็นหลักการออกแบบระบบป้องกันข้อมูลว่า ทุกโปรแกรมและผู้ใช้ควรทำงานด้วยสิทธิ์น้อยที่สุดที่จำเป็นต่อการทำงาน ซึ่งปัจจุบันยังใช้เป็นนิยามในอภิธานศัพท์ของ NIST ว่าระบบควรจำกัดสิทธิ์การเข้าถึงของผู้ใช้ให้น้อยที่สุดเท่าที่จำเป็นต่อการปฏิบัติงานที่ได้รับมอบหมาย [31] ในโครงงานนำมาใช้ดังนี้

1.1) Dashboard เปิดได้เฉพาะอีเมลที่อยู่ในแท็บ “ผู้ใช้” Google Sheets แชร์แบบผู้มีสิทธิ์อ่านเฉพาะคนในรายชื่อ และสคริปต์ขอสิทธิ์อ่าน Google Sheets อย่างเดียว

1.2) แชตบอตตอบเฉพาะกลุ่ม LINE ที่กำหนด และตอบได้เพียงสถานะคดีและสรุปคดีค้าง

1.3) Gemini ได้รับเฉพาะข้อความคำถามที่ตัดเลขคดีและข้อความที่ระบุตัวบุคคลออกแล้ว ไม่ได้รับสิทธิ์เข้าถึง Google Sheets

2) **หลักการใช้ข้อมูลเท่าที่จำเป็น (data minimisation)** สอดคล้องกับมาตรา 22 [7], [29] คือแสดงและส่งต่อเฉพาะข้อมูลที่จำเป็นต่อวัตถุประสงค์ เช่น ข้อความสรุปรายเดือนแสดงเป็นจำนวนคดี ไม่แสดงเลขคดีรายคดี และไม่แสดง DNA profile

3) **การรักษาความลับ ความถูกต้องครบถ้วน และสภาพพร้อมใช้งาน (confidentiality, integrity, availability)** ซึ่งเป็นองค์ประกอบขั้นต่ำของมาตรการรักษาความมั่นคงปลอดภัย [29] ในโครงงานได้แก่ การไม่แชร์ Google Sheets แบบสาธารณะ การเก็บ token และ API key ใน Script Properties แทนการเขียนในโค้ด มาตรการทดแทนการตรวจลายมือชื่อ webhook [22] ซึ่ง Apps Script ทำไม่ได้ (หัวข้อ 2.4.4) และการใช้ข้อมูลจำลองในการพัฒนาและทดสอบ

2.7 งานวิจัยที่เกี่ยวข้อง

2.7.1 แนวปฏิบัติการใช้ระบบสารสนเทศในห้องปฏิบัติการ DNA

National Institute of Justice [3] จัดทำแนวปฏิบัติระดับชาติเพื่อเพิ่มประสิทธิภาพกระบวนการของห้องปฏิบัติการ DNA จากคณะทำงานผู้เชี่ยวชาญด้านนิติวิทยาศาสตร์และการบริหารห้องปฏิบัติการ ข้อเสนอแนะที่เกี่ยวข้องกับโครงงาน ได้แก่ ให้ผู้บริหารใช้ตัวชี้วัดติดตามปริมาณงาน ระยะเวลาดำเนินการ และผลการปฏิบัติงานของผู้ตรวจ และให้ใช้ LIMS ติดตามงาน ระบุแนวโน้มและคอขวด และเปรียบเทียบการรับคดีกับขีดความสามารถของห้องปฏิบัติการ งานนี้สนับสนุนว่าตัวชี้วัดที่โครงงานเลือกแสดงใน Dashboard เป็นตัวชี้วัดที่มีความหมายต่อการบริหารห้องปฏิบัติการ DNA

2.7.2 ระบบรายงานและ Dashboard บน Google Workspace ในหน่วยงานด้านสุขภาพ

สุภัคพงศ์ จินารัตน์ และคณะ [32] พัฒนาระบบรายงานข้อมูลและ Dashboard เพื่อสนับสนุนการตัดสินใจของผู้บริหารสถาบันเวชศาสตร์สมเด็จพระสังฆราชญาณสังวรเพื่อผู้สูงอายุ ซึ่งเดิมจัดการข้อมูลแบบแมนนวลและกระจัดกระจาย โดยใช้ Google Forms, Google Sheets, Looker Studio และ Google Sites ร่วมกับกรอบแนวคิด PDCA และ CRISP-DM ผลการทดลองใช้พบว่าลดระยะเวลาการทำงานจาก 5 วันเหลือ 1 วัน และความพึงพอใจเฉลี่ย 4.73 จาก 5.00 งานนี้แสดงว่าเครื่องมือของ Google Workspace ใช้แก้ปัญหาข้อมูลแบบแมนนวลในหน่วยงานรัฐด้าน

สุขภาพได้ แต่ใช้ Looker Studio ซึ่งต่างจากโครงงานนี้ที่สร้าง Dashboard เองด้วย HtmlService เพื่อควบคุมสิทธิ์ตามรายชื่อและแสดงผลเฉพาะทางได้

2.7.3 แชตบอต LINE ร่วมกับ Google Apps Script และ Google Sheets

1) ชาญชัย ศุภอรรถกร และ ฟ้าพราว เฉียบแหลม [33] พัฒนาแชตบอต LINE ประชาสัมพันธ์หลักสูตรเทคโนโลยีสารสนเทศและการสื่อสาร มหาวิทยาลัยอุบลราชธานี ใช้ Dialogflow ประมวลผลภาษาธรรมชาติ และใช้ Google Apps Script กับ Google Sheets จัดการข้อมูล ผลการประเมินความพึงพอใจอยู่ในระดับมากที่สุด (ค่าเฉลี่ย 4.52 ส่วนเบี่ยงเบนมาตรฐาน 0.60)
2) ชรัช แสวงเจริญ และ ชุติมา พิศาลย์ [34] พัฒนา LINE Chatbot สำหรับแจ้งซ่อมโสตทัศนูปกรณ์ มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ ใช้ LINE Messaging API, Google Apps Script และ Google Sheets ทดลองกับผู้ใช้ 100 คน พบว่าลดระยะเวลาการแจ้งซ่อมได้มากกว่าร้อยละ 50 และมีระบบแจ้งเตือนสถานะงานอัตโนมัติ ความพึงพอใจโดยรวมอยู่ในระดับมาก (ค่าเฉลี่ย 4.50)
3) ปัญญดา พินไธสงค์ และคณะ [35] พัฒนาแอปพลิเคชันไลน์และแชตบอตระบบรับเข้าศึกษาต่อมหาวิทยาลัยกาฬสินธุ์ ด้วย LINE Messaging API และ Dialogflow ประกอบด้วย 5 เมนูหลัก ผล

ประเมินประสิทธิภาพอยู่ในระดับมาก (ค่าเฉลี่ย 4.29) และความพึงพอใจอยู่ในระดับมาก (ค่าเฉลี่ย 4.11)

2.7.4 แชตบอต LINE ที่ใช้ Gemini จำแนกเจตนา

สิปปกร พงศ์เดโชธนบดี และคณะ [36] พัฒนาแชตบอตบน LINE เพื่อสื่อสารและให้บริการข้อมูลแก่ลูกค้าของบริษัทเอกชน โดยใช้ Google Apps Script เป็น webhook ใช้ Google Sheets เป็นฐานข้อมูลสินค้า และใช้ Gemini 2.5 Flash-Lite วิเคราะห์เจตนาของลูกค้าและสร้างคำตอบ โดยดึงข้อมูลสินค้าจาก Google Sheets ส่งไปให้ Gemini พร้อมคำสั่งระบบที่กำหนดให้ตอบจากข้อมูลที่ให้เท่านั้น ผลการประเมินความพึงพอใจอยู่ในระดับมากที่สุด (ค่าเฉลี่ย 4.54) งานนี้มีเทคโนโลยีใกล้เคียงกับโครงงานมากที่สุด แต่มีความแตกต่างสำคัญคือ งานดังกล่าวส่งข้อมูลจากฐานข้อมูลไปให้ LLM ซึ่งเหมาะกับข้อมูลสินค้าที่เปิดเผยได้ ขณะที่โครงงานนี้ส่งเฉพาะข้อความคำถามเพราะข้อมูลคดีเป็นข้อมูลที่ต้องคุ้มครอง

2.7.5 สรุปเปรียบเทียบงานวิจัยที่เกี่ยวข้อง

ตารางที่ 2.4 เปรียบเทียบงานวิจัยที่ทบทวนในหัวข้อ 2.7.2–2.7.4 กับโครงงานนี้ ในด้านแหล่งข้อมูล ช่องทาง การประมวลผลภาษา ข้อมูลที่ส่งให้ AI ภายนอก ผู้ใช้เป้าหมาย และลักษณะข้อมูล

ตารางที่ 2.4 เปรียบเทียบงานวิจัยที่เกี่ยวข้องกับโครงงาน

งาน	แหล่งข้อมูล	ช่องทาง	การประมวลผลภาษา	ส่งข้อมูลให้ AI ภายนอก	ผู้ใช้เป้าหมาย	ลักษณะข้อมูล
[32]	Googl	Looker	–	–	ผู้บริหาร	ข้อมูลการ

งาน	แหล่งข้อมูล	ช่องทาง	การประมวลผลภาษา	ส่งข้อมูลให้ AI ภายนอก	ผู้ใช้เป้าหมาย	ลักษณะข้อมูล
	e Sheets / Forms	Studio, Google Sites			หน่วยงาน	บริหารหน่วยงานสุขภาพ
[33]	Google Sheets	LINE	Dialogflow	ข้อความคำถาม	ผู้สนใจหลักสูตร	ข้อมูลหลักสูตร (เปิดเผยได้)
[34]	Google Sheets	LINE	ข้อความคำสั่ง	–	บุคลากรในมหาวิทยาลัย	ข้อมูลแจ้งซ่อม
[35]	–	LINE	Dialogflow	ข้อความคำถาม	นักเรียนผู้สนใจสมัคร	ข้อมูลรับสมัคร (เปิดเผยได้)
[36]	Google Sheets	LINE	Gemini	ข้อความคำถาม และข้อมูลจากฐานข้อมูล	ลูกค้า	ข้อมูลสินค้า (เปิดเผยได้)
โครงงานนี้	Google Sheets	Dashboard (HtmlService) และ LINE	Gemini (จำแนกเจตนาเท่านั้น)	เฉพาะข้อความคำถาม	เจ้าหน้าที่ที่มีรายชื่อ (Dashboard) และสมาชิกกลุ่ม LINE เดียว (บอต)	ข้อมูลคดีส่งตรวจ DNA (ต้องคุ้มครอง)

หมายเหตุ: “–” หมายถึงงานนั้นไม่ได้ใช้หรือไม่ได้ระบุไว้ในบทความ ข้อมูลในตารางสรุปจากบทคัดย่อและส่วนวิธีดำเนินการของแต่ละบทความ

จากการทบทวนพบว่า งานที่ใช้ LINE ร่วมกับ Google Apps Script และ Google Sheets ในประเทศไทยส่วนใหญ่เป็นบริการข้อมูลที่เปิดเผยได้ และยังไม่พบงานที่ประยุกต์กับการติดตามคดีส่งตรวจทางนิติวิทยาศาสตร์ซึ่งมีข้อมูลที่ต้องคุ้มครองตาม PDPA โครงงานนี้จึงมุ่งผสานการแสดงผลด้วย Dashboard การสอบถามผ่านแชตบอต และการใช้ LLM อย่างจำกัด ภายใต้หลักการให้สิทธิ์น้อยที่สุดและการใช้ข้อมูลเท่าที่จำเป็น

บทที่ 3
การออกแบบระบบ

บทนี้อธิบายการออกแบบระบบ Dashboard และแชตบอต AI สำหรับติดตามและสรุปสถานะคดีส่งตรวจพิสูจน์ DNA ประกอบด้วย สถาปัตยกรรมระบบ โครงสร้างข้อมูล การออกแบบส่วนแสดงผล Dashboard การควบคุมสิทธิ์ การออกแบบแชตบอต LINE การออกแบบรายงานสรุปอัตโนมัติ และการออกแบบด้านความปลอดภัยและการคุ้มครองข้อมูลส่วนบุคคล โครงงานนี้เป็นงานด้านซอฟต์แวร์ทั้งหมด ไม่มีส่วนฮาร์ดแวร์ ภาพหน้าจอและตัวอย่างข้อความทั้งหมดในบทนี้สร้างจากข้อมูลจำลอง ไม่มีข้อมูลคดีจริง

3.1 ภาพรวมและสถาปัตยกรรมระบบ

ระบบออกแบบให้ทำงานบนบริการของ Google ทั้งหมด โดยไม่มีค่าใช้จ่าย และไม่ต้องมีเครื่องแม่ข่ายของหน่วยงานเอง Dashboard แชตบอต LINE และรายงานสรุปอัตโนมัติอยู่ในโปรเจกต์ Google Apps Script เดียวกัน (เจ้าของโครงงานเป็นผู้ตัดสินใจ) เพื่อใช้ไฟล์ `Stats.gs` ซึ่งเป็นฟังก์ชันคำนวณสถิติร่วมกัน แต่ deploy เป็นเว็บแอป 2 รายการที่ตั้งค่าต่างกัน (หัวข้อ 3.4.8) องค์ประกอบหลักมี 5 ส่วน ดังนี้

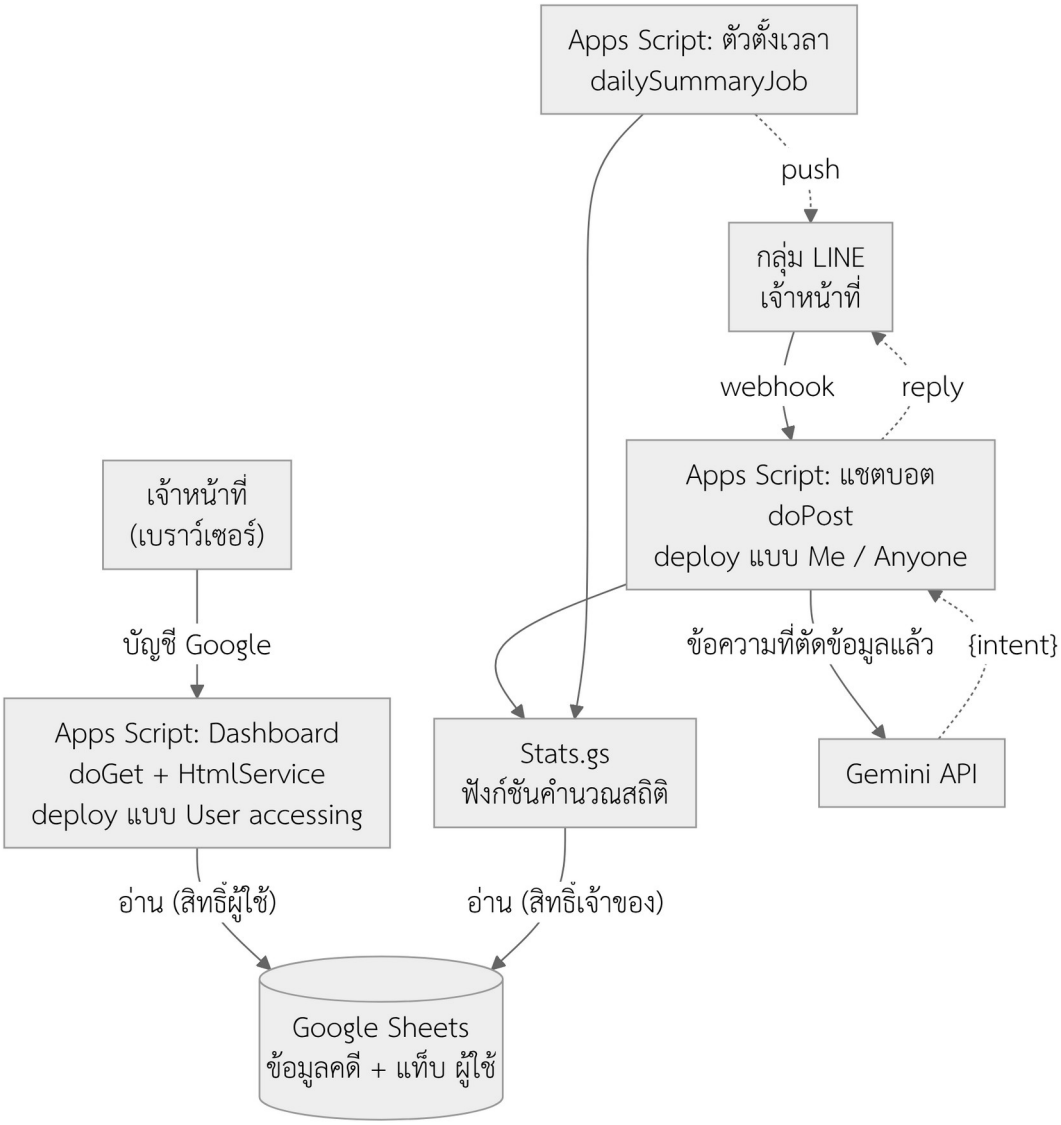
1) **Google Sheets (แหล่งเก็บข้อมูล)** เป็นที่เก็บข้อมูลคดีเพียงแห่งเดียวของระบบ เจ้าหน้าที่ยังคงบันทึกข้อมูลลงตารางในรูปแบบเดิม ระบบเพิ่มเพียงแท็บ “ผู้ใช้” สำหรับควบคุมสิทธิ์ (หัวข้อ 3.2) ไฟล์ไม่เปิดให้

“ทุกคนที่มีลิงก์” อ่านได้ แต่แชร์แบบผู้มีสิทธิ์อ่าน (Viewer) เฉพาะบัญชีที่อยู่ในแท็บ “ผู้ใช้” (หัวข้อ 3.4)

2) **Dashboard (เว็บแอป รายการที่ 1)** หน้าเว็บแสดงผลเสิร์ฟผ่าน `HtmlService` ของ Google Apps Script โดยมีฟังก์ชัน `doGet` เป็นจุดรับคำขอ [15] ข้อมูลอ่านจาก Sheet ฝั่งเซิร์ฟเวอร์ด้วย `SpreadsheetApp` แทนการให้เบราว์เซอร์ดึงข้อมูลเอง จึงไม่ต้องเปิด Sheet เป็นสาธารณะ และตรวจสิทธิ์ผู้ใช้ทุกครั้งที่ขอข้อมูล (หัวข้อ 3.4)
3) **แชตบอต LINE (เว็บแอป รายการที่ 2)** รับข้อความจากกลุ่ม LINE ของเจ้าหน้าที่ผ่าน LINE Messaging API ซึ่งส่งเหตุการณ์ (webhook event) มาเป็นคำขอ HTTP POST [37] ปลายทางของ webhook คือฟังก์ชัน `doPost` ใน `LineBot.gs` ซึ่ง deploy แยกจาก Dashboard แบบ Execute as: Me และ Who has access: Anyone เพราะ LINE Platform เรียก webhook โดยไม่มีบัญชี Google (หัวข้อ 3.4.8 และ 3.5)
4) **ตัวตั้งเวลา (time-driven trigger)** ตัวตั้งเวลารายวันของ Apps Script เรียกฟังก์ชัน `dailySummaryJob` ใน `Summary.gs` ทุกเช้า ฟังก์ชันตรวจว่าวันนั้นมีงวดสรุปรายเดือนหรือรายปีที่ต้องส่งหรือไม่ แล้วส่งข้อความเข้ากลุ่ม LINE ด้วย push message ตัวตั้งเวลากำหนดชั่วโมงและเขตเวลาได้ [38] (หัวข้อ 3.6)

5) **Gemini API (เฉพาะการแยกเจตนา)** ใช้เป็นตัวช่วยเฉพาะเมื่อผู้ใช้เรียกบอตแล้วแต่กฎที่เขียนไว้ในโค้ดเข้าใจคำถามไม่ได้ ระบบส่งให้ Gemini **เฉพาะข้อความคำถามที่ตัดตัวเลขทั้งหมด ชื่อที่มีคำนำหน้า อีเมล ลิงก์ การกล่าวถึงผู้ใช้ และชื่อหน่วยออกแล้ว** และรับกลับเพียงเจตนา (intent) เลขคดีดึงด้วยโค้ดของระบบเอง ไม่ส่งข้อมูลคดีจาก Sheet ไปยังบริการ AI (หัวข้อ 3.5.2)

ความสัมพันธ์ขององค์ประกอบทั้ง 5 ส่วนและทิศทางของข้อมูลแสดงในรูปที่ 3.1 เส้นทึบคือการส่งคำขอหรืออ่านข้อมูล เส้นประคือข้อความหรือผลที่ส่งกลับ ค่าความลับ เช่น channel access token ของ LINE และ API key ของ Gemini เก็บใน Script Properties ของโปรเจกต์ [13] จึงไม่ปรากฏในรูป



รูปที่ 3.1 สถาปัตยกรรมระบบและเส้นทางการไหลของข้อมูล

3.1.1 การไหลของข้อมูล

หลักการสำคัญของสถาปัตยกรรมคือ **ข้อมูลคดีไม่ออกนอก Google Apps Script ไปยังบริการ AI** เส้นทางข้อมูลแบ่งได้ 3 กรณี

1) **Dashboard** ผู้ใช้เปิดลิงก์เว็บแอปด้วยบัญชี Google สคริปต์ตรวจสิทธิ์ แล้วอ่านข้อมูลจาก Sheet ฝั่งเซิร์ฟเวอร์ด้วยสิทธิ์ของผู้ใช้คนนั้น และส่งเฉพาะ

คอลัมน์ที่ Dashboard ใช้กลับไปยังเบราว์เซอร์ เบราว์เซอร์เป็นผู้คำนวณสถิติและวาดกราฟ (หัวข้อ 3.4)

2) **คำถามในกลุ่ม LINE** ข้อความเดินทางจาก LINE Platform มาที่ `doPost` ระบบตรวจรหัสลับใน URL และรหัสกลุ่มก่อน แล้วตรวจว่าข้อความนั้นเป็นการเรียกบอตหรือไม่ ข้อความสนทนาทั่วไปถูกทิ้งโดยไม่อ่าน Sheet และไม่เรียก Gemini ถ้าเป็นการเรียกบอต โค้ดแยกเจตนาและดึงเลขคดีด้วยกฎของตัวเองก่อน และเรียก Gemini เฉพาะเมื่อกฎไม่เข้าใจ โดยส่งข้อความที่ตัดข้อมูลแล้วเท่านั้น ข้อมูลที่ได้กลับมามีเพียงเจตนา จากนั้นโค้ดของระบบเป็นผู้ค้นข้อมูลใน Sheet และประกอบข้อความตอบเอง ข้อความตอบส่งด้วย reply token จึงไปถึงเฉพาะกลุ่มที่ถาม และไม่อยู่ในคำตอบ HTTP ของ `doPost`

3) **สรุปตามเวลา** ตัวตั้งเวลาเรียกฟังก์ชันคำนวณสถิติแล้วส่งข้อความสรุปที่เป็นตัวเลขรวม (ไม่มีเลขคดีหรือรายละเอียดรายคดี) เข้ากลุ่ม LINE ขั้นตอนนี้ไม่ใช้ AI

เหตุผลที่ไม่ส่งข้อมูลคดีให้ AI คือ ข้อกำหนดเพิ่มเติมของ Gemini API (มีผลวันที่ 23 มีนาคม ค.ศ. 2026) ระบุว่าข้อมูลที่ส่งผ่านบริการแบบไม่เสียค่าใช้จ่าย (Unpaid Services) อาจถูกนำไปปรับปรุงผลิตภัณฑ์และอาจมีผู้ตรวจที่เป็นมนุษย์อ่าน รวมถึงห้ามส่งข้อมูลอ่อนไหว ข้อมูลลับ หรือข้อมูลส่วนบุคคลเข้าสู่บริการดังกล่าว [8] การออกแบบให้ AI เห็นเพียงข้อความคำถามที่ตัดข้อมูลออกแล้วจึงลดความเสี่ยงนี้ แม้เลขคดีเพียงอย่างเดียวจะไม่บอกผลการ

ตรวจ เจ้าของโครงงานตัดสินใจไม่ส่งเลขคดีให้ Gemini เพื่อไม่ให้ข้อความที่ผู้ตรวจของ Google อาจอ่านเชื่อมโยงกลับไปยังคดีใดคดีหนึ่งได้

3.2 การออกแบบโครงสร้างข้อมูล

ระบบใช้โครงสร้างตารางเดิมที่เจ้าหน้าที่ใช้อยู่ เพื่อไม่ให้เปลี่ยนวิธีทำงาน โครงสร้างได้จากการอ่านชื่อคอลัมน์ของไฟล์ต้นฉบับ (ไม่ได้คัดลอกค่าข้อมูล) และบันทึกไว้ที่ `docs/data-schema.md` ข้อมูลจำลองสำหรับพัฒนาและทดสอบสร้างให้มีโครงสร้างเดียวกันทุกคอลัมน์

3.2.1 แท็บข้อมูลคดี

แท็บ “ข้อมูลคดี” เป็นตารางหลัก หนึ่งแถวคือหนึ่งคดี คอลัมน์แรกต้องเป็น “เลข บก.” ตารางที่ 3.1 แสดงคอลัมน์ตามลำดับ

ตารางที่ 3.1 คอลัมน์ในแท็บข้อมูลคดี

คอลัมน์	ชนิดข้อมูล	ความหมาย / การใช้งาน
เลข บก.	ข้อความ	เลขคดีรูปแบบ ลำดับ/ปี พ.ศ. เช่น `123/2568` ใช้เป็นคีย์ที่แชตบอตค้นหา
เลขรายงาน	ข้อความ	รูปแบบ `NN-YY-NNNNN`
สภ. (ท้องที่)	ข้อความ	สถานีตำรวจที่ส่งตรวจ
จังหวัด	ข้อความ	จังหวัดของสถานีที่ส่งตรวจ
เลขหนังสือนำส่ง	ข้อความ	เลขที่หนังสือนำส่งของกลาง
ลงวันที่	วันที่	วันที่ของหนังสือนำส่ง
วันที่รับ	วันที่	วันที่รับคดี ใช้กำหนดเดือนและปีของคดีในสถิติและสรุป

คอลัมน์	ชนิดข้อมูล	ความหมาย / การใช้งาน
วันที่ตรวจเสร็จ	วันที่	ใช้คำนวณระยะเวลาตรวจ
วันที่มารับผล	วันที่	ใช้คำนวณระยะเวลารอรับผล
สถานะ	ข้อความ	ค่าที่กำหนด 3 ค่า (หัวข้อ 3.2.3)
ประเภทคดี (เลือก)	ข้อความ (dropdown)	เช่น ทรัพย์ พ.ร.บ.อาวุธปืน เพศ
ผู้ตรวจ (เลือก)	ข้อความ (dropdown)	ชื่อเจ้าหน้าที่ผู้ตรวจ — ข้อมูลส่วนบุคคล
ของกลาง (ชิ้น)	ตัวเลข	จำนวนของกลางของคดี
เดือนที่รับ	วันที่	คอลัมน์คำนวณ
ปี พ.ศ. (รับ)	ตัวเลข	คอลัมน์คำนวณ
นับในสถิติ	ตัวเลข 0/1	คอลัมน์คำนวณ แถวที่เป็น 0 ไม่นับในสถิติ
ข้อสังเกต (ผิดปกติ)	ข้อความ	คอลัมน์คำนวณ ระบุเหตุผลที่แถวควรตรวจสอบ
ปีเลขรายงาน, ลำดับเลขรายงาน	ตัวเลข	คอลัมน์คำนวณ
อันดับ (ช่วยคำนวณ), ตั้งค่า, เกณฑ์ของกลางผิดปกติ (ชิ้น)	—	ช่วยคำนวณและตั้งค่า เช่น เกณฑ์จำนวนของกลางที่ถือว่าสูงผิดปกติ

ไฟล์ต้นฉบับเดิมมีคอลัมน์ “User ID (LINE)” ซึ่งเป็นข้อมูลส่วนบุคคลและไม่จำเป็นต่อการติดตามคดี เจ้าของโครงงานจึงลบคอลัมน์นี้ออกจาก Sheet จริงตามหลักการใช้ข้อมูลเท่าที่จำเป็น (data minimisation) ระบบไม่อ่านและไม่พึ่งคอลัมน์นี้

ในไฟล์เดิมยังมีแท็บสรุปอื่น ได้แก่ “Dashboard” “สภ.และประเภทคดี” และ “ผู้ตรวจ” ซึ่งเป็นสูตรสรุปของ Excel ระบบนี้ไม่อ่านแท็บเหล่านี้ แต่คำนวณสถิติใหม่จากแท็บข้อมูลคดีโดยตรง

นอกจากข้อมูลรายคดี Dashboard ต้นแบบยังรองรับไฟล์ข้อมูลห้องปฏิบัติการ (แท็บ “ข้อมูลเคส” “ข้อมูลงบ” “ข้อมูล Lot” และ “หมายเหตุ”) ซึ่งเป็นสถิติรายเดือนระดับห้องปฏิบัติการ ไม่มีข้อมูลรายคดี รายละเอียดคอลัมน์อยู่ใน `docs/data-schema.md`

3.2.2 แท็บผู้ใช้

แท็บ “ผู้ใช้” เป็นแท็บที่เพิ่มขึ้นใหม่ ใช้เป็นรายชื่อผู้มีสิทธิ์เปิด Dashboard มีคอลัมน์ดังตารางที่ 3.2

ตารางที่ 3.2 คอลัมน์ในแท็บผู้ใช้

คอลัมน์	ชนิดข้อมูล	ความหมาย
อีเมล	ข้อความ	อีเมลบัญชี Google ที่อนุญาตให้เปิด Dashboard
ชื่อ	ข้อความ	ชื่อผู้ใช้ เพื่อให้ผู้ดูแลจำได้
หมายเหตุ	ข้อความ	เช่น ตำแหน่ง หรือวันที่เพิ่มสิทธิ์

การเพิ่มหรือถอนสิทธิ์ทำได้โดยแก้แถวในแท็บนี้ ไม่ต้องแก้โค้ด

3.2.3 ค่าสถานะคดี

คอลัมน์สถานะมีค่าที่กำหนด 3 ค่า เรียงตามลำดับงาน

1) **กำลังดำเนินการ** รับคดีแล้ว ยังตรวจไม่เสร็จ
2) **เสร็จแล้ว รอรับผล** ตรวจเสร็จแล้ว รอพนักงานสอบสวนมารับผล

3) **รับผลไปแล้ว** ปิดงานของคดี

“คดีค้าง” ในระบบหมายถึงคดีที่ยังไม่อยู่ในสถานะ “รับผลไปแล้ว” (เฉพาะแถวที่นับในสถิติ) และ**รายงานแยกเป็น 2 กลุ่มเสมอ** คือ “กำลังดำเนินการ” (งานที่ยังอยู่ในห้องปฏิบัติการ) และ “เสร็จแล้ว รอรับผล” (งานที่รอหน่วยงานผู้ส่งมารับ) เพราะทั้งสองกลุ่มมีความหมายต่อการบริหารต่างกัน การจัดกลุ่มใช้กติกาเดียวกับ Dashboard (เมนูระยะเวลาและงานค้าง) ดังนี้

1) สถานะ “รับผลไปแล้ว” ไม่นับเป็นคดีค้าง
2) สถานะที่มีคำว่า “รอรับผล” นับเป็นกลุ่ม “เสร็จแล้ว รอรับผล”
3) สถานะอื่นทั้งหมด **รวมถึงช่องสถานะว่างหรือสะกดต่างจาก 3 ค่าข้างต้น** นับเป็นกลุ่ม “กำลังดำเนินการ” เพื่อไม่ให้คดีที่ข้อมูลไม่ครบหายไปจากรายการคดีค้าง

ฟังก์ชัน `Stats.pendingCases` ใน `Stats.gs` คืนจำนวนแยกตามกลุ่ม จำนวนแถวที่ช่องสถานะว่าง และรายการคดีค้างเรียงจากวันที่รับเก่าที่สุด ส่วน `Stats.pendingByStatus` ซึ่งแชตบอตใช้ (หัวข้อ 3.5.3) คืนจำนวนของแต่ละกลุ่มพร้อมเลข บก. ที่ค้างนานที่สุดกลุ่มละ 3 คดี โดยกลุ่ม “กำลังดำเนินการ” นับจำนวนวันจากวันที่รับ และกลุ่ม “เสร็จแล้ว รอรับผล” นับจากวันที่ตรวจเสร็จ (ถ้าไม่มีวันที่ตรวจเสร็จใช้วันที่รับแทน) ข้อความตอบของแชตบอตบอกจำนวนแถวที่ช่องสถานะว่างไว้ด้วย เพื่อให้เจ้าหน้าที่ตามกรอกข้อมูล

3.2.4 ข้อมูลส่วนบุคคลและการจัดการ

คอลัมน์ที่เป็นข้อมูลส่วนบุคคลและวิธีจัดการแสดงในตารางที่ 3.3

ตารางที่ 3.3 คอลัมน์ที่เป็นข้อมูลส่วนบุคคลและการจัดการ

คอลัมน์	การจัดการในระบบ
ผู้ตรวจ (เลือก)	แสดงใน Dashboard เฉพาะผู้มีสิทธิ์ ในสรุปอัตโนมัติแสดงเป็นจำนวนคดีต่อคน แยกตามประเภท ไม่แสดงเลขคดี ไม่ส่งให้ AI
User ID (LINE)	ลบออกจาก Sheet จริงแล้ว (หัวข้อ 3.2.1) ถ้ายังมีอยู่ในไฟล์ใด ระบบก็ไม่ส่งออกจากเซิร์ฟเวอร์ เพราะเลือกส่งเฉพาะคอลัมน์ที่กำหนด (หัวข้อ 3.4.5) และ Dashboard ต้นแบบกรองคอลัมน์ที่ชื่อเข้าข่าย เช่น “line” “user id” “เบอร์” “โทร” “บัตร” “เลขประจำตัว” ออกตั้งแต่ขั้นอ่านไฟล์
อีเมลและชื่อในแท็บผู้ใช้	ใช้ตรวจสิทธิ์ฝั่งเซิร์ฟเวอร์เท่านั้น ไม่แสดงบนหน้า Dashboard
`userId` ของผู้ส่งข้อความใน LINE	มากับ webhook event ระบบไม่เก็บ ไม่บันทึก และไม่ส่งให้ Gemini (หัวข้อ 3.7)

ข้อมูลที่ใช้พัฒนาและทดสอบเป็นข้อมูลจำลองทั้งหมด เช่น ชื่อเจ้าหน้าที่ “เจ้าหน้าที่ ก” ถึง “เจ้าหน้าที่ จ” สถานี “สภ.ตัวอย่าง1” ถึง “สภ.ตัวอย่าง15” และอีเมลโดเมน `@example.com` ข้อมูลจำลองสร้างด้วยสคริปต์ `tools/make_mock_data.py` ซึ่งกำหนด seed คงที่ ทำให้สร้างซ้ำได้ผลเหมือนเดิมทุกครั้ง ระบบไม่มีคอลัมน์และไม่แสดง DNA profile

ข้อสังเกตด้านคุณภาพข้อมูล ณ วันที่ได้รับโครงสร้างไฟล์ คอลัมน์ “ประเภทคดี (เลือก)” และ “ผู้ตรวจ (เลือก)” ยังกรอกไว้เพียงส่วนน้อย สถิติแยกประเภทและแยกผู้ตรวจจึงออกแบบให้แสดงกลุ่ม “ไม่ระบุ” ไว้ชัดเจน แทนการตัดแถวทิ้ง

3.3 การออกแบบส่วนแสดงผล Dashboard

Dashboard ต้นแบบพัฒนาเป็นไฟล์ HTML ไฟล์เดียว (`dashboard/index.html`) วาดกราฟด้วย SVG ที่เขียนเอง ไม่พึ่งไลบรารีกราฟภายนอก และใช้ไลบรารี JSZip อ่านไฟล์ .xlsx ในเบราว์เซอร์ หน้าเว็บนี้ย้ายไปเสิร์ฟผ่าน Apps Script `HtmlService` แล้ว (`apps-script/Index.html` สร้างจากต้นแบบด้วยสคริปต์ `apps-script/test/build_index.py`) โดยเปลี่ยนแหล่งข้อมูลเป็นการขอข้อมูลจากเซิร์ฟเวอร์ (หัวข้อ 3.4) และตัดปุ่มเพิ่มลิงก์ Google Sheets ออก การจัดหน้า เมนู และกราฟที่อธิบายต่อไปนี้ใช้ร่วมกันทั้งสองรุ่น

3.3.1 โครงหน้าจอและเมนู

หน้าจอแบ่งเป็นแถบเมนูด้านซ้ายและพื้นที่เนื้อหาด้านขวา ส่วนหัวมีชื่อ Dashboard ตัวเลือกแหล่งข้อมูล ป้ายสถานะการเชื่อมต่อ (เช่น “เชื่อมสด · Google Sheet · อัปเดต …”) และปุ่มสลับโหมดมืด/สว่าง ใต้ส่วนหัวมีแถบปุ่มเลือกปี (chip) สำหรับกรองข้อมูล Dashboard ตรวจรูปแบบข้อมูลจากชื่อแท็บและหัวคอลัมน์ แล้วเลือกชุดเมนูให้ตรงกับข้อมูลโดยอัตโนมัติ ดังตารางที่ 3.4 โครงหน้าจอแสดงในรูปที่ 3.2

ตารางที่ 3.4 เมนูของ Dashboard แยกตามรูปแบบข้อมูล

รูปแบบข้อมูล	เมนู	ตัวกรองปี
คดีส่งตรวจ DNA (รายคดี)	ภาพรวม, พื้นที่และประเภทคดี, ประเภท/ผู้ตรวจรายเดือน, ระยะเวลาและงานค้าง, ตรวจสอบข้อมูล	ปีที่รับคดี (พ.ศ.) หรือ “ทั้งหมด”
งานตรวจ · Lot · งบประมาณ	ภาพรวม, งบประมาณ, Lot พัสดุ	ปีงบประมาณ (พ.ศ.) นับ ต.ค. – ก.ย.
ทุกรูปแบบ	แหล่งข้อมูล	—

สถิติรายคดีนับตามวันที่รับคดี และนับเฉพาะแถวที่คอลัมน์ “นับในสถิติ” ไม่เป็น 0

[รอใส่ภาพ: รูปที่ 3.2]

โครงหน้าจอ Dashboard แถบเมนูด้านซ้าย ส่วนหัว และแถบเลือกปี (ภาพจากข้อมูลจำลอง)

รูปที่ 3.2 โครงหน้าจอ Dashboard แถบเมนูด้านซ้าย ส่วนหัว และแถบเลือกปี

3.3.2 หน้าภาพรวม (ข้อมูลคดี)

หน้าภาพรวมมีการ์ดตัวชี้วัด (KPI card) 6 ใบ ได้แก่ คดีทั้งหมด, ของกลางรวม (ชิ้น), ของกลางเฉลี่ยต่อคดี (พร้อมค่ามัธยฐาน), คดีเดือนล่าสุด (เทียบกับเดือนก่อน), แถวที่ควรตรวจสอบ และอยู่ระหว่างดำเนินการ (พร้อมจำนวนที่เสร็จแล้ว รอรับผล) ถ้ามีคดีที่ของกลางเกินเกณฑ์ที่ตั้งไว้ในชีต ระบบแสดงแถบหมายเหตุด้านบน

ดังแสดงในรูปที่ 3.3 ส่วนกราฟและตารางประกอบด้วย กราฟแท่งจำนวนคดีรายเดือน กราฟแท่งของกลางรายเดือน (ชิ้น) รายการจุดเด่น (เดือนที่มีคดีมากที่สุด คดีเฉลี่ยต่อเดือน จังหวัดที่ส่งตรวจมากที่สุด และมัธยฐานระยะเวลารับถึงตรวจเสร็จ) ตารางสถิติรายปีที่คลิกแถวเพื่อกรองปีได้ ตารางสถิติรายเดือน และตาราง 10 คดีที่มีของกลางมากที่สุด

[รอใส่ภาพ: รูปที่ 3.3]

หน้าภาพรวมของ Dashboard ข้อมูลคดี (ภาพจากข้อมูลจำลอง)

รูปที่ 3.3 หน้าภาพรวมของ Dashboard ข้อมูลคดี

3.3.3 หน้าพื้นที่และประเภทคดี

มีการ์ดตัวชี้วัด 4 ใบ ได้แก่ จำนวนจังหวัดที่ส่งตรวจ จำนวน สภ. ที่ส่งตรวจ คดีที่ระบุประเภทแล้ว และคดีที่ระบุผู้ตรวจแล้ว (แสดงร้อยละของคดีทั้งหมด) ตามด้วยกราฟแท่งแนวนอน 4 ชุด คือ จำนวนคดีแยกตามจังหวัด, สภ. ที่ส่งตรวจมากที่สุด 15 อันดับ, ประเภทคดี และผู้ตรวจ ดังแสดงในรูปที่ 3.4

[รอใส่ภาพ: รูปที่ 3.4]

หน้าพื้นที่และประเภทคดี (ภาพจากข้อมูลจำลอง)

รูปที่ 3.4 หน้าพื้นที่และประเภทคดี

3.3.4 หน้าประเภท/ผู้ตรวจรายเดือน

หน้านี้มีตัวเลือกเดือนที่รับคดี และตัวเลือกซ่อนคดีที่ไม่ระบุประเภทหรือผู้ตรวจ องค์ประกอบได้แก่ กราฟแท่งซ้อนประเภทคดีรายเดือน (แสดง 5 ประเภทที่มากที่สุด ที่เหลือรวมเป็นกลุ่มที่หน้าจอแสดงว่า “อื่นๆ”) ตารางประเภทคดีที่มากที่สุดในแต่ละเดือน ตารางจำนวนคดีแยกประเภทรายเดือน ตารางไขว้จังหวัด × ประเภทคดี และผู้ตรวจ × ประเภทคดี ที่คลิกแถวเพื่อดูรายการคดีได้ และตารางไขว้จังหวัด × เดือน กับผู้ตรวจ × เดือน ถ้าคดีที่ระบุประเภทหรือผู้ตรวจมีน้อยกว่าครึ่งหนึ่ง ระบบแสดงกล่องเตือนว่าข้อมูลส่วนนี้ยังน้อย หน้าจอนี้แสดงในรูปที่ 3.5

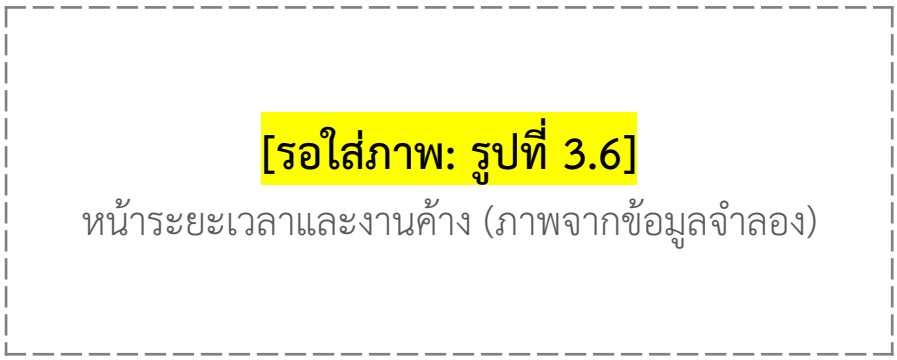
[รอใส่ภาพ: รูปที่ 3.5]

หน้าประเภทคดีและผู้ตรวจรายเดือน (ภาพจากข้อมูลจำลอง)

รูปที่ 3.5 หน้าประเภทคดีและผู้ตรวจรายเดือน

3.3.5 หน้าระยะเวลาและงานค้าง

มีการ์ดตัวชี้วัด 4 ใบ ได้แก่ ระยะเวลาเฉลี่ยจากรับถึงตรวจเสร็จ (วัน) ค่ามัธยฐาน (พร้อมค่าเร็วสุดและนานสุด) มัธยฐานระยะเวลาจากตรวจเสร็จถึงมารับผล และคดีที่ยังไม่รับผล ส่วนกราฟประกอบด้วย กราฟเส้นระยะเวลาตรวจเฉลี่ยรายเดือน แถบสถานะคดีทั้ง 3 ค่า และการกระจายของระยะเวลาตรวจเป็นช่วง (ไม่เกิน 14 วัน, 15–21, 22–28, 29–35 และมากกว่า 35 วัน) ด้านล่างเป็นตารางคดีที่ยังไม่รับผล เรียงจากค้างนานที่สุด กรองตามสถานะได้ และแสดงทีละ 25 แถวพร้อมปุ่มแสดงเพิ่ม ดังแสดงในรูปที่ 3.6

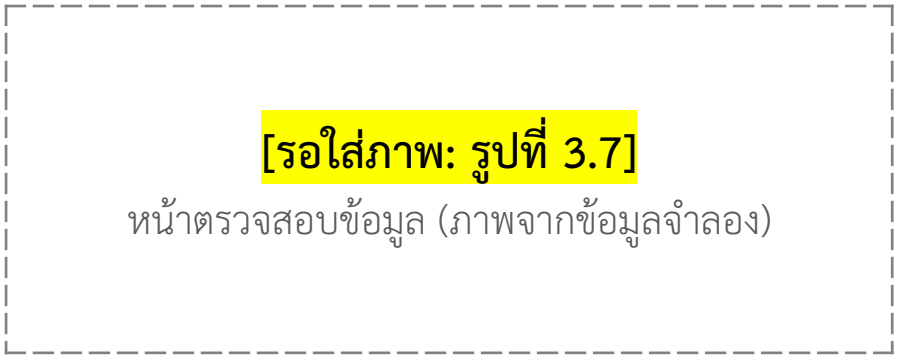


รูปที่ 3.6 หน้าระยะเวลาและงานค้าง

3.3.6 หน้าตรวจสอบข้อมูลและหน้าข้อมูลห้องปฏิบัติการ

หน้าตรวจสอบข้อมูลรวบรวมแถวที่คอลัมน์ “ข้อสังเกต (ผิดปกติ)” มีค่า แสดงการ์ดตัวชี้วัด กราฟสรุปตามเหตุผล และตารางรายการที่ควรตรวจสอบกับต้นฉบับ หน้านี้ช่วยให้เจ้าหน้าที่แก้ข้อมูลต้นทางก่อนนำตัวเลขไปใช้ ดังแสดงในรูปที่ 3.7

สำหรับข้อมูลห้องปฏิบัติการ หน้าภาพรวมแสดงตัวชี้วัด 6 ใบ (ตัวอย่างตรวจ ของกลาง เฉลี่ยตัวอย่างต่อเดือน งบประมาณรวม บาทต่อตัวอย่าง และ Lot พัสดุ) พร้อมกราฟเทียบปีงบประมาณก่อนหน้า หน้างบประมาณแสดงกราฟแท่งซ้อนงบรายปีแยกหมวดและกราฟเส้นต้นทุนวัสดุตรวจต่อตัวอย่าง หน้า Lot พัสดุแสดงจำนวนตัวอย่างรายรอบ 26–25 ของเดือน และเทียบกับรายงานรายเดือน



รูปที่ 3.7 หน้าตรวจสอบข้อมูล

3.3.7 หลักการออกแบบส่วนแสดงผล

1) **ภาษาไทยทั้งหมด** ชื่อเมนู หัวกราฟ คำอธิบาย และข้อความแจ้งเตือนเป็นภาษาไทย ปีแสดงเป็น พ.ศ.
2) **รองรับอุปกรณ์เคลื่อนที่** ที่ความกว้างไม่เกิน 900 พิกเซล เมนูย้ายจากแถบซ้ายเป็นแถบเลื่อนแนวนอนด้านบน การ์ดตัวชี้วัดลดเหลือ 3 และ 2 คอลัมน์ตามความกว้างจอ และบนจอสัมผัสปุ่มมีความสูงอย่างน้อย 44 พิกเซล
3) **โหมดมืด** ใช้ตามการตั้งค่าของระบบปฏิบัติการโดยอัตโนมัติ และสลับเองได้ด้วยปุ่มที่ส่วนหัว สีทั้งหมดกำหนดเป็นตัวแปรที่มีค่าสำหรับทั้งสองโหมด
4) **ค่าที่ขาดหาย** ถ้าคอลัมน์จังหวัด สภ. ประเภทคดี หรือผู้ตรวจว่าง ระบบจัดเป็นกลุ่ม “ไม่ระบุ” ซึ่งเป็นคำเดียวกับที่แชตบอตและรายงานสรุปใช้ ยกเว้นแถวของผู้ตรวจที่ว่างในตารางผู้ตรวจ × ประเภทคดี ซึ่งแสดงว่า “ไม่ระบุผู้ตรวจ” เพื่อไม่ให้สับสนกับคอลัมน์ประเภทคดี “ไม่ระบุ” ในตารางเดียวกัน ตัวชี้วัดที่คำนวณไม่ได้แสดงเป็น “—” พร้อมเหตุผล เช่น “คำนวณเมื่อครบ 12 เดือน”
5) **อ่านได้ทั้งแบบกราฟและตาราง** การ์ดกราฟทุกใบมีปุ่ม “ดูเป็นตาราง” กราฟมีคำอธิบายสำหรับโปรแกรมอ่านหน้าจอ (`aria-label`) มีลิงก์ข้ามไปยังเนื้อหา และงดภาพเคลื่อนไหวเมื่อผู้ใช้ตั้งค่าลดการเคลื่อนไหว
6) **ข้อมูลเป็นปัจจุบัน** หน้าเว็บขอข้อมูลซ้ำตามรอบเวลา (ต้นแบบที่อ่าน Google Sheets เอง 30 วินาที รุ่น

Apps Script 120 วินาทีเท่ากับอายุแคชฝั่งเซิร์ฟเวอร์ หัวข้อ 3.4.4) และวาดใหม่เฉพาะเมื่อข้อมูลเปลี่ยน การ์ดที่ตัวเลขเปลี่ยนจะกะพริบให้เห็น

3.4 การควบคุมสิทธิ์และการทำงานฝั่งเซิร์ฟเวอร์

หัวข้อนี้อธิบายการทำงานของ Dashboard รุ่น Apps Script ตามโค้ดที่พัฒนาแล้วใน `apps-script/` ได้แก่ `Code.gs` (หน้าเว็บ การตรวจสิทธิ์ และการอ่านข้อมูล) `Stats.gs` (ฟังก์ชันคำนวณสถิติที่ใช้ร่วมกับแชตบอต) `Index.html` (หน้า Dashboard) และ `appsscript.json` (การตั้งค่าโปรเจกต์) ค่าที่ไม่ควรอยู่ในโค้ดเก็บใน Script Properties ได้แก่ `CASES_SHEET_ID` (รหัส Sheet ข้อมูลคดี) `LAB_SHEET_ID` (รหัส Sheet ข้อมูลห้องปฏิบัติการ ไม่บังคับ) และ `ADMIN_CONTACT` (ข้อความบอกผู้ไม่มีสิทธิ์ว่าให้ติดต่อใคร ไม่บังคับ)

3.4.1 ปัญหาของการ deploy แบบ “Execute as: Me”

การตรวจสิทธิ์ตามรายชื่อต้องรู้อีเมลของผู้ที่เปิด Dashboard เพื่อนำไปเทียบกับแท็บ “ผู้ใช้” Apps Script ให้ค่านี้ผ่าน `Session.getActiveUser().getEmail()` แต่เอกสารของ Google ระบุว่าเมธอดนี้คืนค่าเป็นสตริงว่างเมื่อนโยบายความปลอดภัยไม่อนุญาตให้เข้าถึงตัวตนของผู้ใช้ ซึ่งรวมถึงเว็บแอปที่ deploy แบบ “execute as me” (สคริปต์ได้รับอนุญาตจากผู้พัฒนา ไม่ใช่จากผู้ใช้) ข้อจำกัดนี้จะผ่อนลงเฉพาะเมื่อผู้พัฒนาและผู้ใช้อยู่ในโดเมน Google Workspace เดียวกัน [12] (ดูบทที่ 2 หัวข้อ 2.3.2 ข้อ 3)

ผู้ใช้ที่ใช้บัญชี Gmail ทั่วไปไม่อยู่ในโดเมน Workspace ของเจ้าของสคริปต์ ถ้า deploy แบบ Execute as: Me ระบบจะ

ได้อีเมลว่างสำหรับผู้ใช้เหล่านี้ และเนื่องจากโค้ดออกแบบให้ปฏิเสธเมื่อไม่ทราบอีเมล (fail closed) ทุกคนยกเว้นเจ้าของสคริปต์จะถูกปฏิเสธ แบบ Execute as: Me จึงใช้กับการตรวจสิทธิ์ตามรายชื่ออีเมลไม่ได้

3.4.2 การควบคุมสิทธิ์ 2 ชั้น

เจ้าของโครงงานเลือกการออกแบบต่อไปนี้ ซึ่งตั้งค่าใน `appsscript.json` และหน้าจอ deploy ดังตารางที่ 3.5

ตารางที่ 3.5 การตั้งค่า deploy ของ Dashboard

ช่องตั้งค่า	ค่าที่ใช้	ค่าใน `appsscript.json`
Execute as	User accessing the web app	`"executeAs": "USER_ACCESSING"`
Who has access	Anyone with Google account	`"access": "ANYONE"`
เขตเวลา	Asia/Bangkok	`"timeZone": "Asia/Bangkok"`

เมื่อ deploy แบบ User accessing the web app สคริปต์ทำงานในนามผู้ใช้ที่เปิดเว็บแอป และผู้ใช้ต้องผ่านหน้าขออนุญาต (OAuth consent) ก่อนใช้งาน [15] เมื่อผู้ใช้อนุญาตแล้ว Google จึงส่งอีเมลของผู้ใช้ให้สคริปต์ และสคริปต์อ่าน Sheet ด้วยสิทธิ์ของผู้ใช้คนนั้น ทำให้เกิดการป้องกัน 2 ชั้น

1) **ชั้นที่ 1 รายชื่อในแท็บ “ผู้ใช้”** ฟังก์ชัน `checkAccess_` อ่านคอลัมน์ “อีเมล” ของแท็บ “ผู้ใช้” แล้วเทียบกับอีเมลของผู้ใช้ (ตัดช่องว่างหัวท้ายและไม่สนตัวพิมพ์เล็กใหญ่) การตรวจนี้ทำทั้งตอนเปิดหน้า (`doGet`) และ**ทุกครั้งที่หน้าเว็บขอข้อมูล** (`getSources`

`getBook` และ `checkSetup` เรียก `requireAccess_` ก่อนทำงาน) ผู้ที่เรียกฟังก์ชันข้อมูลโดยตรงโดยไม่ผ่านหน้าแรกจึงได้รับข้อผิดพลาด “ไม่มีสิทธิ์เข้าถึงข้อมูล” เช่นกัน

2) **ชั้นที่ 2 การแชร์ Sheet แบบผู้มีสิทธิ์อ่าน (Viewer)** เพราะสคริปต์ทำงานในนามผู้ใช้ `SpreadsheetApp.openById` จะเปิด Sheet ได้ก็ต่อเมื่อผู้ใช้ได้รับแชร์ Sheet นั้นแล้ว ผู้ดูแลจึงแชร์ Sheet ทั้งสองไฟล์แบบ Viewer เฉพาะอีเมลที่อยู่ในแท็บ “ผู้ใช้” และตั้งการเข้าถึงทั่วไปเป็น “จำกัด” (Restricted) ผู้ที่อยู่ในรายชื่อแต่ไม่ได้รับแชร์จะเปิด Sheet ไม่ได้และถูกปฏิเสธ

การตรวจสิทธิ์ออกแบบให้ปฏิเสธเมื่อมีเงื่อนไขใดไม่ครบ (fail closed) ผลการตรวจทุกกรณีสรุปในตารางที่ 3.6 แต่ละกรณีแสดงเหตุผลเป็นภาษาไทยบนหน้า “ไม่มีสิทธิ์เข้าถึง” พร้อมอีเมลของบัญชีที่ใช้อยู่และผู้ที่ต้องติดต่อ ข้อความทุกส่วนที่มาจากภายนอก เช่น อีเมล ผ่านการแปลงอักขระพิเศษ (HTML escape) ก่อนแสดง

ตารางที่ 3.6 ผลการตรวจสิทธิ์ของ `checkAccess_`

เหตุผล (`reason`)	เงื่อนไข	ผล
—	อีเมลอยู่ในแท็บ “ผู้ใช้” และเปิด Sheet ได้	อนุญาต
`no-email`	Google ไม่ส่งอีเมลมา (เช่น deploy ผิดแบบ)	ปฏิเสธ
`wrong-deployment`	สคริปต์ทำงานในนามบัญชีอื่นที่ไม่ใช่ผู้	ปฏิเสธ

เหตุผล (reason)	เงื่อนไข	ผล
	เปิด (`Session.getEffectiveUser()` ไม่ตรงกับ `getActiveUser()`) เช่น เปิดผ่านลิงก์ deployment ของแชตบอต (หัวข้อ 3.4.8)	
`no-config`	ยังไม่ได้ตั้ง `CASES_SHEET_ID`	ปฏิเสธ
`no-sheet-access`	ผู้ใช้ไม่ได้รับแชร์ Sheet ข้อมูลคดี (ชั้นที่ 2)	ปฏิเสธ
`no-users-tab`	ไม่พบแท็บ “ผู้ใช้” หรือคอลัมน์ “อีเมล”	ปฏิเสธทุกคน
`not-listed`	อีเมลไม่อยู่ในแท็บ “ผู้ใช้” (ชั้นที่ 1)	ปฏิเสธ

ฟังก์ชันภายในทั้งหมดตั้งชื่อลงท้ายด้วยขีดล่าง (_) เพราะ Apps Script ถือว่าฟังก์ชันลักษณะนี้เป็นฟังก์ชันส่วนตัว หน้าเว็บเรียกผ่าน `google.script.run` ไม่ได้และชื่อไม่ถูกส่งไปยังเบราว์เซอร์ [39] ฟังก์ชันคำนวณใน `Stats.gs` อยู่ภายในอ็อบเจกต์ `Stats` ไม่ใช่ฟังก์ชันระดับบนสุด หน้าเว็บจึงเรียกไม่ได้เช่นกัน ฟังก์ชันที่หน้าเว็บเรียกได้มีเพียง `getSources` `getBook` และ `checkSetup` ซึ่งตรวจสิทธิ์ก่อนทุกครั้ง

3.4.3 ข้อแลกเปลี่ยนของการออกแบบ

1) **ผู้ใช้ในรายชื่อเปิด Sheet ต้นฉบับได้โดยตรง** การแชร์แบบ Viewer ทำให้ผู้ใช้เปิดไฟล์ใน Google Sheets ได้เอง และเห็นทุกแท็บทุกคอลัมน์ ไม่ใช่เฉพาะคอลัมน์ที่ Dashboard ส่งให้ การเลือกคอลัมน์ในหัวข้อ 3.4.5 จึงเป็นการลดข้อมูลที่ส่งออกไปยังเบราว์เซอร์ ไม่ใช่ขอบเขตความปลอดภัยต่อผู้ใช้ที่อยู่

ในรายชื่อ มาตรการที่ใช้คือ แชร์เฉพาะเจ้าหน้าที่ที่จำเป็นและให้สิทธิ์อ่านอย่างเดียว ลบคอลัมน์ User ID (LINE) ออกจาก Sheet จริง (หัวข้อ 3.2.1) และไม่นำข้อมูลอ่อนไหวที่ไม่จำเป็นต่อการติดตามงานมาเก็บใน Sheet นี้

2) **หน้าขออนุญาตและคำเตือนแอปที่ยังไม่ผ่านการตรวจสอบ** ผู้ใช้ทุกคนต้องกดอนุญาตครั้งแรก 1 ครั้ง และเนื่องจากโปรเจกต์ไม่ได้ส่งให้ Google ตรวจสอบ จะเห็นหน้า “Google ยังไม่ได้ยืนยันแอปนี้” ด้วยสิทธิ์ที่ขอมี 4 รายการ (หัวข้อ 3.4.4 และ 3.4.8) คู่มือติดตั้งจึงกำหนดให้ผู้ดูแลแจ้งผู้ใช้ล่วงหน้า บัญชีขององค์กรบางแห่งอาจบล็อกแอปที่ยังไม่ผ่านการตรวจสอบ ซึ่งต้องให้ผู้ดูแลระบบขององค์กรอนุญาตหรือใช้บัญชีอื่น

3) **ต้องปรับสิทธิ์ 2 ที่** การเพิ่มผู้ใช้ต้องทั้งเพิ่มอีเมลในแท็บ “ผู้ใช้” และแชร์ Sheet ทั้งสองไฟล์ การถอนสิทธิ์ต้องทั้งลบแถวและยกเลิกการแชร์ ถ้าลืมแชร์ ผลคือผู้ใช้เข้าไม่ได้ (ปลอดภัย) แต่ถ้าลบแถวแล้วลืมยกเลิกการแชร์ ผู้ใช้จะเปิด Dashboard ไม่ได้แต่ยังเปิด Sheet ต้นฉบับได้ ขั้นตอนทั้งสองจึงเขียนเป็นรายการตรวจในคู่มือ `apps-script/README.md` การลบแถวมีผลภายในประมาณ 1 นาที เพราะระบบจำผลการตรวจที่ “อนุญาต” ไว้ 60 วินาที (หัวข้อ 3.4.4)

ทางเลือกที่ไม่ต้องแชร์ Sheet คือ deploy แบบ Execute as: Me ซึ่งใช้ได้เฉพาะเมื่อผู้ใช้ทุกคนอยู่ในโดเมน Google

Workspace เดียวกับเจ้าของสคริปต์ (หัวข้อ 3.4.1) โครงงานไม่ได้เลือกแนวทางนี้เพราะผู้ใช้ที่ใช้บัญชี Gmail จะเข้าไม่ได้

3.4.4 ฟังก์ชันฝั่งเซิร์ฟเวอร์ แคช และสิทธิ์ที่ขอ

ฟังก์ชันหลักใน `Code.gs` และผู้เรียกแต่ละฟังก์ชันแสดงในตารางที่ 3.7

ตารางที่ 3.7 ฟังก์ชันหลักใน `Code.gs`

ฟังก์ชัน	ผู้เรียก	หน้าที่
`doGet()`	เบราว์เซอร์ (เปิดลิงก์ `/exec`)	ตรวจสิทธิ์ ถ้าผ่านคืนหน้า `Index.html` ถ้าไม่ผ่านคืนหน้า “ไม่มีสิทธิ์เข้าถึง” ใช้ค่า X-Frame-Options เริ่มต้น เว็บไซต์อื่นจึงฝังหน้านี้ใน iframe ไม่ได้
`getSources()`	หน้าเว็บ ผ่าน `google.script.run`	คืนอีเมลผู้ใช้ รายชื่อแหล่งข้อมูลที่ตั้งค่าไว้ (ข้อมูลคดีและข้อมูลห้องปฏิบัติการถ้ามี `LAB_SHEET_ID`) และรอบเวลารีเฟรช
`getBook(key)`	หน้าเว็บ ผ่าน `google.script.run`	คืนข้อมูลของแหล่งข้อมูล `cases` หรือ `lab` ในรูป `{names, tabs, sig, at}` โดย `sig` เป็นค่าแฮชของข้อมูลสำหรับให้หน้าเว็บรู้ว่าข้อมูลเปลี่ยนหรือไม่ ค่า `key` อื่น (เช่น `__proto__`) ถูกปฏิเสธ
`checkSetup()`	ผู้ดูแล กด Run ใน Apps Script editor	ตรวจการตั้งค่าและบอกเฉพาะจำนวนแถวของแต่ละ

ฟังก์ชัน	ผู้เรียก	หน้าที่
		แท็บ ไม่แสดงค่าข้อมูลคดี ผู้รันต้องอยู่ในแท็บ “ผู้ใช้”
`loadCaseRecords_()`	ฟังก์ชันฝั่งเซิร์ฟเวอร์ของแชตบอตและสรุปอัตโนมัติ	อ่านแท็บข้อมูลคดีแล้วแปลงเป็นรายการคดีด้วย `Stats.parseCases` **ไม่ตรวจสิทธิ์ผู้ใช้** ผู้เรียกต้องจำกัดการใช้งานเอง เช่น ตรวจรหัสลับและ `groupId` (หัวข้อ 3.5.5)

หน้าเว็บห่อ `google.script.run` ให้เป็น Promise เรียก `getSources` ครั้งเดียวตอนเปิด แล้วเรียก `getBook` ตามแหล่งข้อมูลที่เลือก จากนั้นเรียก `getBook` ซ้ำทุก 120 วินาทีขณะที่แท็บเบราว์เซอร์แสดงอยู่ (ต่ำสุด 30 วินาที) และเรียกทันทีเมื่อผู้ใช้กลับมาที่แท็บ ถ้าค่า `sig` ไม่เปลี่ยนจะไม่วาดหน้าใหม่ การคำนวณสถิติและวาดกราฟทำในเบราว์เซอร์ด้วยโค้ดชุดเดียวกับต้นแบบ ส่วน `Stats.gs` ใช้กติกาเดียวกันสำหรับแชตบอตและสรุปอัตโนมัติ และผ่านการทดสอบว่าให้ตัวเลขตรงกับ Dashboard ดังจะกล่าวในบทที่ 4

แคช โค้ดใช้ `CacheService.getUserCache()` ซึ่งเป็นแคชที่แยกตามผู้ใช้และสคริปต์ [40] ข้อมูลของผู้ใช้คนหนึ่งจึงไม่ถูกส่งให้ผู้ใช้อีกคนจากแคช มีการเก็บ 2 อย่าง

1) **ผลการตรวจสิทธิ์** เก็บเฉพาะผล “อนุญาต” นาน 60 วินาที เพื่อไม่ให้อ่านแท็บ “ผู้ใช้” ทุกครั้งที่หน้าเว็บขอข้อมูล ผล “ปฏิเสธ” ไม่เก็บ ผู้ที่เพิ่งได้รับสิทธิ์จึงเข้าได้ทันที

2) **ข้อมูลที่อ่านจาก Sheet** เก็บนาน 120 วินาที เนื่องจากค่าในแคชแต่ละคีย์เก็บได้ไม่เกิน 100 KB [41] โค้ดจึงแบ่งข้อความ JSON เป็นชิ้นละ 30,000 ตัวอักษร (ข้อความภาษาไทยหนึ่งตัวอักษรใช้หลายไบต์) และบันทึกด้วย `putAll` ถ้าชิ้นใดหายไปถือว่าไม่มีแคชและอ่าน Sheet ใหม่ ถ้าบันทึกแคชไม่สำเร็จก็อ่าน Sheet ทุกครั้งแทน

ข้อแลกเปลี่ยนของแคชคือ ข้อมูลที่แก้ใน Sheet จะปรากฏบน Dashboard ช้าได้ไม่เกินประมาณ 2 นาที

สิทธิ์ที่ขอ (OAuth scopes) โปรเจกต์กำหนดสิทธิ์ใน `appsscript.json` เองแทนการให้ Apps Script ตรวจหาอัตโนมัติ ซึ่งเอกสารของ Google แนะนำให้ใช้ชุดสิทธิ์ที่แคบที่สุดเท่าที่ทำได้ [42] สิทธิ์ที่ Dashboard ใช้มี 2 รายการ

1) `https://www.googleapis.com/auth/spreadsheets.readonly` อ่าน Google Sheets ได้อย่างเดียว สคริปต์จึงแก้ไขข้อมูลคดีไม่ได้แม้โค้ดผิดพลาด
2) `https://www.googleapis.com/auth/userinfo.email` อ่านอีเมลของผู้ใช้ เพื่อใช้ตรวจสิทธิ์

เนื่องจากแชตบอตอยู่ในโปรเจกต์เดียวกันและใช้ไฟล์ `appsscript.json` ร่วมกัน โปรเจกต์จึงขอสิทธิ์เพิ่มอีก 2 รายการสำหรับเรียกบริการภายนอกและจัดการตัวตั้งเวลา ซึ่งอธิบายในหัวข้อ 3.4.8

3.4.5 ข้อมูลที่ส่งไปยังเบราว์เซอร์

ฟังก์ชัน `readCasesTabs_` เลือกส่งเฉพาะคอลัมน์ที่ Dashboard ใช้ โดยจับคู่หัวคอลัมน์แบบ “ขึ้นต้นด้วย” ตามรายการ `CONFIG.CASES_COLUMNS` และตัดแถวที่ว่างทั้งแถวออก คอลัมน์ที่ส่งและไม่ส่งสรุปในตารางที่ 3.8

ตารางที่ 3.8 คอลัมน์ของแท็บข้อมูลคดีที่ส่งและไม่ส่งไปยังเบราว์เซอร์

ส่ง (14 คอลัมน์)	ไม่ส่ง
เลข บก., เลขรายงาน, สภ. (ท้องที่), จังหวัด, วันที่รับ, วันที่ตรวจเสร็จ, วันที่มารับผล, สถานะ, ประเภทคดี (เลือก), ผู้ตรวจ (เลือก), ของกลาง (ชิ้น), นับในสถิติ, ข้อสังเกต (ผิดปกติ), เกณฑ์ของกลางผิดปกติ (ชิ้น)	เลขหนังสือนำส่ง, ลงวันที่, เดือนที่รับ, ปี พ.ศ. (รับ), ปีเลขรายงาน, ลำดับเลขรายงาน, อันดับ (ช่วยคำนวณ), ตั้งค่า, คอลัมน์อื่นที่ไม่อยู่ในรายการ (รวม User ID (LINE) ถ้ายังมีอยู่ในไฟล์) และแท็บ “ผู้ใช้” ทั้งแท็บ

ชื่อผู้ตรวจยังคงส่งไปยังเบราว์เซอร์ เพราะ Dashboard มีหน้าสถิติแยกตามผู้ตรวจสำหรับผู้มีสิทธิ์ คอลัมน์ “เดือนที่รับ” และ “ปี พ.ศ. (รับ)” ไม่ต้องส่ง เพราะหน้าเว็บคำนวณจาก “วันที่รับ” เอง สำหรับ Sheet ข้อมูลห้องปฏิบัติการ ฟังก์ชัน `readLabTabs_` ส่งทุกคอลัมน์ของแท็บ “ข้อมูลเคส” “ข้อมูลงบ” “ข้อมูล Lot” และ “หมายเหตุ” เพราะเป็นสถิติรายเดือนที่ไม่มีข้อมูลรายคดีหรือข้อมูลส่วนบุคคล การตรวจว่า `getBook("cases")` คืนเพียง 14 คอลัมน์ข้างต้นจะกล่าวในบทที่ 4

นอกจากข้อมูลจาก Sheet แล้ว หน้า “แหล่งข้อมูล” ยังให้ผู้ใช้เปิดไฟล์ .xlsx ในเครื่องของตนเพื่อดูได้ ไฟล์นั้นอ่านในเบราว์เซอร์ด้วย JSZip และไม่ถูกอัปโหลดไปยังเซิร์ฟเวอร์

3.4.6 การจัดการวันที่

`google.script.run` ส่งอ็อบเจกต์ `Date` ระหว่างเซิร์ฟเวอร์กับเบราว์เซอร์ไม่ได้ คำขอที่มีค่า `Date` จะล้มเหลว [43] ฟังก์ชัน `cellOut_` จึงแปลงค่าในเซลล์ก่อนส่ง ดังนี้

1) **วันที่** แปลงเป็นเลขลำดับวันแบบ Sheets/Excel (serial) คือจำนวนวันนับจากวันที่ 30 ธันวาคม ค.ศ. 1899 โดยอ่านปี เดือน วัน และเวลาตามเขตเวลาของ Sheet ด้วย `Utilities.formatDate` แล้วคำนวณดังสมการที่ (3.1) รูปแบบนี้ตรงกับที่ Dashboard ต้นแบบได้จากการอ่านไฟล์ .xlsx จึงใช้โค้ดแปลงข้อมูลชุดเดิมได้

$$\text{serial} = (t_{\text{UTC}}(y, m, d, h, i, s)) / (86{,}400{,}000) + 25{,}569 \quad (3.1)$$

เมื่อ t_{UTC} คือเวลาเป็นมิลลิวินาทีนับจาก 1 มกราคม ค.ศ. 1970 ของวันเวลาที่อ่านได้ และ 25,569 คือเลข serial ของวันที่ 1 มกราคม ค.ศ. 1970 เช่น วันที่ 1 มกราคม ค.ศ. 2022 ได้ค่า 44,562

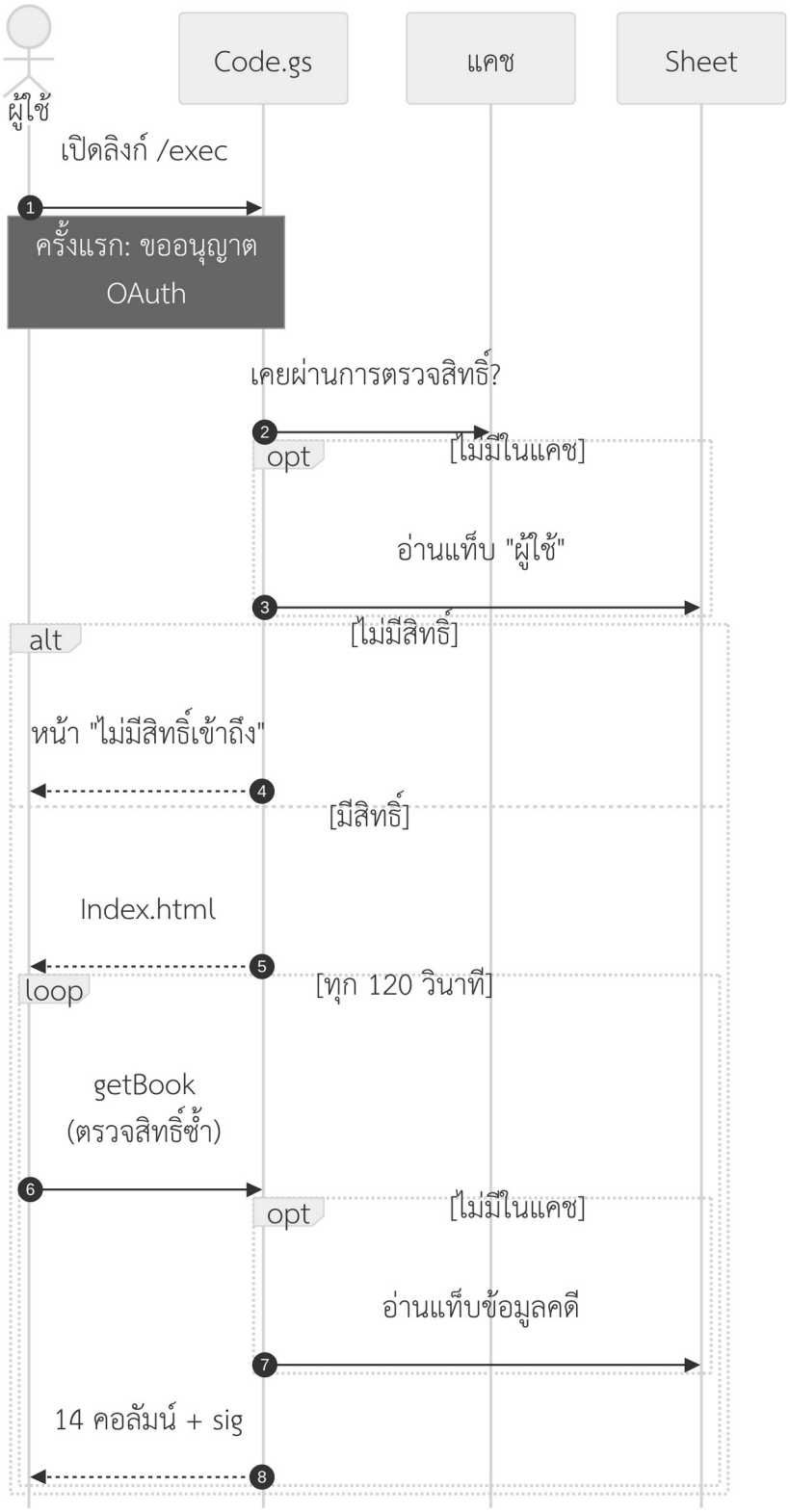
2) **เซลล์ว่าง** แปลงเป็น `null` ตัวเลขและค่าจริง/เท็จส่งตามเดิม ค่าอื่นส่งเป็นข้อความ

ฝั่ง `Stats.gs` แปลงเลข serial กลับเป็นข้อความ `YYYY-MM-DD` (ค.ศ.) เพื่อเปรียบเทียบช่วงวันที่ และใช้กติกาเดียวกับ

Dashboard คือ จัดเดือนและปีของคดีตาม “วันที่รับ” ไม่นับแถวที่ไม่มีวันที่รับ และคำนวณระยะเวลาดำเนินการเป็นจำนวนวันจากวันที่รับถึงวันที่ตรวจเสร็จ (สมการที่ 2.1) “วันนี้” และช่วงเดือนหรือปีก่อนหน้าสำหรับสรุปอัตโนมัติคำนวณตามเวลาประเทศไทย (UTC+7 ซึ่งไม่มีเวลาออมแสง) และโปรเจกต์ตั้งเขตเวลาเป็น Asia/Bangkok การแสดงผลใช้ปี พ.ศ. (ค.ศ. + 543) การทดสอบช่วงวันที่ในกรณีข้ามปี และเดือนกุมภาพันธ์ทั้งปีปกติและปีอธิกสุรทินจะกล่าวในบทที่ 4

3.4.7 ลำดับการทำงานเมื่อผู้ใช้เปิด Dashboard

ลำดับการทำงานตั้งแต่ผู้ใช้เปิดลิงก์ Dashboard จนถึงการขอข้อมูลซ้ำตามรอบเวลาแสดงในรูปที่ 3.8 การตรวจสิทธิ์ในขั้นที่ 2–3 ใช้ `checkAccess_` (ตารางที่ 3.6) และขั้นที่ 6 เรียก `requireAccess_` ตรวจซ้ำทุกครั้ง ข้อมูลที่ส่งกลับในขั้นที่ 8 คือ 14 คอลัมน์ในตารางที่ 3.8 พร้อมค่า `sig`



รูปที่ 3.8 ลำดับการทำงานเมื่อผู้ใช้เปิด Dashboard ตั้งแต่ตรวจสิทธิ์จนถึงการขอข้อมูลซ้ำตามรอบเวลา

3.4.8 การ deploy ของแชตบอต LINE และการป้องกันฟังก์ชันผู้ดูแล

LINE Platform ส่ง webhook มาเป็นคำขอ HTTP POST โดยไม่มีการล็อกอินบัญชี Google จึงใช้ deployment ของ Dashboard ซึ่งกำหนดให้ผู้เรียกต้องมีบัญชี Google และทำงานในนามผู้ใช้ไม่ได้ แชตบอตจึงอยู่ในโปรเจกต์ Apps Script เดียวกันแต่ใช้ **deployment แยกต่างหาก** แบบ Execute as: Me และ Who has access: Anyone เพื่อให้อ่าน Sheet ได้ด้วยสิทธิ์ของเจ้าของสคริปต์ผ่าน `loadCaseRecords_` ซึ่งไม่ตรวจสิทธิ์ผู้ใช้ การควบคุมสิทธิ์ของแชตบอตจึงอาศัยมาตรการในหัวข้อ 3.5.5 แทน ทั้งสอง deployment ใช้โค้ดชุดเดียวกัน เมื่อแก้โค้ดจึงต้องสร้างเวอร์ชันใหม่ให้ทั้ง 2 รายการ

การรวมไว้ในโปรเจกต์เดียวกันทำให้ต้องป้องกัน 3 เรื่อง ดังนี้

1) **การเปิดลิงก์ของแชตบอตในเบราว์เซอร์** ผู้ที่ได้ลิงก์ของ deployment แชตบอตอาจเปิด `doGet` หรือเรียก `getBook` ผ่านลิงก์นั้นได้ ซึ่งจะทำงานในนามเจ้าของสคริปต์และข้ามการตรวจชั้นที่ 2 (การแชร์ Sheet) ไป `checkAccess_` จึงเปรียบเทียบบัญชีที่สคริปต์ทำงานในนาม (`Session.getEffectiveUser()`) กับบัญชีผู้เปิด (`Session.getActiveUser()`) [12] ถ้าไม่ตรงกันจะปฏิเสธด้วยเหตุผล `wrong-deployment` และถ้าไม่ทราบอีเมลผู้เปิดก็ปฏิเสธด้วยเหตุผล `no-email` (ตารางที่ 3.6) ฟังก์ชันข้อมูลของ Dashboard จึงส่ง

ข้อมูลเฉพาะเมื่อทำงานผ่าน deployment แบบ User accessing the web app เท่านั้น

2) **ฟังก์ชันผู้ดูแลถูกเรียกจากหน้า Dashboard** ฟังก์ชันระดับบนสุดที่ไม่ลงท้ายด้วยขีดล่างทุกตัวถูกเรียกผ่าน `google.script.run` ได้ ฟังก์ชันที่ผู้ดูแลใช้รันใน Apps Script editor ได้แก่ `makeWebhookToken` `startSetupMode` `showSetupGroups` `allowSeenGroup` `previewSummary` `previewYearSummary` `installTriggers` และ `removeTriggers` จึงเรียก `requireAdmin_` ก่อนทำงานทุกครั้ง ฟังก์ชันนี้ยอมให้ทำงานเฉพาะเมื่ออีเมลผู้รันตรงกับค่า `ADMIN_EMAIL` ใน Script Properties ถ้ายังไม่ตั้งค่านี้จะปฏิเสธทุกคน
3) **ฟังก์ชันส่งสรุปถูกเรียกโดยตรง** `dailySummaryJob` ต้องเป็นฟังก์ชันระดับบนสุดเพื่อให้ตัวตั้งเวลาเรียกได้ ฟังก์ชันจึงตรวจว่าค่า `triggerUid` ในพารามิเตอร์ตรงกับตัวตั้งเวลาของโปรเจกต์นี้ที่ชี้มายัง `dailySummaryJob` จริง ถ้าไม่ตรงจะไม่ทำงาน การดูข้อความสรุปด้วยตนเองให้ใช้ `previewSummary` หรือ `previewYearSummary` แทน

สิทธิ์ที่เพิ่มขึ้นจากแชตบอต เนื่องจากทั้งสอง deployment ใช้ `appsscript.json` ไฟล์เดียวกัน โปรเจกต์จึงขอสิทธิ์เพิ่มจาก 2 รายการในหัวข้อ 3.4.4 อีก 2 รายการ ได้แก่ `https://www.googleapis.com/auth/script.external_request` สำหรับเรียก LINE Messaging API และ Gemini API ผ่าน `UrlFetchApp` และ

`https://www.googleapis.com/auth/script.scriptapp` สำหรับสร้างและตรวจตัวตั้งเวลา [42] ผลคือผู้ใช้ Dashboard ที่เคยกดอนุญาตแล้วต้องกดอนุญาตเพิ่มอีก 1 ครั้งหลังเพิ่มแชตบอต แม้ Dashboard เองจะไม่ใช้สิทธิ์ 2 รายการนี้ เจ้าของโครงงานยอมรับข้อแลกเปลี่ยนนี้เพื่อให้ Dashboard และแชตบอตใช้ `Stats.gs` ชุดเดียวกัน

3.5 การออกแบบแชตบอต LINE

หัวข้อนี้อธิบายแชตบอตตามโค้ดที่พัฒนาแล้วใน `apps-script/` ได้แก่ `LineBot.gs` (รับ webhook ตรวจความถูกต้องและส่งข้อความ) `Intent.gs` (แยกเจตนา) และ `Stats.gs` (ค้นคดีและนับคดีค้าง) แชตบอตใช้ deployment แยกจาก Dashboard (หัวข้อ 3.4.8) ค่าตั้งค่าทั้งหมดเก็บใน Script Properties ดังตารางที่ 3.9

ตารางที่ 3.9 Script Properties ของแชตบอตและสรุปอัตโนมัติ

ชื่อ	ความหมาย	จำเป็น
`LINE_CHANNEL_TOKEN`	channel access token ของ LINE	ต้องมี
`WEBHOOK_TOKEN`	รหัสลับสุ่ม 64 ตัวอักษรที่ต่อท้าย URL ของ webhook	ต้องมี
`ALLOWED_GROUP_ID`	รหัสกลุ่ม LINE ที่อนุญาต	ต้องมี
`ADMIN_EMAIL`	อีเมลผู้ดูแลที่รันฟังก์ชันผู้ดูแลได้	ต้องมี
`DASHBOARD_URL`	ลิงก์ Dashboard ที่แนบท้ายข้อความสรุป	แนะนำ
`GEMINI_API_KEY`	API key ของ Gemini ถ้าไม่ใส่ แชตบอตใช้เฉพาะกฎ	ไม่บังคับ
`GEMINI_MODEL`	ชื่อรุ่นของ Gemini ค่าเริ่มต้น `gemini-3.5-flash-lite`	ไม่บังคับ

ชื่อ	ความหมาย	จำเป็น
GEMINI_MAX_PER_HOUR	จำนวนครั้งสูงสุดที่เรียก Gemini ต่อชั่วโมง ค่าเริ่มต้น 30	ไม่บังคับ
PUSH_ENABLED	ตั้งเป็น yes เมื่อเจ้าของโครงงานอนุมัติให้ส่งสรุปจริง	ไม่บังคับ

3.5.1 ขอบเขตการใช้งานและวิธีเรียกบอต

แชตบอตใช้ในกลุ่ม LINE ของเจ้าหน้าที่กลุ่มเดียว และตอบคำถาม 2 แบบ คือ สถานะคดีจากเลข บก. และสรุปคดีค้าง กลุ่มนี้ใช้สนทนาเรื่องงานตามปกติด้วย เจ้าของโครงงานจึงกำหนดให้บอต**เงียบ**เมื่อข้อความไม่ได้เรียกบอต เพื่อไม่รบกวนการสนทนา ข้อความถือว่าเรียกบอตเมื่อเข้าเงื่อนไขข้อใดข้อหนึ่งในตารางที่ 3.10

ตารางที่ 3.10 วิธีเรียกแชตบอตและผลที่ได้

วิธีเรียก	ตัวอย่าง	ผล
คำสั่งสั้นที่เป็นเลข บก. 1–3 เลข ทั้งข้อความ (รองรับเลขไทย)	123/2568, คดี ๑๒๓/๒๕๖๘, 123/2568 ถึงไหนแล้ว	ตอบสถานะคดี
คำสั่งสั้นเรื่องคดีค้าง ทั้งข้อความ	คดีค้าง, งานค้าง, สรุปคดีค้าง, มีคดีค้างกี่คดี	ตอบสรุปคดีค้าง
ขึ้นต้นด้วยคำเรียก บอต บอท bot @bot หรือเครื่องหมาย /	บอต คดีที่ยังไม่รับผลมีกี่คดี, /ค้าง	แยกเจตนาด้วยกฎ ถ้าไม่เข้าใจจึงถาม Gemini
กล่าวถึงบอต (mention)	@ชื่อบอต ตามด้วยคำถาม	เหมือนการขึ้นต้นด้วยคำเรียก

คำสั่งสั้นต้องเป็นคำสั่งทั้งข้อความเท่านั้น ข้อความสนทนาที่มีเลขคดีปนอยู่ในประโยค เช่น “ส่งของกลางคดี 123/2568 ไปแล้วนะ” หรือ “งานค้างเยอะจัง” จึงไม่ถือว่าเรียกบอต ระบบไม่ตอบ ไม่อ่าน Sheet และไม่เรียก Gemini ถ้าต้องการถามด้วยประโยคยาว ผู้ใช้ขึ้นต้นด้วยคำเรียกหรือกล่าวถึงบอต บอตตอบวิธีใช้แบบสั้นเฉพาะเมื่อผู้ใช้เรียกบอตแล้วแต่ระบบไม่เข้าใจคำถาม หรือเมื่อ Gemini ใช้งานไม่ได้ ข้อความที่ยาวเกิน 1,000 ตัวอักษรไม่ถูกประมวลผล

3.5.2 การแยกเจตนาแบบผสม

การแยกเจตนา (intent) ทำเป็น 3 ขั้น เพื่อให้คำถามรูปแบบที่พบบ่อยไม่ต้องเรียก AI เลย

1) **ขั้นที่ 1 กฎในโค้ด** `detectIntentByRules_` แปลงเลขไทยเป็นเลขอารบิก ตัดส่วนที่กล่าวถึงผู้ใช้ออก แล้วตรวจคำสั่งสั้นและคำเรียกตามหัวข้อ 3.5.1 เมื่อผู้ใช้เรียกบอต ถ้าพบเลขคดีรูปแบบ ลำดับ/ปี พ.ศ. (ลำดับ 1–5 หลัก ปี 4 หลักขึ้นต้นด้วย 25) จะได้เจตนา “สถานะคดี” ถ้าพบคำ เช่น “ค้าง” “ยังไม่รับผล” “รอรับผล” จะได้เจตนา “สรุปคดีค้าง” และถ้าข้อความว่างหรือถามวิธีใช้จะได้เจตนา “วิธีใช้” **เลขคดีได้จากขั้นนี้เสมอ** ไม่ได้มาจาก AI
2) **ขั้นที่ 2 ตัดข้อมูลก่อนส่ง** ถ้าผู้ใช้เรียกบอตแต่กฎไม่เข้าใจ `sanitizeForAi_` ตัดข้อมูลที่อาจระบุตัวบุคคลหรือคดีออกจากข้อความ ได้แก่ การกล่าวถึงผู้ใช้ URL อีเมล ตัวเลขทุกชนิด (รวมเลขคดี เลขรายงาน และหมายเลขโทรศัพท์) คำนำหน้าชื่อ ยศ ตำแหน่ง และ

ชื่อหน่วย เช่น “นาย” “พ.ต.ท.” “สภ.” “จังหวัด” พร้อมคำ 2 คำที่ตามมา และคำภาษาอังกฤษที่ขึ้นต้นด้วยตัวพิมพ์ใหญ่ซึ่งมักเป็นชื่อ แล้วตัดให้ยาวไม่เกิน 200 ตัวอักษร ตัวอย่างการตัดข้อมูลแสดงในตารางที่ 3.11

3) **ขั้นที่ 3 ถาม Gemini** `classifyWithGemini_` ส่งข้อความที่ตัดแล้วพร้อมคำสั่งระบบ (system instruction) ที่ระบุว่าข้อความถูกตัดเลขคดีและชื่อออกแล้ว และให้ตอบเป็น JSON รูปแบบ `{"intent": ...}` เท่านั้น คำขอกำหนด `responseMimeType` เป็น `application/json` [27] ระบบตรวจผลที่ได้เองอีกครั้ง ค่าที่ไม่อยู่ในรายการ `case_status pending_summary help` และ `unknown` ถือเป็น `unknown` และค่าอื่นที่ Gemini อาจส่งมา เช่น เลขคดี จะถูกทิ้ง เลขคดีที่ใช้ค้นข้อมูลดึงจากข้อความต้นฉบับด้วยโค้ดเท่านั้น ถ้าไม่มี API key เกินโควตา หรือเรียก Gemini ไม่สำเร็จ (เช่น HTTP 429 หรือ 500 หรือ JSON ไม่ถูกต้อง) บอตตอบวิธีใช้แทน

ตารางที่ 3.11 ตัวอย่างการตัดข้อมูลก่อนส่งให้ Gemini (ข้อความสมมติ)

ขั้น	ข้อความ
ผู้ใช้พิมพ์	บอต ช่วยดูหน่อยว่าตอนนี้มีอะไรติดอยู่บ้าง นายสมชาย ใจดี โทร 081-234-5678 อีเมล somchai@example.org เลขรายงาน 03-68-00012 สภ.ตัวอย่าง3 เรื่องเลข 12/68 https://example.org/x

ขั้น	ข้อความ
ส่งให้ Gemini	ช่วยดูหน่อยว่าตอนนี้มีอะไรติดอยู่บ้าง โทร อีเมล เลขรายงาน

ชื่อรุ่นของ Gemini และ API key อ่านจาก Script Properties (`GEMINI_MODEL` และ `GEMINI_API_KEY`) จึงเปลี่ยนรุ่นได้โดยไม่ต้องแก้โค้ด ค่าเริ่มต้นคือ `gemini-3.5-flash-lite` ซึ่งเอกสารของ Google ระบุสถานะเป็นรุ่นเสถียร (Stable) ณ วันที่ตรวจ [44] ชื่อรุ่นต้องผ่านการตรวจรูปแบบก่อนนำไปต่อเป็น URL และระบบจำกัดการเรียก Gemini ไม่เกิน 30 ครั้งต่อชั่วโมง (ตั้งค่าได้) โดยนับใน `CacheService` เพื่อไม่ให้โควตาฟรีหมดหรือถูกใช้ผิดวัตถุประสงค์ ข้อจำกัดของการตัดข้อมูลคือ กฎตัดคำได้เฉพาะรูปแบบที่กำหนดไว้ ชื่อบุคคลที่ไม่มีคำนำหน้าอาจหลุดไปได้ แต่ Gemini จะได้รับเฉพาะข้อความที่ผู้ใช้ตั้งใจเรียกบอตและกฎไม่เข้าใจเท่านั้น ข้อความสนทนาทั่วไปไม่ถูกส่งออก

3.5.3 ข้อความตอบกลับ

เมื่อได้เจตนาแล้ว `buildReply_` อ่าน Sheet ผ่าน `loadCaseRecords_` แล้วประกอบข้อความตอบจากแม่แบบในโค้ด AI ไม่ได้เห็นข้อมูลใน Sheet และไม่ได้เป็นผู้เขียนคำตอบ จึงไม่มีโอกาสที่ AI จะแต่งสถานะคดีขึ้นเอง ข้อความตอบมี 3 แบบ

1) **สถานะคดี** ตอบได้สูงสุด 3 เลขคดีต่อข้อความ แต่ละคดีแสดงเฉพาะข้อมูลที่จำเป็นต่อการติดตามงาน 6 รายการ คือ สถานะ ประเภทคดี จำนวนของกลาง วันที่รับ วันที่ตรวจเสร็จ และวันที่มารับผล โดยแสดงวันที่เป็นปี พ.ศ. **ไม่แสดง**ชื่อผู้ตรวจ สถานีตำรวจ จังหวัด

หรือเลขรายงาน ถ้าช่องสถานะว่างจะแสดงว่า “ไม่ระบุ” และถ้าไม่พบเลขคดีจะแจ้งว่าไม่พบพร้อมรูปแบบเลขที่ถูกต้อง

2) **สรุปคดีค้าง** ใช้ `Stats.pendingByStatus` ซึ่งนับตามนิยามในหัวข้อ 3.2.3 แสดงจำนวนรวมและแยก 2 กลุ่ม พร้อมเลข บก. ที่ค้างนานที่สุดกลุ่มละ 3 คดีและจำนวนวัน กลุ่ม “กำลังดำเนินการ” นับวันจากวันที่รับ ส่วนกลุ่ม “เสร็จแล้ว รอรับผล” นับวันจากวันที่ตรวจเสร็จ ถ้ามีแถวที่ช่องสถานะว่าง ข้อความจะบอกจำนวนไว้ในกลุ่ม “กำลังดำเนินการ” เพื่อให้เจ้าหน้าที่ตามกรอกข้อมูล

3) **วิธีใช้** บอกคำสั่งที่ใช้ได้แบบสั้น

ตัวอย่างข้อความตอบที่ได้จากการรันโค้ดจริงกับข้อมูลจำลองแสดงด้านล่าง (จำนวนวันนับถึงวันที่ทดสอบ คือ 3 พฤศจิกายน 2569) ข้อมูลจำลองมีคดีตั้งแต่ปี พ.ศ. 2565 และไม่ได้ปรับสถานะคดีเก่าให้เป็นปัจจุบัน จำนวนวันที่ค้างจึงสูงกว่าที่คาดว่าจะพบในงานจริง

```
คดี 1/2565
สถานะ: รับผลไปแล้ว
ประเภทคดี: ชีวิตและร่างกาย
ของกลาง: 4 ชิ้น
วันที่รับ: 1 ม.ค. 2565
ตรวจเสร็จ: 21 ม.ค. 2565
มารับผล: 14 ก.พ. 2565
```

```
คดีค้าง (ยังไม่รับผล) 39 คดี
```

```
ข้อมูล ณ 3 พ.ย. 2569

1) กำลังดำเนินการ: 22 คดี
ค้างนานสุด (นับจากวันที่รับ): 14/2565 (1,735 วัน), 75/2565
(1,574 วัน), 137/2565 (1,407 วัน)

2) เสร็จแล้ว รอรับผล: 17 คดี
รอนานสุด (นับจากวันที่ตรวจเสร็จ): 3/2565 (1,752 วัน), 4/2565
(1,745 วัน), 85/2565 (1,529 วัน)
```

3.5.4 การจำกัดกลุ่มและการส่งข้อความ

เหตุการณ์ที่มาจากกลุ่มมีรหัสกลุ่ม (`source.groupId`) [21] ระบบตอบเฉพาะเหตุการณ์ที่รหัสกลุ่มตรงกับ `ALLOWED_GROUP_ID` ข้อความจากแชตส่วนตัว ห้องแชตหลายคน หรือกลุ่มอื่นไม่ถูกประมวลผล ถ้ายังไม่ได้ตั้งค่านี้ บอตเงียบทุกกลุ่ม (fail closed)

การส่งข้อความแบ่งเป็น 2 แบบ [18]

1) **Reply message** ใช้ตอบคำถามทั้งหมด ต้องใช้ reply token ที่มากับ webhook event และไม่นับรวมในจำนวนข้อความของโควตา [19]
2) **Push message** ใช้เฉพาะสรุปตามเวลา (หัวข้อ 3.6) ส่งได้โดยไม่ต้องมีคำถามนำ แต่นับรวมในโควตา โดยนับตามจำนวนผู้รับต่อคำขอ ไม่ใช่จำนวนข้อความในคำขอ [18], [19]

กลุ่ม LINE ของเจ้าหน้าที่มีสมาชิกประมาณ 20 คน (ข้อมูลจากเจ้าของโครงงาน) การส่งสรุป 1 คำขอจึงนับประมาณ 20 ข้อความ ในเดือนมกราคมระบบส่งสรุปเดือนธันวาคมและสรุปทั้งปี

เป็น 2 ข้อความใน**คำขอ push เดียวกัน** จึงยังนับประมาณ 20 ข้อความ (ถ้าแยกเป็น 2 คำขอจะนับประมาณ 40 ข้อความ) การใช้งานทั้งปีจึงประมาณ 20 ข้อความต่อเดือน ต่ำกว่าโควตาข้อความของแพ็กเกจฟรีของ LINE Official Account ในประเทศไทยซึ่งกำหนดไว้ 300 ข้อความต่อเดือนตั้งแต่วันที่ 1 สิงหาคม 2567 [20] ทั้งนี้ โควตาอาจเปลี่ยนแปลงได้ ผู้ดูแลควรตรวจจากหน้าแพ็กเกจของบัญชีก่อนใช้งานจริง

3.5.5 มาตรการทดแทนการตรวจลายมือชื่อ webhook

LINE กำหนดให้ตรวจลายมือชื่อในส่วนหัว `x-line-signature` ของคำขอ webhook ก่อนประมวลผลเหตุการณ์ [22], [37] แต่ออบเจ็กต์เหตุการณ์ที่ Apps Script ส่งให้ `doPost` มีเพียงพารามิเตอร์ใน URL และเนื้อหาของคำขอ ไม่มีส่วนหัวของคำขอ [15] ระบบจึง**ตรวจลายมือชื่อตามที่ LINE แนะนำไม่ได้** ซึ่งหมายความว่าผู้ที่รู้ URL ของ webhook อาจส่งเหตุการณ์ปลอมเข้ามาได้ ระบบจึงใช้มาตรการทดแทนหลายชั้นตามตารางที่ 3.12

ตารางที่ 3.12 มาตรการทดแทนการตรวจลายมือชื่อ webhook ใน `LineBot.gs`

มาตรการ	รายละเอียด	ป้องกันอะไร
รหัสลับใน URL	URL ของ webhook ต้องมี `?token=` ที่ตรงกับ `WEBHOOK_TOKEN` (สุ่มด้วย `makeWebhookToken` ยาว 64 ตัวอักษร) เทียบแบบใช้เวลาคงที่ ถ้าไม่มี ไม่ตรง หรือรหัสที่ตั้งไว้สั้นกว่า 24 ตัวอักษร ทิ้งทั้ง	คำขอจากผู้ที่ไม่รู้รหัสและการเดารหัสจากเวลาตอบ

มาตรการ	รายละเอียด	ป้องกันอะไร
	คำขอโดยไม่อ่าน Sheet	
จำกัดกลุ่ม	ตอบเฉพาะ `ALLOWED_GROUP_ID`	คำถามจากกลุ่มหรือแชตอื่น
ตอบด้วย reply token เท่านั้น	ข้อมูลส่งผ่าน reply API ไปยังห้องที่ LINE ออก reply token ให้ คำขอปลอมไม่มี reply token จริง LINE จึงปฏิเสธ	การสั่งให้บอตส่งข้อมูลไปที่อื่น
คำตอบ HTTP เหมือนกันทุกกรณี	`doPost` ตอบข้อความ `OK` เสมอ ไม่มีข้อมูลคดีและไม่มีรายละเอียดข้อผิดพลาด	การดึงข้อมูลหรือทดสอบรหัสผ่านคำตอบ HTTP
จำกัดขนาด	เนื้อหาคำขอไม่เกิน 100,000 ตัวอักษร ประมวลผลไม่เกิน 20 เหตุการณ์ต่อคำขอ ข้อความไม่เกิน 1,000 ตัวอักษร	คำขอขนาดใหญ่ผิดปกติ
ทิ้งเหตุการณ์ซ้ำและเก่า	จำ `webhookEventId` ไว้ 10 นาที ไม่ตอบเหตุการณ์ที่ส่งซ้ำ (`deliveryContext.isRedelivery`) และเหตุการณ์ที่เก่ากว่า 5 นาที [37]	การตอบซ้ำ และการส่งเหตุการณ์เดิมกลับเข้ามาใหม่
จำกัดการเรียก Gemini	ไม่เกิน 30 ครั้งต่อชั่วโมง (หัวข้อ 3.5.2)	การใช้โควตา Gemini จนหมด
ข้อมูลน้อยที่สุด	คำตอบสถานะคดีไม่มีชื่อผู้ตรวจ สถานี จังหวัด และเลขรายงาน และบันทึกการทำงาน (log) เก็บเฉพาะรหัส	ความเสียหายเมื่อมาตรการอื่นล้มเหลว

มาตรการ	รายละเอียด	ป้องกันอะไร
	สถานะ HTTP ไม่เก็บข้อความของผู้ใช้	

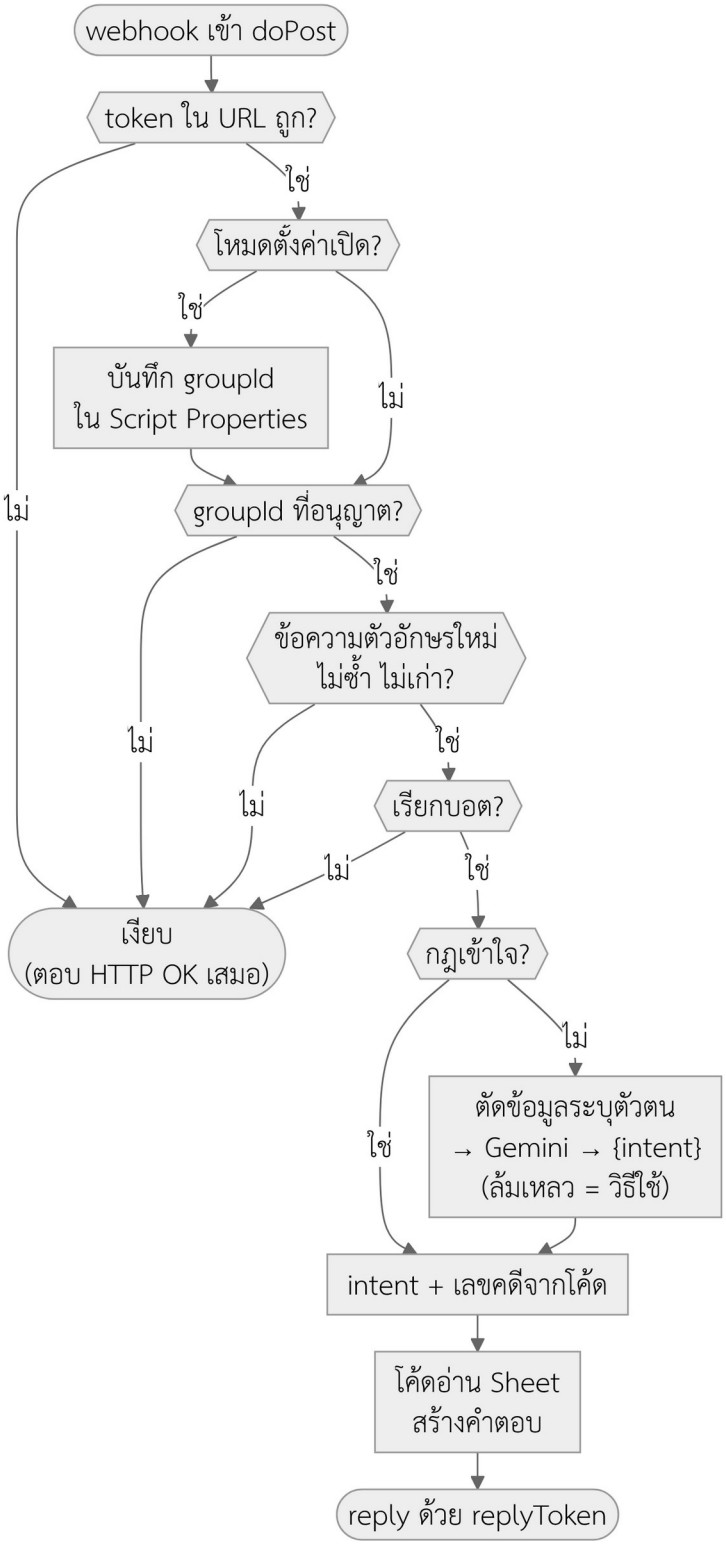
ความเสี่ยงที่ยังเหลือคือ ถ้า URL ที่มีรหัสลับรั่วไหล ผู้ไม่หวังดีจะส่งเหตุการณ์ปลอมได้ แต่ทำได้เพียงให้บอตเรียก reply API ด้วย token ปลอมซึ่ง LINE ปฏิเสธ หรือใช้โควตา Gemini ซึ่งถูกจำกัดต่อชั่วโมง และผู้ดูแลเปลี่ยนรหัสได้ทันทีโดยลบ `WEBHOOK_TOKEN` แล้วรัน `makeWebhookToken` ใหม่และแก้ URL ใน LINE Developers Console

การตอบกลับของ Apps Script และปุ่ม Verify LINE กำหนดให้เซิร์ฟเวอร์ตอบรหัสสถานะ 200 [45] แต่เว็บแอป Apps Script กำหนดรหัสสถานะเองไม่ได้ และจากประสบการณ์ที่ผู้ใช้ทั่วไปรายงาน เว็บแอปจะรันโค้ด `doPost` ก่อนแล้วตอบเป็นการเปลี่ยนเส้นทาง (HTTP 302) ไปยังโดเมน `script.googleusercontent.com` พฤติกรรมนี้ไม่มีในเอกสารของ Google จึงยังต้องตรวจซ้ำเมื่อนำขึ้นใช้งานจริง ผลที่อาจเห็นคือปุ่ม Verify ใน LINE Developers Console แสดงข้อผิดพลาด และสถิติข้อผิดพลาดของ webhook มีรายการ แต่บอตยังตอบได้ เพราะคำตอบถูกส่งผ่าน reply API ระหว่างที่ `doPost` ทำงานแล้ว ปุ่ม Verify ส่งคำขอที่ไม่มีเหตุการณ์ (`"events": []`) [45] ซึ่งระบบตอบ `OK` โดยไม่ทำอะไร คู่มือติดตั้งจึงให้ปิดการส่ง webhook ซ้ำ (webhook redelivery) และให้ทดสอบด้วยการพิมพ์ในกลุ่มจริงแทน

โหมดตั้งค่าสำหรับหารหัสกลุ่ม ในการติดตั้งครั้งแรกผู้ดูแลยังไม่รู้ `groupId` ของกลุ่ม ระบบไม่ให้บอตพิมพ์รหัสกลุ่มลงในแชต เพราะระหว่างนั้นบอตยังไม่มีรายชื่อกลุ่มที่อนุญาต และรหัสจะค้างอยู่ในประวัติแชต ผู้ดูแลจึงรัน `startSetupMode` ใน editor เพื่อเปิดโหมดตั้งค่า 60 นาที ระหว่างนั้นเมื่อบอตถูกเชิญเข้ากลุ่มหรือมีข้อความในกลุ่ม (และคำขอผ่านการตรวจรหัสลับแล้ว) ระบบบันทึกรหัสกลุ่มไว้ใน Script Properties และบันทึกการทำงานซึ่งเจ้าของสคริปต์เท่านั้นที่เห็น โดยไม่ตอบอะไรในแชต จากนั้นผู้ดูแลรัน `showSetupGroups` เพื่อดูรายการ และ `allowSeenGroup` เพื่อตั้ง `ALLOWED_GROUP_ID` ซึ่งจะปิดโหมดตั้งค่าทันที ถ้าไม่ปิดเอง โหมดนี้หมดอายุเองเมื่อครบ 60 นาที

3.5.6 ผังการทำงานของแชตบอต

ลำดับการตรวจของ `doPost` ตั้งแต่รับ webhook จนส่งคำตอบแสดงในรูปที่ 3.9 การตรวจที่ไม่ผ่านทุกขั้นจบที่การเงียบและตอบ HTTP ด้วย `OK` เหมือนกัน การอ่าน Sheet เกิดขึ้นหลังจากรู้แล้วว่าต้องตอบเท่านั้น



รูปที่ 3.9 ผังการทำงานของแชตบอต LINE

3.6 การออกแบบรายงานสรุปอัตโนมัติ

3.6.1 รอบเวลาและการกันส่งซ้ำ

รายงานสรุปสร้างโดย `Summary.gs` ผู้ดูแลรัน `installTriggers` ครั้งเดียวเพื่อสร้างตัวตั้งเวลารายวัน 1 ตัว ที่เรียก `dailySummaryJob` ช่วงเวลา 08:00–09:00 น. ตามเขตเวลา Asia/Bangkok [38] (ตัวตั้งเวลาของ Apps Script ระบุได้เป็นช่วงชั่วโมง [16]) รันซ้ำได้โดยไม่สร้างตัวตั้งเวลาซ้ำ ฟังก์ชันตรวจวันที่ตามเวลาไทยแล้วทำงานดังนี้

1) **รายเดือน** วันที่ 1 ของทุกเดือน สรุปข้อมูลของ**เดือนปฏิทินก่อนหน้า** เช่น วันที่ 1 พฤศจิกายน สรุปเดือนตุลาคม
2) **รายปี** วันที่ 1 มกราคม สรุปข้อมูลของ**ปีปฏิทินก่อนหน้า** (มกราคม–ธันวาคม) วันที่ 1 มกราคมจึงส่งทั้งสรุปเดือนธันวาคมและสรุปทั้งปี เป็น 2 ข้อความในคำขอ push เดียว (หัวข้อ 3.5.4)
3) **ส่งตามหลัง** ถ้าวันที่ 1 ส่งไม่สำเร็จ เช่น LINE ตอบข้อผิดพลาด ตัวตั้งเวลาวันที่ 2 และ 3 จะลองส่งงวดเดิมอีกครั้ง หลังวันที่ 3 จะไม่ส่งงวดนั้นแล้ว

การกันส่งซ้ำมี 3 ชั้น ได้แก่ (1) บันทึกงวดที่ส่งสำเร็จแล้วใน Script Properties (`SUMMARY_SENT_MONTH` เช่น `2026-10` และ `SUMMARY_SENT_YEAR` เช่น `2026`) และไม่ส่งงวดที่บันทึกไว้ (2) ใช้ `LockService.getScriptLock()` ป้องกันไม่ให้การทำงาน 2 ครั้งส่งพร้อมกัน [46] และอ่านงวดที่ต้องส่งใหม่หลังได้ล็อก และ (3) ใส่ส่วนหัว `X-Line-Retry-Key` ที่คำนวณคงที่จากงวด ถ้า LINE เคย

รับคำขอที่มีรหัสเดียวกันแล้ว จะตอบ 409 และไม่ส่งซ้ำ [47] ระบบถือว่ารหัส 409 คือส่งแล้ว

การอนุมัติก่อนส่งจริง ระบบส่ง push จริงเฉพาะเมื่อ Script Property `PUSH_ENABLED` มีค่า `yes` ซึ่งเจ้าของโครงงานเป็นผู้ตั้งหลังอนุมัติ ก่อนหน้านั้นตัวตั้งเวลาเพียงเขียนข้อความตัวอย่างลงบันทึกการทำงานและไม่บันทึกว่าส่งแล้ว ผู้ดูแลตรวจข้อความล่วงหน้าได้ด้วย `previewSummary(ปี, เดือน)` และ `previewYearSummary(ปี)` ซึ่งรับปีเป็น ค.ศ. หรือ พ.ศ. และไม่ส่งข้อความ การหยุดส่งทำได้โดยลบ `PUSH_ENABLED` หรือรัน `removeTriggers`

คดีจัดเข้าเดือนและปีตาม “วันที่รับ” และนับเฉพาะแถวที่ “นับในสถิติ” ไม่เป็น 0 ซึ่งเป็นเกณฑ์เดียวกับ Dashboard

3.6.2 เนื้อหาของรายงาน

1) ชื่อช่วงเวลาและช่วงวันที่รับที่นับ
2) จำนวนคดีในช่วงเวลา
3) จำนวนของกลางรวม (ชิ้น)
4) จำนวนคดีแยกตามประเภทคดี
5) จำนวนคดีของผู้ตรวจแต่ละคน แยกตามประเภทคดี แสดงเป็นจำนวนเท่านั้น ไม่แสดงเลขคดี
6) จำนวนแถวที่ตั้งไม่นับในสถิติ (ถ้ามี)
7) ลิงก์ Dashboard สำหรับดูรายละเอียด ซึ่งเปิดได้เฉพาะผู้มีสิทธิ์

รายงานไม่มีเลขคดี ชื่อสถานี หรือรายละเอียดรายคดี เพราะข้อความใน LINE อาจถูกส่งต่อออกนอกกลุ่มได้ง่าย ประเภทคดีที่ว่างแสดงเป็น “ไม่ระบุ” ตามค่า `Stats.UNSPECIFIED` ใน

`Stats.gs` เหมือนกับ Dashboard ส่วนผู้ตรวจที่ว่างแสดงเป็น “ไม่ระบุผู้ตรวจ” เหมือนแถวในตารางผู้ตรวจ × ประเภทคดีของ Dashboard (หัวข้อ 3.3.7) เพื่อไม่ให้สับสนกับประเภทคดี “ไม่ระบุ” ในบรรทัดเดียวกัน

ข้อความสรุปเป็นข้อความธรรมดา (text message) ไม่ใช้ Flex Message ด้วยเหตุผล 3 ข้อ คือ ข้อความธรรมดาแสดงในการแจ้งเตือนของโทรศัพท์และอ่านได้ทันที ผู้ใช้คัดลอกตัวเลขไปใช้ต่อได้ และรายงานมีเพียงรายการตัวเลขสั้น ๆ ซึ่งอ่านบนโทรศัพท์ได้โดยไม่ต้องจัดรูปแบบเป็นตาราง

3.6.3 ตัวอย่างข้อความ

ตัวอย่างข้อความสรุปรายเดือนต่อไปนี้ได้จากการรัน `formatSummary_` กับข้อมูลจำลอง (สรุปเดือนตุลาคม 2569 ที่ส่งวันที่ 1 พฤศจิกายน) ลิงก์ Dashboard เป็นลิงก์สมมติ ภาพข้อความเมื่อแสดงในแอปพลิเคชัน LINE แสดงในรูปที่ 3.10

```
สรุปคดีส่งตรวจ DNA เดือน ต.ค. 2569
(นับตามวันที่รับ 1 ต.ค. 2569 - 31 ต.ค. 2569)

คดีทั้งหมด 14 คดี
ของกลางรวม 57 ชิ้น

แยกตามประเภทคดี
- ทรัพย์ 4
- ชีวิตและร่างกาย 3
- พรบ.อาวุธปืน 3
- เพศ 2
- ยาเสพติด 1
- อื่น ๆ 1
```

```
แยกตามผู้ตรวจ (จำนวนคดี)
- เจ้าหน้าที่ ก 4: พรบ.อาวุธปืน 3, ทรัพย์ 1
- เจ้าหน้าที่ ค 3: ทรัพย์ 1, ยาเสพติด 1, อื่น ๆ 1
- เจ้าหน้าที่ ง 3: ชีวิตและร่างกาย 2, ทรัพย์ 1
- เจ้าหน้าที่ จ 2: ชีวิตและร่างกาย 1, เพศ 1
- เจ้าหน้าที่ ข 1: ทรัพย์ 1
- ไม่ระบุผู้ตรวจ 1: เพศ 1

ดูรายละเอียด: https://script.google.com/macros/s/MOCK-DEPLOYMENT/exec
(เปิดได้เฉพาะผู้มีสิทธิ์)
```

[รอใส่ภาพ: รูปที่ 3.10]

ตัวอย่างข้อความสรุปรายเดือนในกลุ่ม LINE (ภาพจากข้อมูลจำลอง)

รูปที่ 3.10 ตัวอย่างข้อความสรุปรายเดือนในกลุ่ม LINE

3.6.4 ความสอดคล้องกับ Dashboard

ตัวเลขในรายงานต้องตรงกับตัวเลขที่ผู้ใช้เห็นใน Dashboard เมื่อเลือกช่วงเวลาเดียวกัน คำสั่งสรุปคดีค้างของแชตบอตและรายงานอัตโนมัติจึงเรียกใช้**ฟังก์ชันคำนวณชุดเดียวกัน** ใน `Stats.gs` (`Stats.summarize` และ `Stats.pendingByStatus`) ไม่มีโค้ดคำนวณซ้ำใน `Summary.gs` หรือ `LineBot.gs` ส่วน Dashboard คำนวณในเบราว์เซอร์ด้วยโค้ดชุดเดิมของต้นแบบ `Stats.gs` จึงเขียนให้ใช้กติกาเดียวกับ Dashboard ทุกข้อ ได้แก่

การกรองแถวที่นับในสถิติ การจัดเดือนตามวันที่รับ การจัดกลุ่มค่าว่าง และการจัดกลุ่มสถานะ ความตรงกันของตัวเลขระหว่างข้อความสรุป คำตอบคดีค้าง และหน้า Dashboard ตรวจด้วยชุดทดสอบ ดังจะกล่าวในบทที่ 4

3.7 การออกแบบด้านความปลอดภัยและ PDPA

ข้อมูลคดีมีข้อมูลส่วนบุคคล เช่น ชื่อเจ้าหน้าที่ผู้ตรวจ และระบบยังเกี่ยวข้องกับอีเมลของผู้ใช้ Dashboard รวมทั้ง `userId` ของผู้ส่งข้อความที่มากับเหตุการณ์ webhook [21] จึงต้องจัดการตามพระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562 [7] การออกแบบใช้หลักการต่อไปนี้

1) **สิทธิ์น้อยที่สุด (least privilege)** Sheet ไม่เปิดสาธารณะ แชร์แบบอ่านอย่างเดียวเฉพาะคนในรายชื่อ Dashboard เปิดได้เฉพาะอีเมลในแท็บผู้ใช้และตรวจซ้ำทุกครั้งที่ขอข้อมูล สคริปต์ขอสิทธิ์อ่าน Sheet อย่างเดียว (หัวข้อ 3.4) แชตบอตตอบเฉพาะกลุ่มที่อนุญาต และตอบเฉพาะข้อมูลที่จำเป็นต่อคำถาม ฟังก์ชันผู้ดูแลรันได้เฉพาะบัญชี `ADMIN_EMAIL` (หัวข้อ 3.4.8)
2) **ใช้ข้อมูลเท่าที่จำเป็น (data minimisation)** ลบคอลัมน์ User ID (LINE) ออกจาก Sheet จริง Dashboard ส่งไปยังเบราว์เซอร์เฉพาะ 14 คอลัมน์ที่ใช้ คำตอบสถานะคดีมีเพียงสถานะ ประเภทคดี จำนวนของกลาง และวันที่ 3 รายการ ส่งให้ AI เพียงข้อความคำถามที่ตัดข้อมูลออกแล้ว ไม่ส่งข้อมูลคดี สรุปอัตโนมัติแสดงเป็นจำนวนรวม ไม่แสดงเลขคดี

ระบบไม่เก็บ `userId` ของผู้ส่งข้อความ ไม่ส่งให้ Gemini และไม่บันทึกข้อความของผู้ใช้ลงในบันทึกการทำงาน

3) **เก็บความลับไว้นอกโค้ด** LINE channel access token, รหัสลับของ webhook, Gemini API key, รหัส Sheet และรหัสกลุ่มที่อนุญาต เก็บใน Script Properties ซึ่งเป็นที่เก็บค่าที่ใช้ได้เฉพาะภายในสคริปต์ [13] โค้ดใน repository จึงไม่มีค่าความลับ
4) **พัฒนาด้วยข้อมูลจำลอง** การพัฒนา การทดสอบ และภาพหน้าจอในเล่มใช้ข้อมูลจำลองเท่านั้น ไม่นำข้อมูลจริงเข้า repository การนำโค้ดขึ้นใช้กับ Sheet จริงและการส่งข้อความเข้ากลุ่ม LINE จริงต้องให้เจ้าของโครงงานอนุมัติก่อน (`PUSH_ENABLED` หัวข้อ 3.6.1)
5) **ไม่มี DNA profile** ระบบติดตามเฉพาะสถานะและสถิติของงาน ไม่เก็บ ไม่ประมวลผล และไม่แสดง DNA profile หรือผลการตรวจพิสูจน์ในส่วนใดของระบบ

[รอใส่: มาตราของ พ.ร.บ.คุ้มครองข้อมูลส่วนบุคคล ที่เกี่ยวข้องกับหลักการแต่ละข้อ — ต้องตรวจจากต้นฉบับราชกิจจานุเบกษาก่อนอ้าง]

บทที่ 4
การทดสอบระบบและวิเคราะห์ผล

บทนี้อธิบายวิธีการทดสอบระบบที่ออกแบบไว้ในบทที่ 3 และผลการทดสอบ ครอบคลุมความถูกต้องของข้อมูลจำลองและ Dashboard การควบคุมสิทธิ์ ความถูกต้องของสถิติสรุป การทำงานของแชตบอต LINE และรายงานสรุปอัตโนมัติ และการตรวจตามหลักการคุ้มครองข้อมูลส่วนบุคคล การทดสอบทั้งหมดใช้ข้อมูลจำลองและบริการจำลอง ยังไม่ได้ทดสอบกับบัญชี Google จริง กลุ่ม LINE จริง หรือ Gemini API จริง ข้อจำกัดนี้อธิบายไว้ในหัวข้อ 4.1.3 และ 4.7.2

4.1 วิธีการและสภาพแวดล้อมการทดสอบ

4.1.1 ข้อมูลจำลอง

ข้อมูลคดีของหน่วยงานเป็นข้อมูลจริงที่มีชื่อเจ้าหน้าที่ สถานีตำรวจ และเลขคดี การนำข้อมูลนี้มาใช้ทดสอบหรือแสดงในเล่มจะเป็นการเปิดเผยข้อมูลส่วนบุคคลเกินวัตถุประสงค์ตามพระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562 [7] โครงงานจึงทดสอบด้วยข้อมูลจำลองเท่านั้น (หัวข้อ 3.7)

ข้อมูลจำลองสร้างด้วยสคริปต์ `tools/make_mock_data.py` ซึ่งเขียนไฟล์ด้วยไลบรารี openpyxl [48] ได้ไฟล์ 2 ไฟล์ คือ `cases.xlsx` (ข้อมูลรายคดีและแท็บ “ผู้ใช้”) และ `lab.xlsx` (ข้อมูลห้องปฏิบัติการ) โครงสร้างแท็บและคอลัมน์ตรงกับ `docs/data-schema.md` ทุกคอลัมน์ สคริปต์กำหนดค่าเริ่มต้นของตัวสุ่ม (seed) คงที่เป็น 20260101 และกำหนด “วันนี้” ของข้อมูลคงที่เป็นวันที่

31 ตุลาคม 2569 แทนการอ่านวันที่ของเครื่อง จึงสร้างซ้ำได้ไฟล์เดิมทุกไบต์ ค่าทุกค่าเป็นของสมมติ เช่น ผู้ตรวจ “เจ้าหน้าที่ ก” ถึง “เจ้าหน้าที่ จ” สถานี “สภ.ตัวอย่าง1” ถึง “สภ.ตัวอย่าง15” เลขหนังสือ ตย.0000/… และอีเมลโดเมน @example.com ส่วนค่าความลับที่ชุดทดสอบใช้ เช่น channel access token และ API key เป็นข้อความสมมติที่ขึ้นต้นด้วย mock- ลักษณะของข้อมูลจำลองสรุปในตารางที่ 4.1

ตารางที่ 4.1 ลักษณะของข้อมูลจำลองที่ใช้ทดสอบ

รายการ	**ค่าในข้อมูลจำลอง**	**วัตถุประสงค์ในการทดสอบ**
จำนวนแถวในแท็บข้อมูลคดี	731 แถว วันที่รับ 1 ม.ค. 2565 – 30 ต.ค. 2569 (8–18 คดีต่อเดือน)	ปริมาณข้อมูลใกล้เคียงการใช้งานหลายปี
สถานะคดี	รับผลไปแล้ว 692 แถว, เสร็จแล้ว รอรับผล 17 แถว, กำลังดำเนินการ 22 แถว	ทดสอบการแยกคดีค้าง 2 กลุ่ม
ค่าว่าง (กรณีขอบ)	ประเภทคดีว่างร้อยละ 14.0 ผู้ตรวจว่างร้อยละ 15.2	ทดสอบกลุ่ม “ไม่ระบุ”
วันที่ข้ามปีและข้ามปีงบประมาณ	มีคดีรับวันที่ 31 ธ.ค., 1 ม.ค., 30 ก.ย. และ 1 ต.ค. ครบทุกปีในช่วงข้อมูล	ทดสอบการจัดเดือนและปีของคดี
แถวที่ไม่นับในสถิติ	3 แถว (คอลัมน์ “นับในสถิติ” = 0)	ทดสอบการกรองแถว
แถวที่มีข้อสังเกต	12 แถว 4 ชนิด	ทดสอบหน้าตรวจสอบข้อมูล
ของกลางเกินเกณฑ์	5 คดีเกินเกณฑ์ 40 ชิ้นที่ตั้งไว้ในชีต	ทดสอบแถบหมายเหตุของกลาง

รายการ	ค่าในข้อมูลจำลอง	วัตถุประสงค์ในการทดสอบ
		ผิดปกติ
ข้อมูลห้องปฏิบัติการ	แท็บข้อมูลเคส 58 แถว, ข้อมูลงบ 39 แถว, ข้อมูล Lot 86 แถว, หมายเหตุ 4 แถว	ทดสอบเมนูงบประมาณและ Lot พัสดุ
คอลัมน์ User ID (LINE)	ไม่มี (ตัดออกตามการตัดสินใจของเจ้าของโครงงาน)	ยืนยันว่าระบบไม่พึ่งคอลัมน์นี้

4.1.2 เครื่องมือและขั้นตอนการทดสอบ

โค้ดของระบบเขียนสำหรับ Google Apps Script ซึ่งรันได้เฉพาะบนเซิร์ฟเวอร์ของ Google ชุดทดสอบจึงจำลองสภาพแวดล้อมของ Apps Script ขึ้นในเครื่อง โดยแบ่งเป็น 3 ส่วน ดังนี้

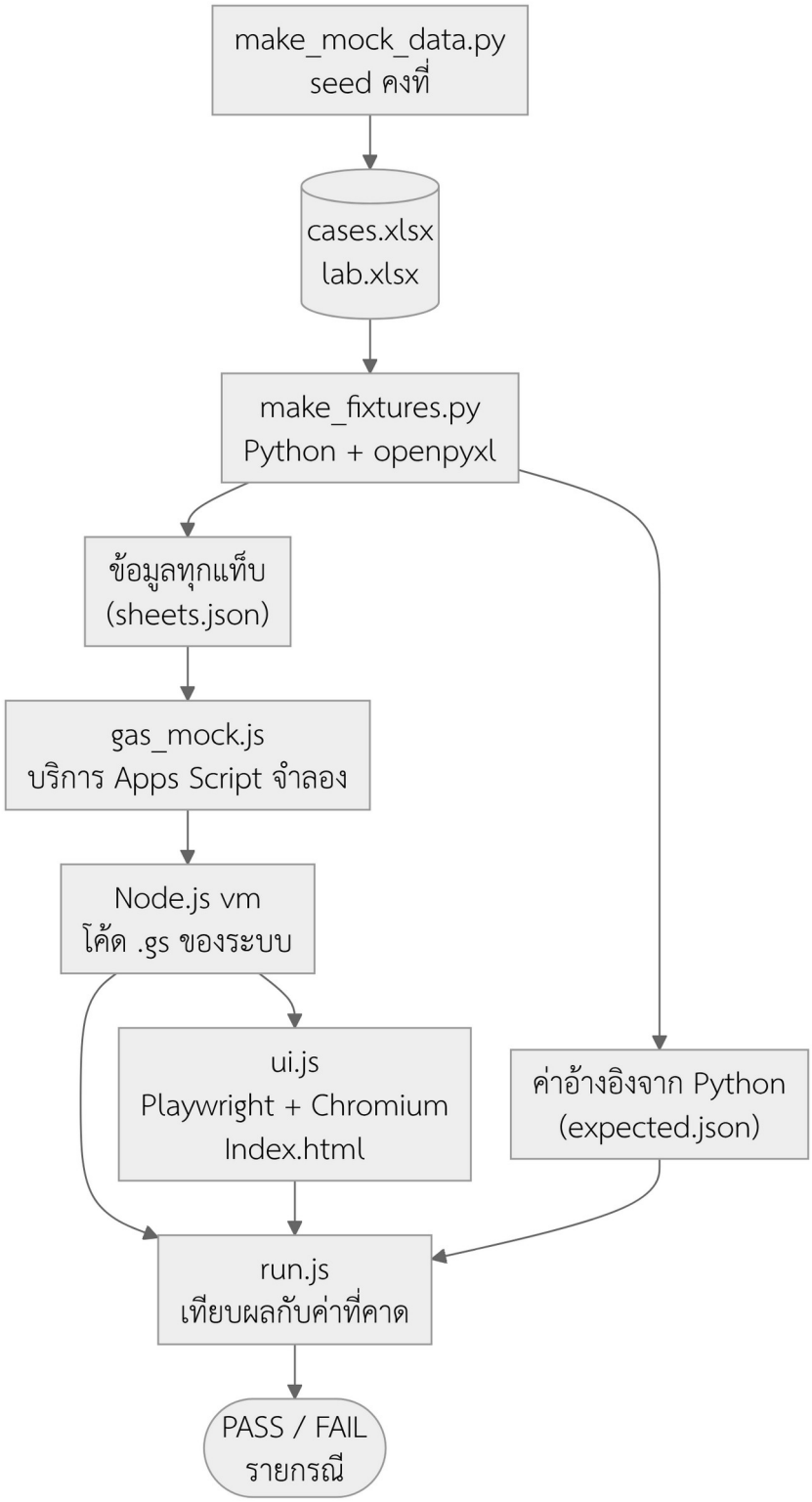
1) **ส่วนเซิร์ฟเวอร์** โหลดไฟล์ `Stats.gs` `Code.gs` `Intent.gs` `LineBot.gs` และ `Summary.gs` ด้วยโมดูล `vm` ของ Node.js [49] ร่วมกับตัวจำลอง (mock) ของบริการ SpreadsheetApp, Session, PropertiesService, CacheService, LockService, ContentService, HtmlService, ScriptApp, Utilities และ UrlFetchApp ในไฟล์ `apps-script/test/gas_mock.js` ตัวจำลองของ SpreadsheetApp ป้อนข้อมูลจากไฟล์ข้อมูลจำลอง ส่วนตัวจำลองของ UrlFetchApp บันทึกทุกคำขอที่ระบบส่งออก ตอบแทน LINE และ Gemini ตามกรณีทดสอบ และโยนข้อผิดพลาดถ้าระบบพยายามเรียก

URL อื่น การทดสอบจึงตรวจได้ว่าระบบส่งอะไรออกไปบ้าง โดยไม่มีคำขอใดออกไปยังเครือข่ายจริง

2) **ค่าอ้างอิงที่คำนวณอิสระ** สคริปต์ `apps-script/test/make_fixtures.py` อ่านไฟล์ข้อมูลจำลองด้วย Python และ openpyxl แล้วคำนวณผลที่ควรได้ เช่น จำนวนคดี จำนวนของกลาง จำนวนคดีแยกประเภท ตารางผู้ตรวจ × ประเภทคดี และคดีค้าง โดยไม่ใช้โค้ด JavaScript ของระบบ ค่านี้ใช้เป็นตัวตัดสินผลการทดสอบ (test oracle) ของการคำนวณ

3) **ส่วนหน้าเว็บ** สคริปต์ `apps-script/test/ui.js` เปิด `Index.html` ด้วยเบราว์เซอร์ Chromium แบบไม่มีหน้าต่าง (headless) ผ่านเครื่องมือ Playwright [50] และแทน `google.script.run` ด้วยตัวจำลองที่คืนผลของ `getBook` จริงจากส่วนเซิร์ฟเวอร์ คำขอทุกรายการของหน้าเว็บถูกดักและตอบจากไฟล์ในเครื่อง คำขอที่ไม่ได้คาดไว้ถูกบันทึกเป็นข้อผิดพลาด ฟอนต์ใช้ฟอนต์ไทยของเครื่องแทนฟอนต์จากอินเทอร์เน็ต

ความสัมพันธ์ของส่วนต่าง ๆ แสดงในรูปที่ 4.1 สคริปต์หลัก `apps-script/test/run.js` รันทั้ง 3 ส่วนด้วยคำสั่งเดียว แล้วพิมพ์ผล PASS หรือ FAIL ของแต่ละกรณี



รูปที่ 4.1 โครงสร้างของชุดทดสอบ ตั้งแต่การสร้างข้อมูลจำลอง จนถึงการตัดสินผล

ผลทดสอบในบทนี้มาจาก 2 แหล่ง คือ (1) การทดสอบข้อมูลจำลองและ Dashboard ต้นแบบ (`dashboard/index.html`) 18 กรณี ซึ่งผู้ตรวจคุณภาพของทีมทดสอบด้วย Chromium ผ่าน Playwright และบันทึกไว้ใน `docs/test-results/01-mock-data.md` และ (2) ชุดทดสอบอัตโนมัติ `run.js` 96 กรณี ซึ่งรันครั้งล่าสุดเพื่อจัดทำบทนี้ เครื่องมือและสภาพแวดล้อมของการรันครั้งนี้แสดงในตารางที่ 4.2

ตารางที่ 4.2 เครื่องมือและสภาพแวดล้อมของการรันชุดทดสอบอัตโนมัติ

รายการ	ค่า
วันที่รัน	3 พฤศจิกายน 2569
รุ่นของโค้ด	commit `bab2b1c` (ใช้คำ “ไม่ระบุ” กับค่าว่างทั้ง Dashboard และแชตบอต)
คำสั่ง	`node apps-script/test/run.js`
ระบบปฏิบัติการ	Linux (x86_64)
Node.js	22.23.4 (โมดูล `vm`)
Python และ openpyxl	Python 3.13.18, openpyxl 3.1.5
เบราว์เซอร์	Playwright 1.63.0 กับ Chromium 153 แบบ headless
ความกว้างหน้าจอที่ทดสอบ	1,440 พิกเซล (คอมพิวเตอร์) และ 390 พิกเซล (โทรศัพท์)
ผลรวม	ผ่าน 96 จาก 96 กรณี ใช้เวลาประมาณ 24 วินาที

4.1.3 สิ่งที่ยังไม่ได้ทดสอบ

การทดสอบด้วยบริการจำลองยืนยันได้ว่าโค้ดทำงานตามที่ออกแบบเมื่อบริการภายนอกตอบตามที่เอกสารระบุ แต่ไม่ได้ยืนยันพฤติกรรมจริงของแพลตฟอร์ม รายการที่ยังไม่ได้ทดสอบแสดงในตารางที่ 4.3 ซึ่งต้องทดสอบหลังนำระบบขึ้นใช้งานจริงตามคู่มือ `apps-script/README.md` (หัวข้อ 4.8)

ตารางที่ 4.3 รายการที่ยังไม่ได้ทดสอบและเหตุผล

รายการ	เหตุผลที่ยังไม่ได้ทดสอบ
การเปิด Dashboard ด้วยบัญชี Google จริง หน้าขออนุญาต OAuth และการแชร์ Sheet จริง	ทดสอบด้วยตัวจำลองของ Session และ SpreadsheetApp เท่านั้น ค่าอีเมลว่างเมื่อ deploy แบบ Execute as: Me อ้างอิงจากเอกสารของ Google [12]
การรับ webhook จาก LINE Platform จริง ปุ่ม Verify และการตอบแบบเปลี่ยนเส้นทาง (HTTP 302) ของ Apps Script	ยังไม่ได้ deploy จริง เหตุการณ์ webhook สร้างตามรูปแบบในเอกสารของ LINE
การส่ง reply และ push ไปยังกลุ่ม LINE จริง	การส่งข้อความเข้ากลุ่มจริงต้องให้เจ้าของโครงงานอนุมัติก่อน
ความถูกต้องของการแยกเจตนาโดย Gemini จริง	ตัวจำลองของ Gemini คืนเจตนาที่กำหนดไว้ล่วงหน้า จึงทดสอบได้เฉพาะการตัดข้อมูลก่อนส่งและการจัดการผลที่ได้กลับมา ไม่ได้วัดความแม่นยำของแบบจำลอง
การทำงานของตัวตั้งเวลาจริงของ Apps Script	ทดสอบโดยเรียก `dailySummaryJob` พร้อมวันเวลาจำลอง
ความเร็ว โควตา และการใช้งาน	ชุดทดสอบไม่ได้วัดเวลาตอบหรือ

รายการ	เหตุผลที่ยังไม่ได้ทดสอบ
พร้อมกันหลายคน	จำลองผู้ใช้พร้อมกัน
การทดลองใช้และความพึงพอใจของเจ้าหน้าที่	ยังไม่ได้ให้เจ้าหน้าที่จริงทดลองใช้

4.2 การทดสอบความถูกต้องของข้อมูลจำลองและ Dashboard

4.2.1 ข้อมูลจำลองและ Dashboard ต้นแบบ

การทดสอบชุดแรกตรวจว่าข้อมูลจำลองมีลักษณะตามที่ออกแบบ และ Dashboard ต้นแบบที่อ่านไฟล์ .xlsx ในเบราว์เซอร์แสดงตัวเลขตรงกับค่าที่คำนวณด้วย Python วิธีทดสอบคือเสิร์ฟโฟลเดอร์ `dashboard/` ในเครื่อง เปิด `index.html` ด้วย Chromium เลือกชุดข้อมูลตัวอย่างทั้งสองชุด และเปิดทุกเมนู ผลทั้ง 18 กรณีสรุปในตารางที่ 4.4

ตารางที่ 4.4 ผลการทดสอบข้อมูลจำลองและ Dashboard ต้นแบบ

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
รันสคริปต์สร้างข้อมูล 2 ครั้ง	ไฟล์เหมือนเดิมทุกไบต์	ค่า SHA-1 ตรงกันทั้งสองไฟล์	ผ่าน
จำนวนแถวและช่วงวันที่	600–800 แถว ช่วงปี 2565–2569	731 แถว 1 ม.ค. 2565 – 30 ต.ค. 2569	ผ่าน
ความสอดคล้องของวันที่กับสถานะ	วันที่รับ ≤ ตรวจเสร็จ ≤ มารับผล และไม่มีวันที่ที่ขัดกับสถานะ	ขัดแย้ง 0 แถว	ผ่าน
ค่าสถานะ	มีเฉพาะ 3 ค่าตามแม่แบบ	692 / 17 / 22 แถว	ผ่าน

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
ประเภทคดีและผู้ตรวจว่าง	ว่างประมาณร้อยละ 15	ร้อยละ 14.0 และ 15.2 Dashboard แสดง “ไม่ระบุ” 101 และ 111 คดี	ผ่าน
วันที่ข้ามปีและข้ามปีงบประมาณ	มีคดีรับ 31 ธ.ค., 1 ม.ค., 30 ก.ย., 1 ต.ค.	มีครบทุกปีในช่วงข้อมูล	ผ่าน
ของกลางเกินเกณฑ์	อ่านเกณฑ์ 40 ชิ้นจากชีต และแจ้ง 5 คดี	แจ้ง 5 คดี รวม 372 ชิ้น	ผ่าน
ภาพรวมทุกปี	คดี 728 (ไม่นับ 3 แถว) ของกลาง 3,832 ชิ้น	ตรงตามคาด เฉลี่ย 5.28 ชิ้นต่อคดี	ผ่าน
กรองปี 2568	154 คดี	154 คดี ของกลาง 789 ชิ้น	ผ่าน
เมนูพื้นที่และประเภทคดี	5 จังหวัด 15 สถานี	5 จังหวัด 15 สถานี ระบุประเภท 627 คดี ระบุผู้ตรวจ 617 คดี	ผ่าน
เมนูระยะเวลาและงานค้าง	คดีค้าง 39 คดี (22 + 17)	39 คดี มัธยฐานรับถึงตรวจเสร็จ 20 วัน	ผ่าน
เมนูตรวจสอบข้อมูลและเมนูประเภท/ผู้ตรวจรายเดือน (2 กรณี)	ข้อสังเกต 12 แถว 4 ชนิด ไม่นับ 3 แถว และกราฟกับตารางแสดงได้	ตรงตามคาด กราฟ 1 ภาพ ตาราง 138 แถว	ผ่าน
ชุดข้อมูลห้องปฏิบัติการ ปีงบประมาณ 2569	ตัวอย่าง 2,050 ของกลาง 1,012 งบประมาณ 2,282,300 บาท Lot 17	ตรงทุกค่า	ผ่าน
เมนูงบประมาณและ	แสดงกราฟและตาราง	งบประมาณ: กราฟ 2	ผ่าน

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
Lot พัสดุ		ตาราง 7 แถว · Lot: กราฟ 2	
ปุ่มเลือกปี	ข้อมูลห้องปฏิบัติการ 2565–2570 ข้อมูลคดี 2565–2569	ตรงตามคาด	ผ่าน
ข้อผิดพลาด JavaScript	ไม่มี pageerror	ไม่มี pageerror (พบคำขอไฟล์ตั้งค่าที่ไม่บังคับได้ผล 404 ซึ่งแก้แล้วในตารางที่ 4.6)	ผ่าน
ข้อมูลในไฟล์จำลองเป็นค่าสมมติ	ไม่มีชื่อ เลข หรืออีเมลจริง ไม่มี DNA profile	ตรงตามคาด	ผ่าน

ข้อสังเกตจากตารางที่ 4.4 คือ ร้อยละของค่าว่างในข้อมูลจำลองกำหนดให้ใกล้เคียงข้อสังเกตในหัวข้อ 3.2.4 ที่ว่าคอลัมน์ประเภทคดีและผู้ตรวจยังกรอกไม่ครบ จำนวน 101 และ 111 คดีที่ Dashboard แสดงเป็น “ไม่ระบุ” รวมกับคดีที่ระบุประเภท 627 คดีและระบุผู้ตรวจ 617 คดี ได้ 728 คดีเท่ากับจำนวนคดีที่นับในสถิติ จึงไม่มีคดีหายไประหว่างการจัดกลุ่ม

4.2.2 การอ่านข้อมูลฝั่งเซิร์ฟเวอร์

กลุ่มนี้ทดสอบฟังก์ชันใน `Code.gs` ที่อ่าน Sheet และส่งข้อมูลให้หน้าเว็บ (หัวข้อ 3.4.4–3.4.6) ผลทั้ง 8 กรณีแสดงในตารางที่ 4.5

ตารางที่ 4.5 ผลการทดสอบการอ่านข้อมูลฝั่งเซิร์ฟเวอร์

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
`getSources`	คืนแหล่งข้อมูล 2 แหล่งและอีเมลผู้ใช้	ตรงตามคาด	ผ่าน
`getBook("cases")` ส่งเฉพาะคอลัมน์ที่ใช้	ไม่มี User ID (LINE) เลขหนังสือนำส่ง ลงวันที่ ตั้งค่า	14 คอลัมน์ตามตารางที่ 3.8	ผ่าน
แปลงวันที่เป็นเลข serial	วันที่รับของแถวแรก (1 ม.ค. 2565) = 44562	44562	ผ่าน
จำนวนแถวข้อมูลคดี	731 แถวและแถวหัวตาราง	732 แถว (รวมหัว)	ผ่าน
`getBook("lab")`	มี 4 แท็บของข้อมูลห้องปฏิบัติการ	ตรงตามคาด	ผ่าน
แคช: เรียก `getBook` ซ้ำ	ครั้งที่ 2 ไม่เปิด Sheet อีก ผลเหมือนเดิม	การเรียกครั้งแรกเปิด Sheet 2 ครั้ง ครั้งที่ 2 ไม่เปิดเพิ่ม ข้อมูลถูกแบ่งเก็บในแคช 4 ชิ้น	ผ่าน
`checkSetup`	บอกจำนวนแถวของทุกแท็บ ไม่มีค่าข้อมูลคดี	ข้อมูลคดี 731 แถว ข้อมูลเคส 58 ข้อมูลงบ 39 ข้อมูล Lot 86 หมายเหตุ 4 แถว	ผ่าน
Sheet ที่ยังมีคอลัมน์ User ID (LINE)	`getBook` และคำตอบของแชตบอตไม่มีคอลัมน์และค่านี้	ตรงตามคาด	ผ่าน

กรณีสุดท้ายในตารางที่ 4.5 จำลองสถานการณ์ที่ Sheet จริงยังไม่ได้ลบคอลัมน์ User ID (LINE) โดยแทรกคอลัมน์ที่มีค่า

สมมติเข้าไปในหน่วยความจำระหว่างทดสอบ ผลยืนยันว่าการเลือกส่งเฉพาะคอลัมน์ที่กำหนดทำงานแม้คอลัมน์นี้ยังอยู่

4.2.3 หน้าเว็บ Dashboard รุ่น Apps Script

กลุ่มนี้เปิด `Index.html` ด้วย Chromium โดยใช้ข้อมูลที่ได้จากส่วนเซิร์ฟเวอร์ในหัวข้อ 4.2.2 กรณีที่ตรวจการแสดงผลของแต่ละเมนูมี 17 กรณี สรุปในตารางที่ 4.6 ส่วนกรณีที่เทียบตัวเลขบนหน้าเว็บกับ `Stats.gs` และแชตบอตอยู่ในหัวข้อ 4.4

ตารางที่ 4.6 ผลการทดสอบการแสดงผลของ Dashboard รุ่น Apps Script

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
เปิดหน้าแรก	แสดงชุดข้อมูลคดีเป็นค่าเริ่มต้น	คดีทั้งหมด 728 สถานะ “เชื่อมสด · Google Sheet · อัปเดต …”	ผ่าน
ป้ายแกน x เมื่อเลือก “ทั้งหมด” ที่ความกว้าง 1,440 พิกเซล	ป้ายเดือนไม่ซ้อนกัน	กราฟละ 10 ป้าย ซ้อน 0	ผ่าน
เมนูของข้อมูลคดี 5 เมนู (พื้นที่และประเภทคดี, ประเภท/ผู้ตรวจราย เดือน, ระยะเวลาและงานค้าง, ตรวจสอบข้อมูล, แหล่งข้อมูล) (5 กรณี)	แสดงได้ ไม่มีข้อความผิดพลาด	ทุกเมนูแสดงได้	ผ่าน
เมนูแหล่งข้อมูล	แสดงอีเมลผู้ใช้ ไม่มีปุ่มเพิ่มลิงก์ Google Sheets	ตรงตามคาด	ผ่าน

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
เปิดไฟล์ Excel ในเครื่อง (`cases.xlsx`)	อ่านได้ ตัวเลขเท่ากับข้อมูลจากเซิร์ฟเวอร์	คดี 728 ตรงกัน	ผ่าน
ชุดข้อมูลห้องปฏิบัติการ เมนูภาพรวม งบประมาณ และ Lot พัสดุ (3 กรณี)	แสดงได้ ไม่มีข้อความผิดพลาด	ภาพรวม 3 กราฟ งบประมาณ 2 กราฟ Lot พัสดุ 2 กราฟ	ผ่าน
หน้าจอโทรศัพท์ 390 พิกเซล เลือก “ทั้งหมด”	ป้ายแกน x ไม่ซ้อน ไม่มีแถบเลื่อนแนวนอน	กราฟละ 5 ป้าย	ผ่าน
หน้า “ไม่มีสิทธิ์เข้าถึง”	แสดงอีเมลและผู้ที่ต้องติดต่อเป็นภาษาไทย	ตรงตามคาด	ผ่าน
Dashboard ต้นแบบไม่ขอไฟล์ `sources.json`	ไม่มีคำขอไฟล์นี้ (เดิมได้ผล 404)	ตรงตามคาด	ผ่าน
Dashboard ต้นแบบ ป้ายแกน x เมื่อเลือก “ทั้งหมด”	ป้ายเดือนไม่ซ้อน คดีทั้งหมด 728	กราฟละ 10 ป้าย ซ้อน 0	ผ่าน
ข้อผิดพลาดในคอนโซลและคำขอออกนอกเครื่อง (ทุกหน้า)	0 รายการ	0 รายการ	ผ่าน

ข้อมูลจำลองมี 58 เดือน เมื่อเลือก “ทั้งหมด” หน้าจอขนาด 1,440 พิกเซลแสดงป้ายเดือนทุก 6 เดือน และหน้าจอโทรศัพท์แสดงทุก 12 เดือน ป้ายจึงไม่ซ้อนกัน ตัวอย่างหน้าจอที่ได้จากการทดสอบแสดงในรูปที่ 4.2 และรูปที่ 4.3

[รอใส่ภาพ: รูปที่ 4.2]
หน้าภาพรวมของ Dashboard รุ่น Apps Script เมื่อกรองปี 2568 ขณะทดสอบ (ภาพจากข้อมูลจำลอง)

รูปที่ 4.2 หน้าภาพรวมของ Dashboard รุ่น Apps Script เมื่อกรองปี 2568 ขณะทดสอบ

[รอใส่ภาพ: รูปที่ 4.3]
หน้าภาพรวมบนหน้าจอโทรศัพท์ความกว้าง 390 พิกเซล (ภาพจากข้อมูลจำลอง)

รูปที่ 4.3 หน้าภาพรวมบนหน้าจอโทรศัพท์ความกว้าง 390 พิกเซล

4.2.4 ตัวชี้วัดบน Dashboard เทียบกับค่าที่คำนวณอิสระ

ตัวชี้วัดหลักที่ Dashboard แสดงเทียบกับค่าที่คำนวณด้วย Python ซึ่งไม่ใช้โค้ดของระบบ แสดงในตารางที่ 4.7 ทุกค่าตรงกัน

ตารางที่ 4.7 ตัวชี้วัดบน Dashboard เทียบกับค่าที่คำนวณด้วย Python

ตัวชี้วัด	ค่าที่คำนวณด้วย Python	ค่าที่ Dashboard แสดง	ผล
คดีทั้งหมด (ทุกปี)	728	728	ตรงกัน
ของกลางรวม (ทุกปี)	3,832 ชิ้น	3,832 ชิ้น	ตรงกัน
คดีปี 2565 / ของกลาง	138 / 621	138 / 621	ตรงกัน

ตัวชี้วัด	ค่าที่คำนวณด้วย Python	ค่าที่ Dashboard แสดง	ผล
คดีปี 2568 / ของกลาง	154 / 789	154 / 789	ตรงกัน
คดีค้าง: กำลังดำเนินการ / เสร็จแล้ว รอรับผล	22 / 17 (รวม 39)	22 / 17 (รวม 39)	ตรงกัน
ข้อมูลห้องปฏิบัติการ ปีงบประมาณ 2569: ตัวอย่าง / ของกลาง	2,050 / 1,012	2,050 / 1,012	ตรงกัน
ข้อมูลห้องปฏิบัติการ ปีงบประมาณ 2569: งบประมาณ / Lot	2,282,300 บาท / 17	2,282,300 บาท / 17	ตรงกัน

4.3 การทดสอบการควบคุมสิทธิ์

4.3.1 การควบคุมสิทธิ์ของ Dashboard

กลุ่มนี้ทดสอบ `checkAccess_` และ `requireAccess_` ตามกรณีในตารางที่ 3.6 โดยกำหนดอีเมลของผู้เปิด สิทธิ์การอ่าน Sheet และ Script Properties ผ่านตัวจำลอง ผลทั้ง 11 กรณีแสดงในตารางที่ 4.8

ตารางที่ 4.8 ผลการทดสอบการควบคุมสิทธิ์ของ Dashboard

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
อีเมลอยู่ในแท็บ “ผู้ใช้”	ได้หน้า Dashboard	ได้หน้า Dashboard พร้อมชื่อหน้าและ viewport ใช้ X-Frame-Options เริ่มต้น	ผ่าน

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
อีเมลเดียวกันแต่เป็นตัวพิมพ์ใหญ่และมีช่องว่าง	เปิดได้	ตรงตามคาด	ผ่าน
อีเมลไม่อยู่ในรายชื่อ (`not-listed`)	หน้า “ไม่มีสิทธิ์เข้าถึง” พร้อมอีเมลและผู้ที่ต้องติดต่อ ไม่มีเนื้อหา Dashboard	ตรงตามคาด	ผ่าน
อีเมลว่าง (`no-email`)	ปฏิเสธ และแจ้งว่าระบบไม่ทราบอีเมล	ตรงตามคาด	ผ่าน
อีเมลที่มีแท็ก `<script>`	แสดงแบบแปลงอักขระพิเศษแล้ว	ตรงตามคาด	ผ่าน
ผู้ใช้ไม่ได้รับแชร์ Sheet (`no-sheet-access`)	ปฏิเสธ พร้อมเหตุผล	ปฏิเสธ แจ้งว่ายังไม่ได้รับสิทธิ์ผู้มีสิทธิ์อ่าน	ผ่าน
ไม่มีแท็บ “ผู้ใช้” (`no-users-tab`)	ปฏิเสธทุกคน	ตรงตามคาด	ผ่าน
ยังไม่ตั้ง `CASES_SHEET_ID` (`no-config`)	ปฏิเสธ และบอกให้ตั้งค่า	ตรงตามคาด	ผ่าน
เรียก `getBook` และ `getSources` โดยตรงโดยไม่มีสิทธิ์	โยนข้อผิดพลาด “ไม่มีสิทธิ์” ทุกฟังก์ชัน	ตรงตามคาด	ผ่าน
ลบอีเมลออกจากแท็บ “ผู้ใช้” ระหว่างใช้งาน	คำขอข้อมูลครั้งถัดไปหลังแคชสิทธิ์หมดอายุถูกปฏิเสธ	ตรงตามคาด	ผ่าน
`getBook` ด้วยชื่อแหล่งข้อมูลแปลก	โยนข้อผิดพลาด	ตรงตามคาด	ผ่าน

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
(`__proto__`, x)			

ตารางที่ 4.8 ครอบคลุมเหตุผลการปฏิเสธในตารางที่ 3.6 ครบทุกข้อ ยกเว้น `wrong-deployment` ซึ่งทดสอบในหัวข้อ 4.3.2 ทุกกรณีที่เงื่อนไขไม่ครบให้ผลเป็นการปฏิเสธ ตรงกับหลักการปฏิเสธเมื่อไม่แน่ใจ (fail closed) ที่ออกแบบไว้ หน้า “ไม่มีสิทธิ์เข้าถึง” ที่ได้จากการทดสอบแสดงในรูปที่ 4.4

[รอใส่ภาพ: รูปที่ 4.4]
หน้า “ไม่มีสิทธิ์เข้าถึง” เมื่อเปิดด้วยอีเมลที่ไม่อยู่ในรายชื่อ (ภาพจากข้อมูลจำลอง)

รูปที่ 4.4 หน้า “ไม่มีสิทธิ์เข้าถึง” เมื่อเปิดด้วยอีเมลที่ไม่อยู่ในรายชื่อ

4.3.2 การป้องกันจาก deployment ของแชตบอตและฟังก์ชันผู้ดูแล

เนื่องจากแชตบอตอยู่ในโปรเจกต์เดียวกับ Dashboard แต่ deploy แบบ Execute as: Me (หัวข้อ 3.4.8) จึงทดสอบเพิ่มว่าลิงก์ของแชตบอตใช้เปิดข้อมูล Dashboard ไม่ได้ และฟังก์ชันผู้ดูแลถูกเรียกจากหน้าเว็บไม่ได้ ผลแสดงในตารางที่ 4.9

ตารางที่ 4.9 ผลการทดสอบการป้องกันจาก deployment ของแชตบอตและฟังก์ชันผู้ดูแล

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
เปิดลิงก์ของแชตบอตโดย	`doGet` คืนหน้าไม่มีสิทธิ์	ตรงตามคาด	ผ่าน

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
ไม่ล็อกอิน (ไม่ทราบอีเมลผู้เปิด)	`getBook` `getSources` และ `checkSetup` โยนข้อผิดพลาด		
ผู้ใช้ในรายชื่อเปิดลิงก์ของแชตบอต (Google ส่งอีเมลมา แต่สคริปต์ทำงานในนามเจ้าของ)	ปฏิเสธด้วยเหตุผล `wrong-deployment` และไม่เปิด Sheet ข้อมูล	ตรงตามคาด Sheet ถูกเปิด 0 ครั้ง	ผ่าน
`doPost` ตอบกลับทาง HTTP	เนื้อหาคำตอบเป็น `OK` ทุกกรณี ไม่มีข้อมูลคดี	ตรงตามคาด	ผ่าน
ผู้ใช้ Dashboard ที่ไม่ใช่ `ADMIN_EMAIL` เรียกฟังก์ชันผู้ดูแล 8 ฟังก์ชัน และกรณียังไม่ตั้ง `ADMIN_EMAIL`	โยนข้อผิดพลาดทุกฟังก์ชัน	ตรงตามคาด	ผ่าน
เรียก `dailySummaryJob` โดยไม่ผ่านตัวตั้งเวลา หรือส่ง `triggerUid` ปลอม	ปฏิเสธ ไม่ส่งข้อความ	ตรงตามคาด	ผ่าน

4.4 การทดสอบความถูกต้องของสถิติสรุป

4.4.1 สถิติรายเดือนและรายปี

กลุ่มนี้เทียบผลของ `Stats.summarize` กับค่าที่คำนวณด้วย Python ทุกค่า ได้แก่ จำนวนคดี จำนวนของกลาง จำนวนคดีแยกประเภท และจำนวนคดีของผู้ตรวจแต่ละคนแยกตามประเภท ช่วงเวลาที่เลือกครอบคลุมปีเต็ม เดือนธันวาคมต่อเดือนมกราคม (ข้ามปี) เดือนกันยายนต่อเดือนตุลาคม (ข้ามปีงบประมาณ) และเดือนล่าสุดของข้อมูล จากนั้นนำตัวเลขเดียวกันไปเทียบกับตาราง “สถิติ

รายเดือน” และตัวชี้วัดเมื่อกรองปีบนหน้า Dashboard ผลแสดงในตารางที่ 4.10

ตารางที่ 4.10 สถิติจาก `stats.gs` เทียบกับ Python และ Dashboard (จำนวนคดี / ของกลาง)

ช่วงเวลา	Python	`stats.gs` (กลุ่มประเภท, กลุ่มผู้ตรวจ)	Dashboard	ผล
ปี 2565 (ม.ค.–ธ.ค.)	138 / 621	138 / 621 (7, 6)	138 / 621	ตรงทุกค่า
ปี 2568 (ม.ค.–ธ.ค.)	154 / 789	154 / 789 (7, 6)	154 / 789	ตรงทุกค่า
ธ.ค. 2565	13 / 44	13 / 44 (6, 6)	13 / 44	ตรงทุกค่า
ม.ค. 2566 (ข้ามปี)	14 / 54	14 / 54 (5, 6)	14 / 54	ตรงทุกค่า
ก.ย. 2568	12 / 80	12 / 80 (5, 5)	12 / 80	ตรงทุกค่า
ต.ค. 2568 (ข้ามปีงบประมาณ)	12 / 62	12 / 62 (5, 5)	12 / 62	ตรงทุกค่า
ต.ค. 2569 (เดือนล่าสุด)	14 / 57	14 / 57 (6, 6)	14 / 57	ตรงทุกค่า

จำนวนกลุ่มในวงเล็บนับกลุ่ม “ไม่ระบุ” ด้วยเมื่อช่วงเวลานั้นมีค่าว่าง คำว่า “ตรงทุกค่า” หมายถึงจำนวนคดีของทุกประเภทและทุกคู่ผู้ตรวจ × ประเภทตรงกับ Python ด้วย ไม่ใช่เฉพาะยอดรวม

4.4.2 การคำนวณช่วงเวลาของรายงาน

รายงานอัตโนมัติคำนวณ “เดือนก่อนหน้า” และ “ปีก่อนหน้า” จากเวลาประเทศไทย (หัวข้อ 3.4.6) กรณีทดสอบเลือกเวลา

ที่วันที่ตามเวลาไทยต่างจากวันที่ตามเวลา UTC เช่น เวลา 00:05 น. ของวันที่ 1 มกราคมตามเวลาไทยยังเป็นวันที่ 31 ธันวาคมตามเวลา UTC และเลือกเดือนกุมภาพันธ์ทั้งปีปกติและปีอธิกสุรทิน ผลแสดงในตารางที่ 4.11

ตารางที่ 4.11 ผลการทดสอบการคำนวณช่วงเวลาของรายงาน

เวลาไทยที่จำลอง	ช่วงที่ต้องสรุป	ผลที่คาด	ผล
1 ม.ค. 2570 เวลา 00:05 น.	เดือนก่อนหน้า	1–31 ธ.ค. 2569	ผ่าน
1 ม.ค. 2570 เวลา 00:05 น.	ปีก่อนหน้า	1 ม.ค. – 31 ธ.ค. 2569	ผ่าน
1 ต.ค. 2569 เวลา 08:00 น.	เดือนก่อนหน้า	1–30 ก.ย. 2569	ผ่าน
1 มี.ค. 2569 เวลา 06:59 น.	เดือนก่อนหน้า (ปีปกติ)	1–28 ก.พ. 2569	ผ่าน
1 มี.ค. 2567 เวลา 01:00 น.	เดือนก่อนหน้า (ปีอธิกสุรทิน)	1–29 ก.พ. 2567	ผ่าน

4.4.3 คดีค้าง การค้นคดี และความตรงกันระหว่าง Dashboard กับแชตบอต

กรณีที่เหลือของ `Stats.gs` และกรณีที่เทียบตัวเลขระหว่างหน้า Dashboard `Stats.gs` และข้อความของแชตบอต แสดงในตารางที่ 4.12

ตารางที่ 4.12 ผลการทดสอบคดีค้าง การค้นคดี และความตรงกันระหว่างส่วนต่าง ๆ ของระบบ

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
คดีค้างจาก `Stats.pendingCases`	39 คดี (Python)	39 คดี กำลังดำเนินการ 22 เสร็จแล้ว รอรับผล 17	ผ่าน
ค้นคดี 1/2565	พบ สถานะรับผลไปแล้ว	พบ 1 รายการ ผลที่คืนไม่มีชื่อผู้ตรวจ	ผ่าน
ค้นคดีด้วยเลขไทย เลขที่มี 0 นำหน้า และช่องว่าง	พบคดีเดียวกัน	ตรงตามคาด	ผ่าน
ค้นคดีที่ไม่มี (9999/2569) และข้อความว่าง	ไม่พบ	ตรงตามคาด	ผ่าน
แถวที่ช่องสถานะว่างหรือสะกดต่างไป	นับเป็น “กำลังดำเนินการ” และบอกจำนวนสถานะว่าง	ตรงตามคาด	ผ่าน
ฟังก์ชันใน `Stats` เรียกจากหน้าเว็บไม่ได้	ไม่มีฟังก์ชันระดับบนสุดชื่อ `summarize` หรือ `findCase`	ตรงตามคาด	ผ่าน
ตารางผู้ตรวจ × ประเภทคดี เดือน ต.ค. 2569 บน Dashboard เทียบกับ `Stats.gs`	ทุกเซลล์ตรงกัน	6 แถวผู้ตรวจ (รวม “ไม่ระบุผู้ตรวจ”) 48 เซลล์ตรงทั้งหมด	ผ่าน
คดีที่ยังไม่รับผลบนหน้าระยะเวลาและงานค้างเทียบกับ	39 คดี	ตรงตามคาด	ผ่าน

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
`Stats.pendingCases`			
คดีค้างแยก 2 กลุ่มบน Dashboard เทียบกับข้อความของแชตบอต	22 / 17	Dashboard 22 / 17 เท่ากับแชตบอต	ผ่าน

กรณีแถวที่ช่องสถานะว่างทดสอบโดยแก้สถานะของ 3 แถวในหน่วยความจำ เพราะข้อมูลจำลองไม่มีแถวลักษณะนี้ ผลในตารางที่ 4.10 ถึงตารางที่ 4.12 แสดงว่าตัวเลขชุดเดียวกันได้ค่าเท่ากันทั้ง 4 ทาง คือ Python, `Stats.gs`, หน้า Dashboard และข้อความของแชตบอต (ข้อความสรุปเทียบในหัวข้อ 4.5.4) ซึ่งเป็นไปตามข้อกำหนดในหัวข้อ 3.6.4 ว่าตัวเลขในรายงานต้องตรงกับ Dashboard

4.5 การทดสอบแชตบอต LINE

การทดสอบแชตบอตสร้างเหตุการณ์ webhook ตามรูปแบบในเอกสารของ LINE [37] แล้วส่งให้ `doPost` ในรูปออบเจ็กต์เดียวกับที่ Apps Script ส่งให้ (พารามิเตอร์ `token` ใน URL และเนื้อหาคำขอ) จากนั้นตรวจคำขอที่ระบบส่งออกผ่านตัวจำลองของ UrlFetchApp ว่ามีการตอบกลับ (reply) กี่ครั้ง มีการเรียก Gemini หรือไม่ และเนื้อหาที่ส่งคืออะไร กลุ่มแชตบอตมี 22 กรณี ในจำนวนนี้ 1 กรณี (ฟังก์ชันผู้ดูแล) อยู่ในตารางที่ 4.9 แล้ว ส่วนกลุ่มสรุปอัตโนมัติมี 11 กรณี ในจำนวนนี้ 1 กรณี (การเรียก `dailySummaryJob` โดยตรง) อยู่ในตารางที่ 4.9 เช่นกัน

4.5.1 การเรียกบอต การเงียบ และข้อความตอบ

ตารางที่ 4.13 แสดงผลการทดสอบว่าบอตตอบเมื่อถูกเรียกตามตารางที่ 3.10 และเงียบเมื่อไม่ได้ถูกเรียก

ตารางที่ 4.13 ผลการทดสอบการเรียกบอต การเงียบ และข้อความตอบ

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
พิมพ์เลข บก. 1/2565 จากกลุ่มที่อนุญาต	ตอบ 1 ครั้ง มีสถานะ ประเภท ของกลาง และวันที่ 3 รายการ ไม่มีชื่อผู้ตรวจ สถานี จังหวัด เลขรายงาน ไม่เรียก Gemini	ตอบตามตัวอย่างใน หัวข้อ 3.5.3	ผ่าน
เลข บก. แบบเลขไทย “คดี ๑/๒๕๖๕ ถึงไหนแล้ว”	ตอบเหมือนเลขอารบิก	ข้อความตอบเหมือนกันทุกตัวอักษร	ผ่าน
เลข บก. ที่ไม่มีในข้อมูล (9999/2569)	แจ้งว่าไม่พบ พร้อมรูปแบบเลขที่ถูกต้อง	ตรงตามคาด	ผ่าน
หลายเลขในข้อความเดียว “บอต 1/2565, 2/2565”	ตอบทั้ง 2 คดีในข้อความเดียว	ตรงตามคาด	ผ่าน
คำสั่ง “คดีค้าง”	แยกกำลังดำเนินการ 22 และเสร็จแล้ว รอรับผล 17 พร้อม 3 คดีที่ค้างนานที่สุดต่อกลุ่ม ตรงกับ Python	ตรงทุกค่า ข้อความยาว 293 ตัวอักษร	ผ่าน
คำสั่งคดีค้างแบบอื่น (“มีคดีค้างกี่คดี”, “บอ	ตอบสรุปคดีค้าง ไม่เรียก Gemini	ตรงตามคาด	ผ่าน

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
ต งานที่ยังไม่รับผล”, “/ค้าง”, “สรุปคดีค้าง?”)			
กล่าวถึงบอต (mention) ตามด้วย “ขอดูคดีค้าง”	ถือว่าเรียกบอต	ตรงตามคาด	ผ่าน
ข้อความสนทนาทั่วไป 9 ข้อความ การกล่าวถึงผู้ใช้อื่น และรูปภาพ	เงียบ ไม่มีคำขอออกจากระบบเลย	ไม่มีคำขอทั้ง 11 ข้อความ	ผ่าน
“บอต” เพียงคำเดียว “บอต วิธีใช้” และ “bot help”	ตอบวิธีใช้ ไม่เรียก Gemini	ตรงตามคาด	ผ่าน

ข้อความสนทนาทั่วไปที่ใช้ทดสอบการเงียบ ได้แก่ “สวัสดีครับ” “บอทนี่ใช้ดีนะ” “ส่งของกลางคดี 123/2568 ไปแล้วนะ” “พรุ่งนี้ประชุม 9 โมง” “งานค้างเยอะจัง” “ใครเห็นคดี 45/2569 บ้าง” “โอเค” “บอตตอง” และ “5555” ข้อความเหล่านี้มีคำที่ใกล้เคียงคำสั่ง เช่น เลขคดีในประโยค คำว่า “ค้าง” และคำที่ขึ้นต้นด้วย “บอต” แต่ไม่ใช่คำเรียก ผลคือบอตไม่ตอบ ไม่อ่าน Sheet และไม่เรียก Gemini ทุกข้อความ

4.5.2 การส่งข้อความให้ Gemini

ตารางที่ 4.14 แสดงผลการทดสอบส่วนที่เกี่ยวข้องกับ Gemini ซึ่งใช้ตัวจำลองแทน Gemini API จริง

ตารางที่ 4.14 ผลการทดสอบการส่งข้อความให้ Gemini

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
เรียกบอตด้วยข้อความที่กฎไม่เข้าใจ ซึ่งมีชื่อ หมายเลขโทรศัพท์ อีเมล เลขรายงาน ชื่อสถานี เลขคดี และ URL	ส่งเฉพาะข้อความที่ตัดข้อมูลแล้ว ไม่มีข้อมูลจาก Sheet ได้เจตนาแล้วตอบด้วยโค้ด	ข้อความที่ส่งตรงกับตารางที่ 3.11	ผ่าน
Gemini ตอบเจตนา “สถานะคดี” พร้อมเลขคดีมาด้วย	ไม่ใช้เลขคดีจาก Gemini ขอให้ผู้ใช้พิมพ์เลข บก. และไม่ค้น Sheet	ตรงตามคาด	ผ่าน
Gemini ใช้งานไม่ได้: HTTP 429, HTTP 500, หมดเวลา, JSON ไม่ถูกต้อง, เจตนาที่ไม่รู้จัก (5 แบบ)	ตอบวิธีใช้แบบสั้นทุกแบบ	ตรงตามคาด	ผ่าน
ไม่มี `GEMINI_API_KEY`, เกินโควตาที่ตั้ง (`GEMINI_MAX_PER_HOUR` = 1), ชื่อรุ่นในรูปแบบที่ไม่ถูกต้อง	ไม่เรียก Gemini ตอบวิธีใช้	ตรงตามคาด	ผ่าน

ในกรณีแรกของตารางที่ 4.14 ชุดทดสอบไม่ได้ตรวจเพียงข้อความผลลัพธ์ แต่ตรวจเนื้อหาคำขอทั้งหมดที่ส่งไปยัง Gemini ว่า (1) ข้อความคำถามและคำสั่งระบบไม่มีตัวเลขทั้งเลขอารบิกและเลขไทย (2) ไม่มีชื่อ “สมชาย” “ใจดี” อีเมล โดเมน คำว่า “สภ.”

ชื่อสถานี URL รหัสกลุ่ม `userId` ของผู้ส่ง และ reply token (3) ไม่มีข้อความใดที่ยาว 3 ตัวอักษรขึ้นไปจากแท็บข้อมูลคดี และ (4) API key ส่งในส่วนหัว `x-goog-api-key` ไม่อยู่ใน URL นอกจากนี้ ตัวจำลองของ Gemini ตอบเลขคดี 999/2568 ปนมากับเจตนา และข้อความตอบของบอตไม่มีเลขนี้ ยืนยันว่าระบบไม่นำค่าอื่นจาก Gemini มาใช้

4.5.3 การจำกัดกลุ่มและการป้องกัน webhook

ตารางที่ 4.15 แสดงผลการทดสอบมาตรการทดแทนการตรวจลายมือชื่อ webhook ในตารางที่ 3.12 และโหมดตั้งค่า

ตารางที่ 4.15 ผลการทดสอบการจำกัดกลุ่มและการป้องกัน webhook

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
ข้อความจากกลุ่มอื่น แชตส่วนตัว และห้องแชต	เงียบ	ตรงตามคาด	ผ่าน
ยังไม่ตั้ง `ALLOWED_GROUP_ID`	เงียบทุกกลุ่ม	ตรงตามคาด	ผ่าน
รหัสลับใน URL ผิด ขาดไป 1 ตัว ไม่มี ยังไม่ตั้ง `WEBHOOK_TOKEN` หรือสั้นเกินไป (5 แบบ)	ทิ้งคำขอ ไม่ตอบ ไม่อ่าน Sheet คำตอบ HTTP เป็น `OK` เหมือนกรณีปกติ	ตรงตามคาด	ผ่าน
คำขอจากปุ่ม Verify (ไม่มีเหตุการณ์) และเนื้อหาที่ไม่ใช่ JSON	ตอบ `OK` ไม่มีคำขอออก	ตรงตามคาด	ผ่าน
เหตุการณ์ซ้ำ (`webhookEventId` เดิม) การส่งซ้ำ (redelivery) และเหตุการณ์เก่า 10 นาที	ตอบครั้งเดียว ไม่ตอบการส่งซ้ำและเหตุการณ์เก่า	ตรงตามคาด	ผ่าน

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
โหมดตั้งค่า: เชิญบอตเข้ากลุ่มใหม่	บันทึกรหัสกลุ่มใน Script Properties ไม่ตอบในแชต และ `allowSeenGroup` ตั้ง `ALLOWED_GROUP_ID` แล้วปิดโหมด	ตรงตามคาด	ผ่าน
โหมดตั้งค่าหมดเวลาหรือไม่ได้เปิด	ไม่บันทึกรหัสกลุ่ม	ตรงตามคาด	ผ่าน
`makeWebhookToken`	สร้างรหัสสุ่ม 64 ตัวอักษร ไม่พิมพ์ค่าในบันทึกการทำงาน รันซ้ำไม่เปลี่ยนรหัสเดิม	ตรงตามคาด	ผ่าน

4.5.4 รายงานสรุปอัตโนมัติ

กลุ่มนี้ทดสอบ `Summary.gs` โดยเรียก `dailySummaryJob` พร้อมวันเวลาจำลอง และแปลงข้อความสรุปกลับเป็นตัวเลขเพื่อเทียบกับ Python ผล 10 กรณีแสดงในตารางที่ 4.16 (อีก 1 กรณีอยู่ในตารางที่ 4.9)

ตารางที่ 4.16 ผลการทดสอบรายงานสรุปอัตโนมัติ

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
ข้อความสรุปเดือน ต.ค. 2569 (`previewSummary`)	ตัวเลขทุกค่าตรงกับ Python และ `Stats.summarize` ไม่ส่งจริง ไม่มีเลขคดีและชื่อสถานี	คดี 14 ของกลาง 57 ชิ้น 6 กลุ่มประเภท 6 กลุ่มผู้ตรวจ ยาว 576 ตัวอักษร	ผ่าน
ข้อความสรุปปี 2568 (`previewYearSummary`)	เช่นเดียวกับข้างต้น	คดี 154 ของกลาง 789 ชิ้น 7 กลุ่ม	ผ่าน

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
		ประเภท 6 กลุ่มผู้ตรวจ ยาว 938 ตัวอักษร	
รัน `installTriggers` 3 ครั้ง	มีตัวตั้งเวลารายวัน 1 ตัว เวลา 08:00 น. เขตเวลา Asia/Bangkok	ตรงตามคาด	ผ่าน
วันที่ 1 พ.ย. 2569 เวลา 08:10 น.	push 1 คำขอ 1 ข้อความ (สรุป ต.ค. 2569) ไปยังกลุ่มที่อนุญาต พร้อม `X-Line-Retry-Key`	ส่งงวด 2026-10	ผ่าน
ตัวตั้งเวลาทำงานซ้ำในวันเดียวกัน วันที่ 2–3 และวันที่ 15	ไม่ส่งซ้ำ	ตรงตามคาด	ผ่าน
ยังไม่ตั้ง `PUSH_ENABLED` เป็น `yes`	ไม่ส่ง บันทึกเฉพาะตัวอย่างข้อความ ไม่บันทึกว่าส่งแล้ว	ตรงตามคาด	ผ่าน
วันที่ 1 ม.ค. 2569 เวลา 08:10 น.	push 1 คำขอ 2 ข้อความ (สรุป ธ.ค. 2568 และสรุปปี 2568) ตรงกับ Python	ส่งงวด 2025-12 และ 2025 ในคำขอเดียว	ผ่าน
LINE ตอบ HTTP 500 ในวันที่ 1 และตอบ 409	วันที่ 1 ไม่บันทึกว่าส่งแล้ว วันที่ 2 ส่งงวดเดิมด้วย retry key เดิม รหัส 409 ถือว่าส่งแล้ว และเลยวันที่ 3 ไม่ส่งย้อนหลัง	ตรงตามคาด	ผ่าน
ความยาวข้อความ	ทุกข้อความสั้นกว่า	576, 938, 535,	ผ่าน

กรณีทดสอบ	ผลที่คาด	ผลจริง	ผล
	2,000 ตัวอักษร	938 และ 293 ตัวอักษร	
ข้อความยาวเกิน 5,000 ตัวอักษร	ถูกตัดให้ไม่เกิน 5,000 ตัวอักษรพร้อมหมายเหตุให้ดูต่อใน Dashboard	ตรงตามคาด	ผ่าน

ความยาว 5 ค่าในตารางที่ 4.16 คือ สรุปเดือน ต.ค. 2569, สรุปปี 2568, สรุปเดือน ธ.ค. 2568 และสรุปปี 2568 ที่ส่งในวันที่ 1 มกราคม และคำตอบคดีค้าง ตามลำดับ นอกจากนี้ ชุดทดสอบสร้างข้อมูลกรณีหนักที่สุดที่คาดไว้ คือ ผู้ตรวจ 5 คนที่มีชื่อยาวและกลุ่ม “ไม่ระบุผู้ตรวจ” รวม 6 กลุ่ม คูณประเภทคดี 7 กลุ่มครบทุกคู่ และจำนวนของกลางหลักหมื่น ข้อความสรุปยาว 1,233 ตัวอักษร ยังต่ำกว่าเพดานที่ระบบตัดไว้ 5,000 ตัวอักษร และการส่ง 2 ข้อความในวันที่ 1 มกราคมยังอยู่ในขีดจำกัด 5 ข้อความต่อคำขอของ LINE [18]

4.6 การทดสอบด้าน PDPA

ข้อกำหนดด้านการคุ้มครองข้อมูลส่วนบุคคลจากหัวข้อ 3.7 ถูกตรวจผ่านกรณีทดสอบในหัวข้อก่อนหน้า ตารางที่ 4.17 รวบรวมข้อกำหนดแต่ละข้อกับหลักฐานจากการทดสอบ

ตารางที่ 4.17 รายการตรวจด้าน PDPA และผลการทดสอบ

ข้อกำหนด	หลักฐานจากการทดสอบ	ผล
ใช้ข้อมูลจำลองเท่านั้น ไม่มี DNA profile	ค่าทุกค่าในไฟล์จำลองเป็นค่าสมมติ (ตารางที่ 4.4) ภาพหน้าจอจากการทดสอบเก็บในโฟลเดอร์ผลลัพธ์ชั่วคราว	ผ่าน

ข้อกำหนด	หลักฐานจากการทดสอบ	ผล
	ที่ไม่ถูกนำเข้า repository	
ระบบไม่พึ่งคอลัมน์ User ID (LINE)	ข้อมูลจำลองไม่มีคอลัมน์นี้ และเมื่อแทรกคอลัมน์นี้เข้าไป ระบบก็ไม่ส่งออก (ตารางที่ 4.5)	ผ่าน
ส่งข้อมูลไปยังเบราว์เซอร์เท่าที่จำเป็น	`getBook("cases")` ส่ง 14 คอลัมน์ `checkSetup` แสดงเฉพาะจำนวนแถว (ตารางที่ 4.5)	ผ่าน
ผู้ไม่มีสิทธิ์เข้าไม่ถึงข้อมูล	ปฏิเสธทุกกรณีที่เงื่อนไขไม่ครบ รวมการเปิดผ่านลิงก์ของแชตบอต (ตารางที่ 4.8 และ 4.9)	ผ่าน
คำตอบสถานะคดีมีข้อมูลน้อยที่สุด	ไม่มีชื่อผู้ตรวจ สถานี จังหวัด และเลขรายงาน (ตารางที่ 4.13)	ผ่าน
รายงานสรุปไม่มีเลขคดี	ข้อความสรุปไม่มีรูปแบบเลขคดีและชื่อสถานี (ตารางที่ 4.16)	ผ่าน
ไม่ส่งข้อมูลคดีและข้อมูลระบุตัวบุคคลให้ Gemini	เนื้อหาคำขอไม่มีตัวเลข ชื่อ อีเมล URL รหัสกลุ่ม `userId` และค่าจาก Sheet (หัวข้อ 4.5.2)	ผ่าน
ข้อความสนทนาทั่วไปไม่ออกนอกระบบ	ไม่มีคำขอใดออกจากระบบสำหรับข้อความที่ไม่ได้เรียกบอต (ตารางที่ 4.13)	ผ่าน
คำตอบ HTTP ของ webhook ไม่มีข้อมูล	`doPost` ตอบ `OK` ทุกกรณี (ตารางที่ 4.9)	ผ่าน
หน้าเว็บไม่ส่งคำขอออกนอกระบบ	คำขอที่ไม่ได้คาดไว้ 0 รายการ (ตารางที่ 4.6)	ผ่าน
ค่าความลับไม่รั่วในบันทึกการทำงานหรือ URL	รหัสลับของ webhook ไม่ถูกพิมพ์ในบันทึกการทำงาน (ตารางที่ 4.15) API key ของ Gemini อยู่ในส่วนหัวของ	ผ่าน

ข้อกำหนด	หลักฐานจากการทดสอบ	ผล
	คำขอ (หัวข้อ 4.5.2)	
ส่งข้อความเข้ากลุ่มจริงเมื่อเจ้าของโครงงานอนุมัติแล้วเท่านั้น	ไม่ส่ง push เมื่อยังไม่ตั้ง PUSH_ENABLED (ตารางที่ 4.16)	ผ่าน

ข้อกำหนดเรื่องการเก็บค่าความลับใน Script Properties แทนการเขียนในโค้ด (หัวข้อ 3.7 ข้อ 3) ไม่มีกรณีทดสอบอัตโนมัติที่ค้นหาค่าความลับในโค้ดโดยตรง ชุดทดสอบเพียงป้อนค่าความลับทั้งหมดผ่านตัวจำลองของ PropertiesService ซึ่งแสดงว่าโค้ดอ่านค่าจากที่นั้น

4.7 สรุปผลการทดสอบ

4.7.1 ผลการทดสอบรวม

จำนวนกรณีทดสอบและผลแยกตามหมวดแสดงในตารางที่ 4.18 หมวดในตารางตรงกับกลุ่มที่ชุดทดสอบพิมพ์ออกมา

ตารางที่ 4.18 สรุปจำนวนกรณีทดสอบแยกตามหมวด

หมวด	แหล่งผลทดสอบ	หัวข้อในบทนี้	จำนวนกรณี	ผ่าน
ข้อมูลจำลองและ Dashboard ต้นแบบ	`01-mock-data.md`	4.2.1	18	18
การควบคุมสิทธิ์	`run.js`	4.3.1	11	11
deployment ของแชตบอต	`run.js`	4.3.2	3	3
การอ่านข้อมูลฝั่งเซิร์ฟเวอร์	`run.js`	4.2.2	8	8

หมวด	แหล่งผลทดสอบ	หัวข้อในบทนี้	จำนวนกรณี	ผ่าน
สถิติ (Stats.gs)	run.js	4.4	18	18
แชตบอต LINE	run.js	4.5.1–4.5.3, 4.3.2	22	22
รายงานสรุปอัตโนมัติ	run.js	4.5.4, 4.3.2	11	11
หน้าเว็บ (Playwright)	run.js	4.2.3, 4.4.3	23	23
รวมชุดทดสอบอัตโนมัติ run.js			96	96
รวมทั้งหมด			114	114

4.7.2 วิเคราะห์ผล

ผลการทดสอบผ่านครบทุกกรณี และให้ข้อสรุปตามวัตถุประสงค์ข้อที่ 4 ในหัวข้อ 1.2 ดังนี้

1) **ความถูกต้องของการคำนวณ** ตัวเลขจาก Stats.gs หน้า Dashboard และข้อความของแชตบอต ตรงกับค่าที่คำนวณด้วย Python ทุกค่าที่ทดสอบ ทั้งยอดรวม การแยกประเภท และการแยกผู้ตรวจตามประเภท (ตารางที่ 4.7 และตารางที่ 4.10) ช่วงเวลาที่ทดสอบรวมกรณีข้ามปี ข้ามปีงบประมาณ และเวลาที่วันที่ตามเวลาไทยต่างจากเวลา UTC ซึ่งเป็นจุดที่การคำนวณวันที่มักผิดพลาด ผลนี้สนับสนุนการออกแบบให้แชตบอตและรายงานอัตโนมัติใช้ฟังก์ชันคำนวณชุดเดียวกัน (หัวข้อ 3.6.4)

2) **การควบคุมสิทธิ์** ตรรกะการตรวจสิทธิ์ปฏิเสธทุกกรณีที่เงื่อนไขไม่ครบ ทั้งที่หน้าแรกและเมื่อเรียกฟังก์ชันข้อมูลโดยตรง และลิงก์ของแชตบอตเปิดข้อมูล Dashboard ไม่ได้ (ตารางที่ 4.8 และ 4.9)
3) **พฤติกรรมของแชตบอต** บอตเงียบเมื่อไม่ได้ถูกเรียก ตอบเฉพาะกลุ่มที่อนุญาต ไม่ตอบซ้ำ และยังตอบได้เมื่อ Gemini ใช้งานไม่ได้ (ตารางที่ 4.13 ถึง 4.15) รายงานอัตโนมัติไม่ส่งซ้ำ ส่งตามหลังได้เมื่อ LINE ขัดข้อง และวันที่ 1 มกราคมรวม 2 ข้อความในคำขอเดียว (ตารางที่ 4.16)
4) **การคุ้มครองข้อมูลส่วนบุคคล** ข้อมูลที่ส่งออกจากระบบทุกทาง ได้แก่ ไปยังเบราว์เซอร์ ไปยังกลุ่ม LINE และไปยัง Gemini มีเฉพาะข้อมูลที่ออกแบบไว้ (ตารางที่ 4.17)

ข้อจำกัดของผลการทดสอบมีดังนี้

1) **ทดสอบกับบริการจำลอง** ผลยืนยันว่าโค้ดทำงานถูกต้องเมื่อบริการของ Google และ LINE ทำงานตามที่ตัวจำลองกำหนด ถ้าพฤติกรรมจริงต่างจากที่จำลอง เช่น ค่าที่ `Session.getActiveUser()` คืนในแต่ละแบบ deploy [12] หรือการตอบแบบเปลี่ยนเส้นทางของ Apps Script ต่อปุ่ม Verify [45] ผลจริงอาจต่างไป จึงต้องทดสอบซ้ำหลังนำขึ้นใช้งาน (หัวข้อ 4.8)
2) **ไม่ได้วัดความแม่นยำของการแยกเจตนา** กฎในโค้ดทดสอบด้วยข้อความตัวอย่างจำนวนน้อย และ Gemini ถูกแทนด้วยตัวจำลองที่ตอบตามที่กำหนด ผล

จึงยังไม่บอกว่าแชตบอตเข้าใจคำถามจริงของเจ้าหน้าที่ได้ดีเพียงใด

3) **การตัดข้อมูลก่อนส่ง Gemini ทดสอบด้วยข้อความตัวอย่างข้อความเดียว** ผลยืนยันกฎที่เขียนไว้ แต่ไม่ครอบคลุมทุกรูปแบบที่ผู้ใช้อาจพิมพ์ เช่น ชื่อบุคคลที่ไม่มีคำนำหน้า (หัวข้อ 3.5.2)
4) **ค่าอ้างอิงเขียนโดยทีมเดียวกัน** โค้ด Python ใช้คนละภาษาและไม่ใช้โค้ดของระบบ จึงจับข้อผิดพลาดในการเขียนโปรแกรมได้ แต่ทั้งสองฝั่งตีความกติกาการนับจากข้อกำหนดชุดเดียวกัน ถ้าข้อกำหนดผิด ทั้งสองฝั่งจะผิดเหมือนกัน
5) **ข้อมูลจำลองสะอาดกว่าข้อมูลจริง** ข้อมูลจำลองสร้างให้วันที่สอดคล้องกับสถานะทุกแถว ส่วนกรณีที่ข้อมูลไม่สมบูรณ์ เช่น ช่องสถานะว่าง ทดสอบด้วยการแก้ข้อมูลในหน่วยความจำเพียงไม่กี่แถว ข้อมูลจริงอาจมีรูปแบบที่ไม่ได้คาดไว้
6) **ไม่ได้ทดสอบด้านประสิทธิภาพและการใช้งาน** ชุดทดสอบไม่ได้วัดเวลาตอบ ไม่ได้ทดสอบโควตาของ Apps Script และ LINE ภายใต้การใช้งานจริง และยังไม่มีผลการประเมินจากผู้ใช้

4.8 การทดสอบกับการใช้งานจริง

[รอใส่: ผลการทดสอบหลัง deploy จริง — การเปิด Dashboard ด้วยบัญชีจริง, ปุ่ม Verify webhook/302, การตอบในกลุ่ม LINE จริง, ภาพหน้าจอ]

[รอใส่: ผลแบบประเมินความพึงพอใจของเจ้าหน้าที่ ถ้ามี]

เอกสารอ้างอิง

[1] J. M. Butler, *Fundamentals of Forensic DNA Typing*. Amsterdam; Boston: Academic Press/Elsevier, 2010.

[2] Technical Working Group on Biological Evidence Preservation (S. Ballou *et al.*), “The biological evidence preservation handbook: Best practices for evidence handlers,” National Institute of Standards and Technology, Gaithersburg, MD, USA, NISTIR 7928, Apr. 2013. [Online]. Available: https://doi.org/10.6028/NIST.IR.7928 (เข้าถึงเมื่อ 3 พ.ย. 2569)

[3] National Institute of Justice, “National best practices for improving DNA laboratory process efficiency,” U.S. Department of Justice, Office of Justice Programs, Washington, DC, USA, NCJ 304051, May 2022. [Online]. Available: https://www.ojp.gov/pdffiles1/nij/304051.pdf (เข้าถึงเมื่อ 3 พ.ย. 2569)

[4] S. Few, *Information Dashboard Design: The Effective Visual Communication of Data*. Sebastopol, CA: O’Reilly Media, 2006.

[5] Google for Developers. Apps Script overview [Online]. Available:

https://developers.google.com/apps-script/overview (เข้าถึงเมื่อ 3 พ.ย. 2569)

[6] LINE Developers. Messaging API overview [Online]. Available: https://developers.line.biz/en/docs/messaging-api/overview/ (เข้าถึงเมื่อ 3 พ.ย. 2569)

[7] “พระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562,” *ราชกิจจานุเบกษา*, เล่ม 136, ตอนที่ 69 ก, 27 พ.ค. 2562.

[8] Google. Gemini API additional terms of service (มีผลวันที่ 23 มี.ค. 2026) [Online]. Available: https://ai.google.dev/gemini-api/terms (เข้าถึงเมื่อ 3 พ.ย. 2569)

[9] Google. How to use Google Sheets, Google Docs Editors Help [Online]. Available: https://support.google.com/docs/answer/6000292 (เข้าถึงเมื่อ 3 พ.ย. 2569)

[10] Google for Developers. Class SpreadsheetApp, Apps Script reference [Online]. Available: https://developers.google.com/apps-script/reference/spreadsheet/spreadsheet-app (เข้าถึงเมื่อ 3 พ.ย. 2569)

[11] Google for Developers. Class HtmlService, Apps Script reference [Online]. Available: https://developers.google.com/apps-script/reference/html/html-service (เข้าถึงเมื่อ 3 พ.ย. 2569)

[12] Google for Developers. Class Session, Apps Script reference [Online]. Available: https://developers.google.com/apps-script/reference/base/session (เข้าถึงเมื่อ 3 พ.ย. 2569)

[13] Google for Developers. Class PropertiesService, Apps Script reference [Online]. Available: https://developers.google.com/apps-script/reference/properties/properties-service (เข้าถึงเมื่อ 3 พ.ย. 2569)

[14] Google for Developers. Class UrlFetchApp, Apps Script reference [Online]. Available: https://developers.google.com/apps-script/reference/url-fetch/url-fetch-app (เข้าถึงเมื่อ 3 พ.ย. 2569)

[15] Google for Developers. Web apps, Apps Script guides [Online]. Available: https://developers.google.com/apps-script/guides/web (เข้าถึงเมื่อ 3 พ.ย. 2569)

[16] Google for Developers. Installable triggers, Apps Script guides [Online]. Available: https://developers.google.com/apps-script/guides/triggers/installable (เข้าถึงเมื่อ 3 พ.ย. 2569)

[17] Google for Developers. Quotas for Google services, Apps Script guides [Online]. Available: https://developers.google.com/apps-script/guides/services/quotas (เข้าถึงเมื่อ 3 พ.ย. 2569)

[18] LINE Developers. Send messages, Messaging API documentation [Online]. Available: https://developers.line.biz/en/docs/messaging-api/sending-messages/ (เข้าถึงเมื่อ 3 พ.ย. 2569)

[19] LINE Developers. Messaging API pricing [Online]. Available: https://developers.line.biz/en/docs/messaging-api/pricing/ (เข้าถึงเมื่อ 3 พ.ย. 2569)

[20] LINE for Business. แจ้งปรับเปลี่ยนแพ็กเกจรายเดือน LINE Official Account (18 มิ.ย. 2567) [Online]. Available: https://lineforbusiness.com/th/news/20240619_1 (เข้าถึงเมื่อ 3 พ.ย. 2569)

[21] LINE Developers. Group chats and multi-person chats, Messaging API documentation [Online]. Available: https://developers.line.biz/en/docs/messaging-api/group-chats/ (เข้าถึงเมื่อ 3 พ.ย. 2569)

[22] LINE Developers. Verify webhook signature, Messaging API documentation [Online]. Available: https://developers.line.biz/en/docs/messaging-api/verify-webhook-signature/ (เข้าถึงเมื่อ 3 พ.ย. 2569)

[23] E. Adamopoulou and L. Moussiades, “Chatbots: History, technology, and applications,” *Machine Learning with Applications*, vol. 2, Art. no. 100006, Dec. 2020, doi: 10.1016/j.mlwa.2020.100006.

[24] A. Vaswani *et al.*, “Attention is all you need,” arXiv:1706.03762, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762 (เข้าถึงเมื่อ 3 พ.ย. 2569)

[25] T. B. Brown *et al.*, “Language models are few-shot learners,” arXiv:2005.14165, 2020. [Online]. Available: https://arxiv.org/abs/2005.14165 (เข้าถึงเมื่อ 3 พ.ย. 2569)

[26] Google. Text generation, Gemini API documentation [Online]. Available: https://ai.google.dev/gemini-api/docs/text-generation (เข้าถึงเมื่อ 3 พ.ย. 2569)

[27] Google. Structured outputs, Gemini API documentation [Online]. Available: https://ai.google.dev/gemini-api/docs/structured-output (เข้าถึงเมื่อ 3 พ.ย. 2569)

[28] Google. Rate limits, Gemini API documentation [Online]. Available: https://ai.google.dev/gemini-api/docs/rate-limits (เข้าถึงเมื่อ 3 พ.ย. 2569)

[29] กรมทรัพย์สินทางปัญญา. คู่มือการคุ้มครองข้อมูลส่วนบุคคล (PDPA Guideline) [Online]. Available: https://ipthailand.go.th/images/26881/PDPA_Guideline_v_1.pdf (เข้าถึงเมื่อ 3 พ.ย. 2569)

[30] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings*

of the IEEE, vol. 63, no. 9, pp. 1278–1308, Sep. 1975.

[31] National Institute of Standards and Technology. Least privilege, CSRC glossary [Online]. Available: https://csrc.nist.gov/glossary/term/least_privilege (เข้าถึงเมื่อ 3 พ.ย. 2569)

[32] สุภัคพงศ์ จินารัตน์, จุฑาทิพย์ เปล่งผิว และ นัศพ์ชาณัณ ชินปัญช์ธนะ, “การพัฒนาระบบรายงานข้อมูล และ Dashboard เพื่อสนับสนุนการตัดสินใจของผู้บริหาร กรณีศึกษา: สถาบันเวชศาสตร์สมเด็จพระสังฆราชญาณสังวรเพื่อผู้สูงอายุ,” *วารสารวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยสุโขทัยธรรมาธิราช*, ปีที่ 5, ฉบับที่ 2, หน้า 1–14, 2568.

[33] ชาญชัย ศุภอรรถกร และ ฟ้าพราว เฉียบแหลม, “การพัฒนาแอปพลิเคชันไลน์แชทบอทสำหรับประชาสัมพันธ์หลักสูตรเทคโนโลยีสารสนเทศและการสื่อสาร มหาวิทยาลัยอุบลราชธานี,” *Journal of Information Science and Technology*, vol. 15, no. 1, pp. 49–59, 2025.

[34] ชรัช แสวงเจริญ และ ชุติมา พิศาลย์, “การใช้ LINE Chatbot สำหรับการแจ้งซ่อมโสตทัศนูปกรณ์ งานเทคโนโลยีการศึกษา มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ,” *วารสารวิทยาศาสตร์และเทคโนโลยี มทร.สุวรรณภูมิ*, ปีที่ 9, ฉบับที่ 3, หน้า 76–84, 2568.

[35] ปัญญดา พินไธสงค์, ธรรมนูญ ปัญญาทิพย์ และ ปนัดดา โพธินาม, “การพัฒนาแอปพลิเคชันไลน์และแชทบอทระบบรับเข้าศึกษาต่อ มหาวิทยาลัยกาฬสินธุ์,” *วารสารวิชาการการ*

ประยุกต์ใช้เทคโนโลยีสารสนเทศ, ปีที่ 11, ฉบับที่ 1, หน้า 111–120, 2568.

[36] สิปปกร พงศ์เดโชธนบดี, พิชัย ระเวงวัน, ดนัย ศรีษะนอก และ วิจิตร เดชทะสอน, “การพัฒนาระบบแชทบอทเพื่อการสื่อสารและให้บริการข้อมูลแก่ลูกค้า: กรณีศึกษา บริษัท บัณฑิตเวิลด์ จำกัด,” *วารสารวิชาการและวิจัย มหาวิทยาลัยภาคตะวันออกเฉียงเหนือ*, ปีที่ 16, ฉบับที่ 2, หน้า 1–15, 2569.

[37] LINE Developers. Receive messages (webhook), Messaging API documentation [Online]. Available: https://developers.line.biz/en/docs/messaging-api/receiving-messages/ (เข้าถึงเมื่อ 3 พ.ย. 2569)

[38] Google for Developers. Class ClockTriggerBuilder, Apps Script reference [Online]. Available: https://developers.google.com/apps-script/reference/script/clock-trigger-builder (เข้าถึงเมื่อ 3 พ.ย. 2569)

[39] Google for Developers. HTML Service: Communicate with server functions, Apps Script guides [Online]. Available: https://developers.google.com/apps-script/guides/html/communication (เข้าถึงเมื่อ 3 พ.ย. 2569)

[40] Google for Developers. Class CacheService, Apps Script reference [Online]. Available: https://developers.google.com/apps-script/reference/cache/cache-service (เข้าถึงเมื่อ 3 พ.ย. 2569)

[41] Google for Developers. Class Cache, Apps Script reference [Online]. Available: https://developers.google.com/apps-script/reference/cache/cache (เข้าถึงเมื่อ 3 พ.ย. 2569)

[42] Google for Developers. Authorization scopes, Apps Script concepts [Online]. Available: https://developers.google.com/apps-script/concepts/scopes (เข้าถึงเมื่อ 3 พ.ย. 2569)

[43] Google for Developers. Class google.script.run (client-side API), Apps Script reference [Online]. Available: https://developers.google.com/apps-script/guides/html/reference/run (เข้าถึงเมื่อ 3 พ.ย. 2569)

[44] Google. Models, Gemini API documentation [Online]. Available: https://ai.google.dev/gemini-api/docs/models (เข้าถึงเมื่อ 3 พ.ย. 2569)

[45] LINE Developers. Verify webhook URL, Messaging API documentation [Online]. Available: https://developers.line.biz/en/docs/messaging-api/verify-webhook-url/ (เข้าถึงเมื่อ 3 พ.ย. 2569)

[46] Google for Developers. Class LockService, Apps Script reference [Online]. Available: https://developers.google.com/apps-script/reference/lock/lock-service (เข้าถึงเมื่อ 3 พ.ย. 2569)

[47] LINE Developers. Retry failed API requests, Messaging API documentation [Online]. Available: https://developers.line.biz/en/docs/messaging-api/retrying-api-request/ (เข้าถึงเมื่อ 3 พ.ย. 2569)

[48] E. Gazoni and C. Clark. openpyxl - A Python library to read/write Excel 2010 xlsx/xlsm files [Online]. Available: https://openpyxl.readthedocs.io/en/stable/ (เข้าถึงเมื่อ 3 พ.ย. 2569)

[49] OpenJS Foundation. VM (executing JavaScript), Node.js documentation [Online]. Available: https://nodejs.org/api/vm.html (เข้าถึงเมื่อ 3 พ.ย. 2569)

[50] Microsoft. Installation, Playwright documentation [Online]. Available: https://playwright.dev/docs/intro (เข้าถึงเมื่อ 3 พ.ย. 2569)

ภาคผนวก

ภาคผนวก ก

[รอใส่: รายละเอียดภาคผนวก ก เช่น คู่มือการติดตั้ง/การใช้งาน โค้ดหลัก หรือผลการทดสอบฉบับเต็ม — ใช้ข้อมูลจำลองเท่านั้น]

ภาคผนวก ข

[รอใส่: รายละเอียดภาคผนวก ข เช่น คู่มือการติดตั้ง/การใช้งาน โค้ดหลัก หรือผลการทดสอบฉบับเต็ม — ใช้ข้อมูลจำลองเท่านั้น]